01001011 0110
1101 0010 1110
0111 00101

CMP103 · Introduction to Computer Engineering and Career Planning · Hafta 5 · Vize hazırlığı

Veri Nasıl Saklanır ve İletilir?

İşaretli sayılar, sinyaller, hat kodlama ve sayısallaştırma

13 Ekim 2026 | Kaynak materyale dayalı öğretici çalışma notu

İçindekiler

Başlangıç 4

0.1 Bu not nasıl kullanılmalı? . . . 4
0.2 Dosya kodları ve etiketler. . . . 4
0.3 Gereken ön bilgiler . . . 4
0.4 Dersin haritası. . . . 5
0.5 Kutular ve etiketler ne anlama geliyor? . . . 5
0.6 Ders bilgileri ve yaptığım varsayımlar . . . 5
0.7 Çalışma planı önerisi . . . 6

1 Negatif Sayılar Bitlerle Nasıl Yazılır? 7

1.1 Eksi işareti için tuş yok: neden özel bir gösterim gerekiyor? ❖ Kaynak: D1 s. 4–6 . . . 7
1.2 İlk akla gelen fikir: işaret-büyüklük ❖ Kaynak: D1 s. 7–9 . . . 8
1.3 Bitleri ters çevir: 1'e tümleyen ❖ Kaynak: D1 s. 10–13 . . . 10
1.4 Ters çevir, 1 ekle: 2'ye tümleyen ❖ Kaynak: D1 s. 14–19 . . . 11
1.5 Üçünü yan yana koyalım ❖ Kaynak: D1 s. 18 . . . 14
1.6 Çıkarma = toplama: 2'ye tümleyenle aritmetik ❖ Kaynak: D1 s. 20–22 . . . 15
1.7 Taşma: bit genişliği sonucu taşıyamıyor ❖ Kaynak: D1 s. 20, 23 . . . 16
1.8 Bağlantılar . . . 18
1.9 Bölüm cevapları . . . 19

2 Veri ve Sinyal: Bitler Kabloda Nasıl Yolculuk Eder? 21

2.1 Haberleşmenin üç parçası ❖ Kaynak: D1 s. 27 . . . 21
2.2 Veri ≠ sinyal ❖ Kaynak: D1 s. 28 . . . 22
2.3 Analog mu, sayısal mı? ❖ Kaynak: D1 s. 29 . . . 22
2.4 Dört dönüşüm: veriden sinyale ❖ Kaynak: D1 s. 30–32 . . . 23
2.5 İki seviyeli ve çok seviyeli sinyaller ❖ Kaynak: D1 s. 33–34 . . . 25
2.6 Bağlantılar . . . 26
2.7 Bölüm cevapları . . . 28

3 Kanal Bozulmaları ve İletim Ortamları 29

3.1 Yolda ne olur? Üç bozulma ❖ Kaynak: D1 s. 36–38 . . . 29
3.2 Sinyal-gürültü oranı ❖ Kaynak: D1 s. 39 . . . 31
3.3 İletim ortamları ❖ Kaynak: D1 s. 40–43 . . . 32
3.4 Bağlantılar . . . 34
3.5 Bölüm cevapları . . . 35

4 Bitleri Tele Dökmek: Hat Kodlama 36

4.1 Kodlayıcı, kanal, çözücü ❖ Kaynak: D1 s. 45 . . . 36
4.2 İyi bir hat kodu neye benzer? ❖ Kaynak: D1 s. 46 . . . 37

4.3 Saatler neden anlaşmak zorunda? ❖ Kaynak: D1 s. 47–48 . . . 37
4.4 Hat kodlama ailesi ❖ Kaynak: D1 s. 49 . . . 38
4.5 Unipolar NRZ: en basit kod ❖ Kaynak: D1 s. 50 . . . 39
4.6 Polar NRZ-L ve NRZ-I ❖ Kaynak: D1 s. 51 . . . 39
4.7 Polar RZ: her bitte sıfıra dön ❖ Kaynak: D1 s. 52 . . . 41
4.8 Manchester ve fark Manchester ❖ Kaynak: D1 s. 53 . . . 42
4.9 Hepsini yan yana: karşılaştırma ❖ Kaynak: D1 s. 54 . . . 43
4.10 Hat kodlamanın ötesi: blok kodlama ve karıştırma ❖ Kaynak: D1 s. 55 . . . 44
4.11 Bağlantılar . . . 45
4.12 Bölüm cevapları . . . 46

5 Sesten Bitlere: PCM ve Delta Modülasyonu 47
5.1 Neden sayısallaştırıyoruz? ❖ Kaynak: D1 s. 57 . . . 47
5.2 PCM: üç adımlı zincir ❖ Kaynak: D1 s. 58 . . . 48
5.3 Adım 1: Örnekleme ❖ Kaynak: D1 s. 59–60 . . . 48
5.4 Adım 2 ve 3: Nicemleme ve kodlama ❖ Kaynak: D1 s. 61 . . . 50
5.5 PCM bit hızı: 64 kbps nereden geliyor? ❖ Kaynak: D1 s. 62 . . . 52
5.6 Alternatif: delta modülasyonu ❖ Kaynak: D1 s. 63–64 . . . 53
5.7 PCM ile delta modülasyonu karşılaştırması ❖ Kaynak: D1 s. 65 . . . 55
5.8 Bağlantılar . . . 56
5.9 Bölüm cevapları . . . 57

6 Paralel mi, Seri mi? İletim Modları 58
6.1 İletim modları ailesi ❖ Kaynak: D1 s. 67 . . . 58
6.2 Paralel iletim: çok bit, çok hat ❖ Kaynak: D1 s. 68 . . . 59
6.3 Seri iletim: bitler sırayla ❖ Kaynak: D1 s. 69 . . . 60
6.4 Seri iletimin üç zamanlama düzeni ❖ Kaynak: D1 s. 70 . . . 61
6.5 Bağlantılar . . . 62
6.6 Bölüm cevapları . . . 63

Konular Arası Bağlantılar 65
Büyük resim: bir sesin yolculuğu . . . 65
Haftanın kalıpları . . . 66
Gelecek hafta . . . 66

Sınav Öncesi Hızlı Tekrar 67
Ezberlenecek tek sayfa . . . 68
On kritik tuzak . . . 68
Sınav öncesi kontrol listesi . . . 69

Deneme Sınavı 70
Bölüm A çoktan seçmeli, 20 soru, 40 puan . . . 70
Bölüm B klasik sorular, 60 puan . . . 73
Sınav İngilizcesi kalıpları . . . 74

Deneme Sınavı Cevap Anahtarı 76
Bölüm A çoktan seçmeli cevapları . . . 76
Bölüm B klasik soru cevapları . . . 77
Puan tablosu . . . 79
Hangi soru hangi bölümü sınıyor? . . . 79

Terim Sözlüğü **80**

Hocaya Sorulacaklar **83**

Sınavı doğrudan etkileyebilecekler 83

Küçük düzeltmeler 83

Genel sınav soruları. 84

Kaynak Kapsam Kontrolü **85**

Kitapta slayt dışı olan (ek bilgi) ve işaretlenmiş içerik 87

Başlangıç

Hoş geldin! Bu kitap, CMP103 dersinin 5. haftasında gördüğün “How Is Data Stored and Transmitted?” slaytlarını temel alıyor. Ama bir çeviri ya da özet değil: slaytların tek cümleyle geçtiği her şeyi “neden?” sorusuyla açıyoruz, slayttaki her örneği adım adım çözüyoruz ve seni sınava hazırlıyoruz.

Haftanın büyük sorusu şu: Bilgisayarın içinde sayılar, sesler ve mesajlar sonunda **sadece 0 ve 1**. Peki bu 0 ve 1’ler bir kablodan, bir fiberden ya da havadan karşı tarafa nasıl gidiyor ve yolda neden bozulabiliyor?

0.1 Bu not nasıl kullanılmalı?

- **Sırayla oku.** Bölümler birbirinin üstüne kuruluyor: önce bitlerin *anlamını* (işaretli sayılar), sonra bitlerin *fiziksel yolculuğunu* (sinyal, kanal, hat kodlama), sonra analog dünyanın *bite dönüşmesini* (PCM) öğreniyoruz.
- **Kalem kâğıt yanında olsun.** “Durup düşün” ve “Kendin çöz” kutularını cevaba bakmadan yap. Cevaplar her bölümün sonunda.
- **Sınav haftası** “Sınav öncesi hızlı tekrar” bölümünü ve deneme sınavını kullan.

0.2 Dosya kodları ve etiketler

Kod	Dosya	İçerik
D1	CMP103_05-2.pdf	Hafta 5 slaytları, 72 sayfa

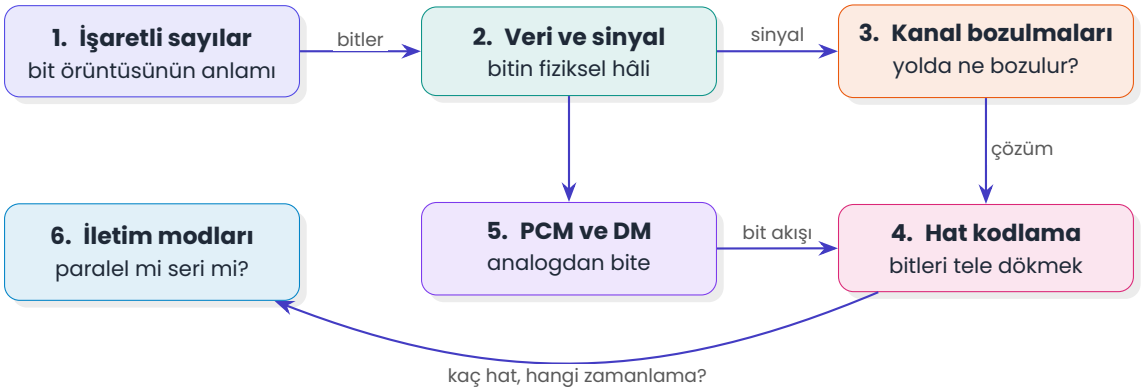
Bir başlığın yanında ❖ Kaynak: D1 s. 14 görürsen, o bilginin D1 dosyasının 14. sayfasından geldiğini anlarsın. Bu PDF’te fiziksel sayfa numarası ile slaytın köşesinde yazan numara **aynı** (kapak ve bölüm ayraçlarında numara basılı değil), bu yüzden tek numara yazıyoruz.

0.3 Gereken ön bilgiler

- İkilik (**binary**) sayıları onluğa ve onluktan ikiliğe çevirmek (önceki haftalar). Unuttuysan 1. bölümdeki hatırlatma kutusu seni toparlar.
- İkilik toplama ve elde (**carry**). Yine 1. bölümde tazeliyoruz.
- Lise matematiğinden üslü sayılar (2^n) ve logaritma ($\log_2$). 2. bölümde hatırlatıyoruz.
- Frekans ve periyot fikri ($f = 1/T$). 2. bölümde hatırlatıyoruz.

0.4 Dersin haritası

Aşağıdaki harita, haftanın konularının birbirine nasıl bağlandığını gösteriyor. Oklar "bunu anlamak için şuna ihtiyacın var" demek.



Şekil 1: Haftanın kavram haritası: bitin anlamından fiziksel yolculuğuna

0.5 Kutular ve etiketler ne anlama geliyor?

Kutu	Ne için?
BU BÖLÜMDE	Bölümde öğreneceklerin, okuma süresi ve zorluk (●●○)
BU KONUYA NEDEN GEÇİYORUZ?	Önceki konuyla bağ: hangi boşluğu dolduruyoruz?
HATIRLATMA	Lise ya da önceki hafta bilgisi tazeleme
TANIM / FORMÜL	Sınavda yazabilmen gereken tanım ve formüller
ÖRNEK	Tam çözümlü örnekler; "Slayt örneği" etiketliler hocanın örnekleri
DURUP DÜŞÜN / KENDİN ÇÖZ	Kendi kendine dene; cevaplar bölüm sonunda
SINAVDA DİKKAT / NASIL SORU GELİR?	Sınav odaklı uyarılar ve soru kalıpları
SIK YAPILAN HATA	Yanlış düşünce → doğrusu → neden
EK BİLGİ (KAYNAK DIŞI)	Slaytlarda olmayan, anlamana yardım eden ek bilgi
MERAKLISINA (SINAV DIŞI)	Sınava girmez; konuyu akılda kalıcı kılar
HIZLI TEKRAR	Bölümün özü; anlatımın yerine geçmez

Rozetler: SINAV ÖNCELİĞİ: YÜKSEK SINAV ÖNCELİĞİ: ORTA SINAV ÖNCELİĞİ: DÜŞÜK sınav önceliği tahminini, ❖ Kaynak: D1 s. 4–25 kaynağı, ✔ python3 ile doğrulandı sayısal sonucun bilgisayarda çalıştırılarak kontrol edildiğini gösterir.

0.6 Ders bilgileri ve yaptığım varsayımlar

- **Ders adı ve kodu:** CMP103 — Introduction to Computer Engineering and Career Planning, Biruni Üniversitesi, Güz 2026 (D1 kapak, s. 1).
- **Kapsam:** Hafta 5, "How Is Data Stored and Transmitted?" (D1 s. 1–72). Haftanın altı sorusu s. 2'de, öğrenme çıktıları s. 3'te.
- **Hangi sınav:** Dönemin ilk yarısında bir hafta olduğu için **vize** hazırlığı olarak ele aldım (tahmin).

- **Sınav tarihi:** Bilinmiyor; bu yüzden aşağıda "sınava kaç gün kala" planı var.
- **Sınav formatı:** Geçmiş yıl sorusu yok; deneme sınavı karışık: çoktan seçmeli + klasik (hesap, çizim, açıklama). Ders programlama dersi olmadığı için kod yazma sorusu yok.
- **Sınav dili:** Slaytlar İngilizce, sınav dili bilinmiyor. Deneme sınavı soruları İngilizce, altlarında Türkçe çevirileriyle.
- **Hocanın vurguları:** Slaytların alt kutularında tekrar eden mesajlardan çıkardım (ör. "gösterimi bilmeden bit örüntüsü yorumlanamaz", "iki sıfır problemi", "taşma işaretlerle kontrol edilir").
- **Zor kabul ettiğim konular:** 2'ye tümleyenin *neden* çalıştığı, taşma tespiti, NRZ-I ve fark Manchester çizimi, Nyquist ve örtüşme, nicemleme tablosu, delta modülasyonu hataları.
- **Ondalık ayırıcı:** Slaytlarla uyumlu olsun diye **nokta** kullandım (0.02 ms, −1.22 gibi).
- **Sınav öncelikleri** (Yüksek / Orta / Düşük) benim tahminimdir; hocanın gerçek sınavını görmedim.

0.7 Çalışma planı önerisi

Ne zaman?	Ne yap?
Sınava 7 gün kala	1. ve 2. bölümü oku, tüm örnekleri kendin yeniden çöz.
6–5 gün kala	3. ve 4. bölüm. Hat kodlama çizimlerini kâğıda kendin çiz.
4 gün kala	5. ve 6. bölüm. PCM ve DM hesaplarını tekrarla.
3 gün kala	Deneme sınavını süre tutarak çöz (75 dk), sonra cevap anahtarıyla karşılaştır.
2 gün kala	Yanlış yaptığın konuların "Sık yapılan hatalar" ve "Sınavda dikkat" kutuları.
1 gün kala	"Sınav öncesi hızlı tekrar" bölümü + her bölümün HIZLI TEKRAR kutusu.

BÖLÜM 1

Negatif Sayılar Bitlerle Nasıl Yazılır?

◆ **BU BÖLÜMDE** Okuma süresi: 60 dk Zorluk: ●●●

- Aynı bit örüntüsünün neden farklı sayılar anlamına gelebileceğini
- İşaret-büyüklük, 1'e tümleyen ve 2'ye tümleyen gösterimlerini; her birinin aralığını ve sıfır sayısını
- 2'ye tümleyende negatif sayı oluşturmayı ve bir bit örüntüsünü onluğa çevirmeyi
- Çıkarmayı toplama devresiyle yapmayı ve taşmayı (**overflow**) doğru tespit etmeyi

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 4–25

Sınav önceliği yüksek, çünkü bu bölüm haftanın en çok hesap içeren kısmı; slaytlarda üç çözümlü örnek ve bir sınıf etkinliği var, üstelik hem çoktan seçmeli hem klasik soruya çok elverişli.

→ BU KONUYA NEDEN GEÇİYORUZ?

Önceki haftalarda pozitif tam sayıları ikilik tabanda yazmayı öğrendin: 13 = 1101 gibi. Ama gerçek hayatta eksi sayılar da var: banka hesabındaki eksi bakiye, −5 °C hava, oyunda sola doğru hız. Elimizde ise yalnızca 0 ve 1 var; "−" işareti için ayrı bir sembol yok. Bu bölümde, bitlerle eksi sayı yazmanın üç yolunu görecek ve bugün neredeyse bütün bilgisayarların neden üçüncüsünü, yani 2'ye tümleyeni kullandığını kendin çıkaracaksın.

↺ HATIRLATMA

İkilikten onluğa: Her bit, bulunduğu basamağın ağırlığıyla çarpılır. 4 bitte ağırlıklar soldan sağa 8, 4, 2, 1'dir. Örneğin $1101_2 = 8 + 4 + 0 + 1 = 13$.
İkilik toplama kuralları: $0 + 0 = 0$, $0 + 1 = 1$, $1 + 1 = 10$ (yani 0 yaz, 1 elde), $1 + 1 + 1 = 11$ (1 yaz, 1 elde). Elde (**carry**) bir sola, bir sonraki sütuna gider; tıpkı onlukta $7 + 5 = 12$ yapınca 1'i onlar basamağına taşıman gibi.
n **bitle kaç örüntü?** Her bit 2 değer alır, n bit birbirinden bağımsızdır: $2 \cdot 2 \cdots 2 = 2^n$ örüntü. 4 bit → 16, 8 bit → 256.

1.1 Eksi işareti için tuş yok: neden özel bir gösterim gerekiyor?

❖ Kaynak: D1 s. 4–6

Diyelim ki bir arkadaşın sana kâğıda yalnızca `1101` yazıp gönderdi. Bu sayı kaç? "13" demek istiyorsun, değil mi? Ama arkadaşın aklında başka bir kural olabilir. Slaytın ana mesajı tam olarak bu:

bir bit örüntüsünün anlamını, seçilen sayı gösterimi (**number representation**) belirler.

Slaytın vurguladığı cümle: *"We cannot interpret a bit pattern correctly until we know its representation."* Yani gösterimi bilmeden bir bit örüntüsünü doğru yorumlayamayız.

Örüntü `1101`	Okuma kuralı	Değer
İşaretsiz (**unsigned**)	Bütün bitler büyüklük	13
İşaret-büyüklük	İlk bit işaret, kalanı büyüklük	−5
1'e tümleyen	Negatifse bitleri ters çevir	−2
2'ye tümleyen	Negatifse ters çevir, 1 ekle	−3

Bu dört değeri bu bölümün sonunda kendin çıkarabileceksin. Gösterim seçimi şu üç şeyi etkiler (D1 s. 5):

- **Aralık (range):** Hangi en küçük ve en büyük sayıyı yazabiliyorum?
- **Sıfır sayısı:** Sıfırın bir tane mi, iki tane mi gösterimi var?
- **Aritmetik devrenin karmaşıklığı:** Toplama ve çıkarma için ne kadar karmaşık bir donanım gerekiyor?

Slaytın dördüncü maddesi sonucu baştan söylüyor: modern genel amaçlı bilgisayarlar işaretli tam sayılar için **2'ye tümleyen** (**two's complement**) kullanır. Şimdi bu sonuca adım adım ulaşalım. Üç aday var (D1 s. 6):

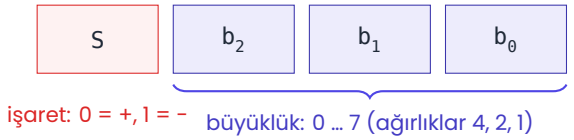
Gösterim	Negatif sayı nasıl oluşur?
İşaret-büyüklük (**signed magnitude**)	En soldaki bit işarettir; kalan bitler büyüklüğü verir.
1'e tümleyen (**one's complement**)	Pozitif sayının bütün bitleri ters çevrilir.
2'ye tümleyen (**two's complement**)	Bitler ters çevrilir, sonuca 1 eklenir.

✎ DURUP DÜŞÜN

D1.1 `1000` örüntüsü işaretsiz, işaret-büyüklük, 1'e tümleyen ve 2'ye tümleyen gösterimlerde kaç anlama gelir? (Bölümü okudukça cevabını doldur; cevap bölüm sonunda.)

1.2 İlk akla gelen fikir: işaret-büyüklük ❖ Kaynak: D1 s. 7–9

Kâğıt üzerinde eksi sayıyı nasıl yazıyorsun? Başına bir "−" koyuyorsun, sayının kendisine dokunmuyorsun. İşaret-büyüklük gösterimi tam olarak bunu bitlerle yapar: en soldaki biti "işaret kutusu" olarak ayırır.



Şekil 1.1: 4 bitlik işaret-büyüklük: 1 işaret biti + 3 büyüklük biti (D1 s. 7)

∑ TANIM / FORMÜL — İşaret-büyüklük gösterimi

n bitlik bir örüntüde en soldaki bit **işaret bitidir** (**sign bit**): 0 ise sayı pozitif, 1 ise negatiftir. Kalan $n-1$ bit, sayının mutlak değerini (büyüklüğünü, **magnitude**) sıradan ikilik sayı olarak verir.
Slayttaki ifade: *"The leftmost bit gives the sign; the remaining bits give the magnitude."*
Aralık: $-(2^{n-1}-1) \ \ldots \ +(2^{n-1}-1)$
Sıfır sayısı: iki ($+0$ ve -0)

▶ ÖRNEK 1.1 — Slayt örneği [D1 s. 7]: 1101 kaçtır?

Verilen: 4 bitlik işaret-büyüklük örüntüsü `1101`. **İstenen:** onluk değeri.

1. İşaret biti en soldaki bit: **1** → sayı negatif.
2. Kalan üç bit `101` → $4+0+1=5$ → büyüklük 5.
3. Sonuç: $1101=-5$.

Sık hata: `1101`'i 13 diye okuyup başına eksi koymak (−13). İşaret biti büyüklüğe dahil değildir!

Slayttaki tablo (D1 s. 8) 4 bitlik bütün değerleri gösteriyor. Dikkat et: pozitif ve negatif sütunlar arasındaki tek fark en soldaki bit.

N	$+N$	$-N$
0	`0000`	`1000`
1	`0001`	`1001`
2	`0010`	`1010`
3	`0011`	`1011`
4	`0100`	`1100`
5	`0101`	`1101`
6	`0110`	`1110`
7	`0111`	`1111`

1.2.1 Aralık formülü nereden geliyor?

Formülü ezberlemek yerine çıkaralım. n bitin biri işarete gitti, büyüklüğe $n-1$ bit kaldı. $n-1$ bitle yazılabilecek en büyük işaretsiz sayı, bütün bitlerin 1 olduğu sayıdır: $\underbrace{11\ldots1}_{n-1}$. Bu da $2^{n-1}-1$'e eşittir.

Neden? `111` sayısına 1 eklersen `1000` olur, yani 2^3. Demek ki `111` $=2^3-1=7$. Genel olarak $n-1$ tane 1, $2^{n-1}-1$ eder. En büyük pozitif $+(2^{n-1}-1)$, en küçük negatif $-(2^{n-1}-1)$. 4 bit için $-7 \ldots +7$ (D1 s. 9).

1.2.2 İki sıfır neden dert?

Tabloda iki tane sıfır var: `0000` = +0 ve `1000` = −0. Matematikte $+0 \ = \ -0$, ama donanım için bunlar farklı örüntüler. Slaytın uyarısı: iki sıfır olması donanımı ve karşılaştırmaları gereksiz yere karmaşıklaştırır (D1 s. 9).

Somut düşün: bir program `if (x == 0)` diye soruyorsa, devre artık iki örüntüyü kontrol etmek zorunda. Üstelik 16 örüntüden biri "boşa gidiyor"; 15 farklı sayı yazabiliyoruz.

Bir de toplama sorunu var. Sıradan bir ikilik toplayıcıya (+3) ile (−3)'ü verelim:

▶ ÖRNEK 1.2 — Ek örnek: işaret-büyüklükte sıradan toplama çalışmaz

$+3 = 0011$, $-3 = 1011$. Bunları işaretsiz sayılar gibi toplarsak: $0011 + 1011 = 1110$. `1110` işaret-büyüklükte -6 demek. Ama $(+3) + (-3) = 0$ olmalıydı!
Sonuç: İşaret-büyüklükte toplama yapmak için devre önce işaretlere bakmalı, işaretler farklıysa büyüklükleri karşılaştırıp büyükten küçüğü çıkarmalı, sonucun işaretini ayrıca belirlemeli. Yani ayrı bir çıkarıcı devre ve karar mantığı gerekir. Kâğıt üstünde doğal ama donanım için pahalı bir gösterim.

1.3 Bitleri ters çevir: 1'e tümleyen

❖ Kaynak: D1 s. 10–13

İşaret-büyüklükte sadece işaret biti değişiyordu. Yeni fikir: negatif sayıyı yazarken pozitif sayının **bütün** bitlerini ters çevir (0 → 1, 1 → 0). Slaytın notu önemli: 1'e tümleyende işaret biti ayrıca ele alınmaz, örüntünün tamamı dönüştürülür (D1 s. 10).

∑ TANIM / FORMÜL — 1'e tümleyen gösterimi

Pozitif sayılar sıradan ikilik yazılır (en soldaki bit 0). $-x$ elde etmek için $+x$'in **her biti ters çevrilir**.
Slayttaki ifade: *"A negative value is formed by inverting every bit of the positive value."*
Aralık: $-(2^{n-1}-1)$ … $+(2^{n-1}-1)$
Sıfır sayısı: iki (`0000` = +0, `1111` = −0)

▶ ÖRNEK 1.3 — Slayt örnekleri [D1 s. 10, 12]: -5 ve -3

−5 (4 bit): $+5 = 0101$ → her biti ters çevir → 1010. Yani $-5 = 1010$.
−3 (4 bit): $+3 = 0011$ → ters çevir → 1100. Yani $-3 = 1100$.
Geri dönüş: Tekrar ters çevirirsen orijinal sayıya dönersin: $1100 \to 0011 = +3$ (D1 s. 12).
Bu yüzden negatif bir örüntüyü okumak da kolay: ters çevir, büyüklüğü oku, başına eksi koy.

N	$+N$	$-N$ (1'e tümleyen)
0	`0000`	`1111`
1	`0001`	`1110`
2	`0010`	`1101`
3	`0011`	`1100`
4	`0100`	`1011`
5	`0101`	`1010`
6	`0110`	`1001`
7	`0111`	`1000`

1.3.1 Tümleyen kelimesi nereden geliyor?

Bir sayıyı ve ters çevrilmiş hâlini toplayalım: $0101 + 1010 = 1111$. Her sütunda biri 0, diğeri 1 olduğu için hiç elde oluşmaz ve sonuç hep `1111` olur. Yani $x + \bar{x} = 1111 = 2^n - 1$.

Buradan $\bar{x} = (2^n - 1) - x$ çıkar. 1'e tümleyen, $-x$'i "$(2^n - 1)$'den x'i çıkarınca kalan" ile temsil ediyor. Sayının, `1111`'i **tamamlayan** parçası; adı buradan geliyor.

1.3.2 Hâlâ iki sıfır var, üstüne bir de çevrimsel elde

Sıfırın tersi: `0000` ters çevrilince `1111` olur. İkisi de sıfır; demek ki sorun çözülmedi (D1 s. 13). Toplamada da küçük bir pürüz var: en soldan dışarı taşan elde atılmaz, sonucun en sağına **geri eklenir**. Buna çevrimsel elde (**end-around carry**) denir.

▶ ÖRNEK 1.4 — Ek örnek: 1'e tümleyende 5 + (-3)

$+5 = 0101, -3 = 1100.$

1. Topla: $0101 + 1100 = 1\,0001$ (5 bit çıktı; en soldaki 1 dışarı taşan elde).
2. Çevrimsel elde: dışarı taşan 1'i sonucun en sağına ekle: $0001 + 1 = 0010$.
3. Sonuç: $0010 = +2$. Doğru, çünkü $5 - 3 = 2$.

Eldeyi geri eklemeseydik `0001` = 1 bulurduk, yani yanlış. Bu fazladan ekleme, devreye ek bir adım (ve gecikme) getirir.

Slaytın bu alt bölümü bir sonraki adımı da müjdeliyor: *"Two's complement solves these issues with a simpler rule."*

1.4 Ters çevir, 1 ekle: 2'ye tümleyen ❖ Kaynak: D1 s. 14–19

İşte bilgisayarların gerçekten kullandığı yöntem. Kural 1'e tümleyene sadece bir adım ekliyor: ters çevir, sonra **1 ekle**.

∑ TANIM / FORMÜL — 2'ye tümleyen gösterimi

Pozitif sayılar sıradan ikilik yazılır. $-x$ elde etmek için: (1) $+x$'i yaz, (2) bütün bitleri ters çevir, (3) sonuca 1 ekle. **Sonucu aynı bit genişliğinde tut** (taşan elde atılır).
Eşdeğer tanım (D1 s. 16): $-x$, $b^D - x$ sayısıyla temsil edilir. İkilik tabanda $b = 2$, D = bit sayısı (n), yani $-x \leftrightarrow 2^n - x$.
Aralık: -2^{n-1} ... $+(2^{n-1} - 1)$
Sıfır sayısı: bir

▶ ÖRNEK 1.5 — Slayt örneği [D1 s. 14]: -5'i 4 bitte yaz

1. Pozitif değeri yaz: $+5 = 0101$.
2. Bitleri ters çevir: 1010.
3. 1 ekle: $1010 + 0001 = 1011$.

Sonuç: $-5 = 1011$. Kontrol (D1 s. 16 yöntemi): $2^4 - 5 = 16 - 5 = 11 = 1011_2$. İki yol aynı sonucu veriyor.

▶ ÖRNEK 1.6 — Slayt örneği [D1 s. 16]: b üzeri D - x yöntemiyle -1

$b = 2, D = 4$: $-1 \leftrightarrow 2^4 - 1 = 15 = 1111_2$.
Ters çevir + 1 ile kontrol: $0001 \to 1110 \to 1110 + 1 = 1111$. ✔

1.4.1 Neden ters çevir + 1, $2^n - x$ ile aynı şey?

İşte işin püf noktası. 1'e tümleyende $\bar{x} = (2^n - 1) - x$ olduğunu gördük. Buna 1 eklersek:

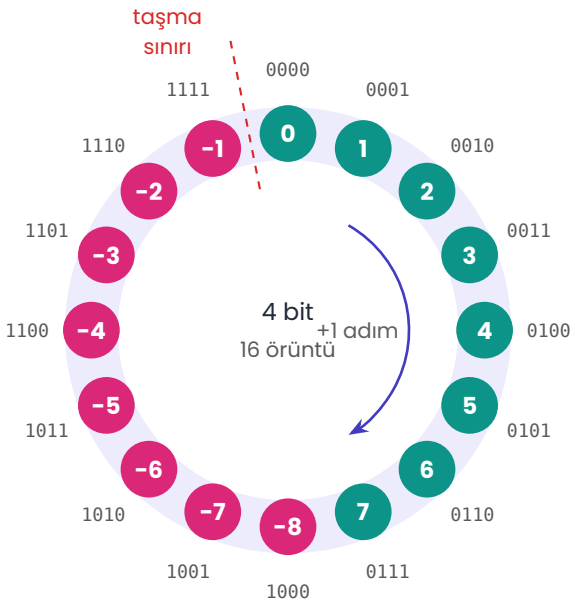
$$\bar{x} + 1 = (2^n - 1) - x + 1 = 2^n - x.$$

Yani "ters çevir, 1 ekle" kuralı, $2^n - x$ hesabının kâğıt kalem gerektirmeyen kısayoludur. Slayt bu soruyu kırmızıyla soruyor (*"Shortcut for base 2?"*, D1 s. 16) ve cevabı alt kutuda veriyor: ters çevir, 1 ekle.

1.4.2 2'ye tümleyen neden çalışıyor? Saat benzetmesi

Bir duvar saati düşün. Saat 3'teyken 5 saat geri gitmek ile 7 saat ileri gitmek seni aynı yere götürür (10'a), çünkü saat 12'de başa döner. Yani 12'lik saatte "−5" ile "+7" aynı işi görür: $7 = 12 - 5$.

4 bitlik bir sayaç da 16'da başa döner: `1111`'e 1 eklersen `0000` olur (taşan elde atılır). Bu yüzden "−5", "+11" ile aynı işi görür ve $11 = 16 - 5$. 2'ye tümleyen, örüntüleri bu çemberin üzerine yerleştirir.



Şekil 1.2: 4 bitlik 2'ye tümleyen çemberi. Turkuaz: pozitifler, pembe: negatifler. Saat yönünde bir adım = +1. 7'den sonra −8'e "atlamak" taşmadır.

Çemberden üç sonuç çıkıyor:

- **Tek sıfır:** `0000` bir kez geçiyor. `1111` artık −0 değil, −1.
- **Toplama doğal çalışır:** -3'ten (`1101`) 5 adım ileri git: $-2, -1, 0, 1, 2$. Sonuç 2. Devre sadece "ileri say"ı (sıradan toplamayı) bilmek zorunda.
- **İşaret biti bedava:** Negatiflerin hepsi 1 ile, pozitiflerin hepsi 0 ile başlıyor. En soldaki bit hâlâ işareti söylüyor; ama bu bit artık ayrı bir kutu değil, sayının parçası.

N	$+N$	$-N$ (2'ye tümleyen)
0	`0000`	`0000`
1	`0001`	`1111`
2	`0010`	`1110`
3	`0011`	`1101`
4	`0100`	`1100`
5	`0101`	`1011`
6	`0110`	`1010`
7	`0111`	`1001`
8	—	`1000`

Son satıra dikkat: slayttaki tablo (D1 s. 15) −7'de bitiyor, ama alt kutu aralığın −8...+7 olduğunu söylüyor. Eksik satır `1000` = −8'dir. (-0'ın tersi de `0000` çıkıyor: $0000 \to 1111 \to 1111 + 1 = 1\,0000$, dışarı taşan 1 atılınca yine `0000`.)

1.4.3 Aralık ve asimetri

16 örüntüden biri sıfıra gitti. Kalan 15 örüntü 7 pozitife ve 8 negatife paylaşıldı. Bu yüzden 2'ye tümleyenin aralığı asimetriktir:

$$-2^{n-1} \ \ldots \ +(2^{n-1}-1)$$



Şekil 1.3: 4 bitte −8...+7: negatif taraf bir fazla (D1 s. 15, 19)

Bit sayısı	Aralık (D1 s. 19)
4	−8 ... +7
8	−128 ... +127
16	−32,768 ... +32,767

Slaytın uyarısı: en negatif değerin aynı genişlikte pozitif karşılığı yoktur (D1 s. 19). 4 bitte −8 var ama +8 yok.

✎ DURUP DÜŞÜN

D1.2 4 bitlik 2'ye tümleyende −8'in (`1000`) negatifini "ters çevir, 1 ekle" kuralıyla bul. Ne çıkıyor ve bu neden mantıklı?

1.4.4 Bir örüntüyü onluğa çevirmek

Sınavda en çok bu sorulur: "Bu 8 bitlik 2'ye tümleyen örüntü kaçtır?" Slayttaki yöntem üç adım:

1. En soldaki bite bak. 0 ise sayı pozitif, doğrudan oku; bitti.
2. 1 ise sayı negatif: bitleri ters çevir, 1 ekle.
3. Çıkan büyüklüğü oku, başına eksi koy.

▶ ÖRNEK 1.7 — Slayt örneği [D1 s. 17]: 1111 0110 kaçtır?

Verilen: 8 bitlik 2'ye tümleyen `1111 0110`.

1. En soldaki bit 1 → sayı negatif.
2. Ters çevir: `0000 1001`.
3. 1 ekle: `0000 1001` + 1 = `0000 1010` → $8 + 2 = 10$.
4. Sonuç: $1111\,0110 = -10$.

✔ python3 ile doğrulandı

+ EK BİLGİ (KAYNAK DIŞI) — İkinci yol: en soldaki bitin ağırlığı negatiftir

2'ye tümleyende en soldaki bitin ağırlığını -2^{n-1} kabul edip diğerlerini normal toplarsan doğrudan sonucu bulursun. 8 bitte ağırlıklar: $-128, 64, 32, 16, 8, 4, 2, 1$.
`1111 0110` → $-128+64+32+16+0+4+2+0=-10$. ✔
Bu yöntem slaytlarda yok ama sonucu kontrol etmek için çok hızlı. Özellikle `1000 0000` gibi tuzaklı örüntülerde işe yarar: -128.

▶ ÖRNEK 1.8 — Slayt sınıf etkinliği [D1 s. 24]: dört örüntüyü çevir

Örüntü	En soldaki bit	İşlem	Sonuç
`0000 1101`	0 → pozitif	$8+4+1$	**13**
`1111 0011`	1 → negatif	ters: `0000 1100`, +1: `0000 1101` = 13	**−13**
`1000 0000`	1 → negatif	ters: `0111 1111`, +1: `1000 0000` = 128 (işaretsiz okuyunca)	**−128**
`0111 1111`	0 → pozitif	$64+32+16+8+4+2+1$	**127**

Slaytın cevapları: 13, −13, −128, 127. ✔ ✔ python3 ile doğrulandı
Püf noktası: `1000 0000`'da ters çevir + 1 seni aynı örüntüye geri getirir. Bunu "negatifi kendisi" diye okuyup panik yapma: çıkan `1000 0000`'ı **işaretsiz** okursan 128 eder, başına eksi koyunca −128. 8 bitin en küçük sayısı ve pozitif karşılığı yok.

+ EK BİLGİ (KAYNAK DIŞI) — Bit genişliğini büyütmek: işaret genişletme

Slayt "sonucu aynı bit genişliğinde tut" diyor (D1 s. 16). Peki 4 bitlik −3'ü (`1101`) 8 bite taşımak istersek? Sola **işaret bitinin kopyasını** ekleriz: `1111 1101` (= −3). Pozitif sayılarda sola 0 eklenir: `0011` → `0000 0011`. Buna işaret genişletme (**sign extension**) denir. Sola 0 eklersen (`0000 1101`) sayı 13'e dönüşür; sık yapılan bir hata.

1.5 Üçünü yan yana koyalım ❖ Kaynak: D1 s. 18

Slaytın karşılaştırma tablosu bölümün özeti. Bunu sınava kadar ezbere bilmeni değil, **her hücresini açıklayabilmeni** istiyorum.

Gösterim	Negatif nasıl oluşur?	Sıfır sayısı	n bitlik aralık
İşaret-büyüklük	İşaret bitini değiştir	İki	$-(2^{n-1}-1)\ldots+(2^{n-1}-1)$
1'e tümleyen	Her biti ters çevir	İki	$-(2^{n-1}-1)\ldots+(2^{n-1}-1)$
2'ye tümleyen	Ters çevir, 1 ekle	Bir	$-2^{n-1}\ldots+(2^{n-1}-1)$

Bir de 4 bitin 16 örüntüsünün hepsini dört gözlükle okuyalım. Bu tablo aynı örüntünün nasıl farklı sayılara dönüştüğünü bir bakışta gösteriyor:

Örüntü	İşaretsiz	İşaret-büyüklük	1'e tümleyen	2'ye tümleyen
`0000`	0	+0	+0	0
`0001`	1	1	1	1
`0010`	2	2	2	2
`0011`	3	3	3	3
`0100`	4	4	4	4
`0101`	5	5	5	5
`0110`	6	6	6	6
`0111`	7	7	7	7
`1000`	8	−0	−7	−8
`1001`	9	−1	−6	−7
`1010`	10	−2	−5	−6
`1011`	11	−3	−4	−5
`1100`	12	−4	−3	−4
`1101`	13	−5	−2	−3
`1110`	14	−6	−1	−2
`1111`	15	−7	−0	−1

✔ python3 ile doğrulandı

Gözüne çarpması gereken desen: **pozitif yarı** (`0xxx`) dört gösterimde de aynı. Farklar sadece en soldaki bit 1 olduğunda başlıyor.

1.6 Çıkarma = toplama: 2'ye tümleyenle aritmetik

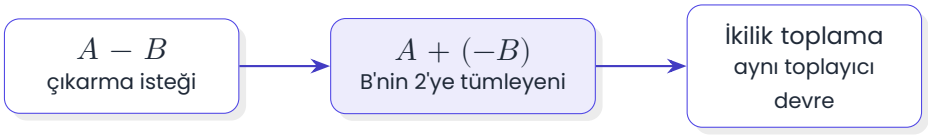
❖ Kaynak: D1 s. 20–22

2'ye tümleyenin asıl süper gücü burada. Slaytın toplama kuralları (D1 s. 20):

1. Bitleri sıradan ikilik toplama ile topla (işarete ayrıca bakma!).
2. Sabit bit genişliğinin dışına taşan eldeyi **at**.
3. Sonuç, seçilen bit genişliğinin aralığındaysa geçerlidir.
4. Aynı işaretli iki sayıyı toplamak ters işaretli bir sonuç verirse **taşma** vardır.

Çıkarma için ayrı bir kural yok, çünkü:

$$A - B = A + (-B).$$



Şekil 1.4: Çıkarma, toplamaya dönüştürülür; ayrı bir çıkarıcı devre gerekmez (D1 s. 21)

Donanım için müthiş bir kazanç: aynı toplayıcı (**adder**) devre hem toplama hem çıkarma yapıyor. Tek ek iş B'yi ters çevirip 1 eklemek ve ters çevirmek basit kapılarla yapılır.

▶ ÖRNEK 1.9 — Slayt örneği [D1 s. 22]: 58 - 40 = 18 (8 bit)

Adım 1 — İkiliğe çevir. $58 = 32 + 16 + 8 + 2 = 0011\,1010$. $40 = 32 + 8 = 0010\,1000$.
Adım 2 — −40'ı bul. `0010 1000` → ters çevir: `1101 0111` → 1 ekle: `1101 1000`. Kontrol: $256 - 40 = 216 = 1101\,1000_2$. ✔
Adım 3 — Topla. Elde zincirini sağdan sola izle: b3 sütununda $1 + 1 = 10$ ile ilk elde doğar ve sola doğru dört sütun boyunca taşınır.

elde	1	1	1	1					
	0	0	1	1	1	0	1	0	58
+	1	1	0	1	1	0	0	0	-40
dış elde	1	**0**	**0**	**0**	**1**	**0**	**0**	**1**	**0**	**18**

Adım 4 — Dış eldeyi at. 9. bite taşan 1 atılır (8 bitlik bir kayıtta yeri yok). Kalan: `0001 0010` $= 16 + 2 = 18$. ✔
Taşma var mı? İşaretler farklı (+58 ve −40) olduğu için taşma olamaz; sonuç da aralıkta.
✔ python3 ile doğrulandı

▶ ÖRNEK 1.10 — Ek örnek: 40 - 58 = -18 (dış elde çıkmıyor)

-58: `0011 1010` → `1100 0101` → +1 → `1100 0110`.
$0010\,1000 + 1100\,0110 = 1110\,1110$. Bu kez dışarı elde taşmadı.
Sonuç negatif (en soldaki bit 1). Ters çevir: `0001 0001`, +1: `0001 0010` = 18 → sonuç -18. ✔
✔ python3 ile doğrulandı
Ders: Dış elde çıksa da çıkmasa da 2'ye tümleyende sonuç doğru olabilir. Dış elde "hata" demek değildir.

✎ KENDİN ÇÖZ — Yarı çözümlü örnek

Y1.1 8 bitlik 2'ye tümleyende $25 - 37$'yi hesapla.
1. $25 = 0001\,1001$, $37 = 0010\,0101$.
2. -37: ters çevir `1101 1010`, 1 ekle → `?`
3. $0001\,1001 + (-37) = ?$ Sonucu onluğa çevir ve $25 - 37$ ile karşılaştır.

1.7 Taşma: bit genişliği sonucu taşıyamıyor

❖ Kaynak: D1 s. 20, 23

8 bitlik bir 2'ye tümleyen sayı en fazla +127 olabilir. Peki toplam 127'yi aşarsa ne olur? Çemberi hatırla: 7'den sonra −8'e "atlıyorduk". 8 bitte de 127'den sonra −128 geliyor.

▶ ÖRNEK 1.11 — Slayt örneği [D1 s. 23]: 100 + 60 (8 bit)

1. Aralık: $-128 \ldots +127$.
2. Gerçek sonuç 160, aralığın dışında.
3. Bitlerle yapalım: $100 = 0110\,0100$, $60 = 0011\,1100$.
$0110\,0100 + 0011\,1100 = 1010\,0000$.
4. En soldaki bit 1, örüntü negatif görünüyor: ters çevir `0101 1111`, +1 → `0110 0000` = 96 → **−96**. İki pozitif sayıyı toplayıp negatif bir sonuç bulduk: **taşma**. Bu sonuç kullanılmamalıdır.

İlginç ayrıntı: $160 - 256 = -96$. Taşma, sonucu tam 2^8 kadar kaydırır (çemberde bir tur). Dışarı elde de çıkmadı! ✔ python3 ile doğrulandı

∑ TANIM / FORMÜL — Taşma (overflow) kuralı

2'ye tümleyen toplamada **taşma**, gerçek sonucun seçilen bit genişliğinin aralığına sığmamasıdır.
Tespit: İki toplanan **aynı işaretliyse** ve sonuç **ters işaretliyse** taşma vardır. Toplananların işaretleri farklıysa taşma **asla** olmaz.
Slaytın uyarısı: taşma sadece son eldeye bakılarak tespit edilmez; işaretler de kontrol edilmelidir (D1 s. 23).

1.7.1 Neden farklı işaretlilerde taşma olmaz?

Bir pozitif ($A \geq 0$) ile bir negatifi ($B < 0$) toplarsan sonuç ikisinin **arasında** kalır: $B \leq A + B < A$. İki sayı da aralıkta olduğu için aradaki her şey de aralıkta. Taşmanın tek yolu, iki sayının aynı yöne "itmesi".

1.7.2 Elde ile taşma aynı şey değil

Toplama (4 bit)	İkilik	Dış elde	Taşma	Neden
$(-3) + (-4)$	$1101 + 1100 = 1\,1001$	1	Yok	Sonuç `1001` = −7, aralıkta; işaret korunmuş
$5 + 4$	$0101 + 0100 = 0\,1001$	0	Var	İki pozitif → `1001` = −7 (negatif!)
$(-5) + (-6)$	$1011 + 1010 = 1\,0101$	1	Var	İki negatif → `0101` = +5 (pozitif!)
$5 + (-3)$	$0101 + 1101 = 1\,0010$	1	Yok	Farklı işaretler; sonuç 2

✔ python3 ile doğrulandı

Gördüğün gibi dış elde 1 olup taşma olmayabilir; 0 olup taşma olabilir. **Kural işaretlerde.**

+ EK BİLGİ (KAYNAK DIŞI) — Donanım taşmayı nasıl görür?

İşlemciler taşmayı genelde şöyle bulur: en soldaki sütuna **giren** elde ile o sütundan **çıkan** elde farklıysa taşma vardır. $5 + 4$ örneğinde en sol sütuna 1 girer, 0 çıkar → taşma. Bu, "aynı işaret → ters işaret" kuralının devre hâlidir. Slaytlarda yok; sınavda işaret kuralını kullan.

✎ DURUP DÜŞÜN

D1.3 Bir pozitif ile bir negatif sayıyı 2'ye tümleyende topladığında dış elde 1 çıktı. Taşma var mı? Neden?

★ MERAKLISINA (SINAV DIŞI) — Gerçek hayatta taşma

2014'te YouTube'un izlenme sayacı bir videoda 32 bitlik işaretli sayı sınırına (2,147,483,647 = $2^{31} - 1$) yaklaşınca YouTube sayacı 64 bite çıkarmak zorunda kaldı. Benzer bir saat bombası da "2038 problemi": zamanı 1970'ten beri geçen saniye olarak 32 bitlik işaretli sayıda tutan eski sistemler, 19 Ocak 2038'de $2^{31} - 1$ saniyeyi aşınca negatife, yani 1901 yılına "atlayacak". Çemberdeki o pembe-turkuaz sınırın gerçek hayattaki karşılığı bu.

✎ KENDİN ÇÖZ — Kendin çöz

K1.1 8 bitlik 2'ye tümleyende $(-100) + (-50)$ işlemini bitlerle yap. Sonuç örüntüsü nedir, onluk karşılığı kaçtır, taşma var mı?
K1.2 6 bitlik 2'ye tümleyenin aralığı nedir? 6 bitlik işaret-büyüklüğün aralığı nedir?
K1.3 `1101` örüntüsünün işaret-büyüklük, 1'e tümleyen ve 2'ye tümleyendeki değerlerini gerekçeleriyle bul (bölüm başındaki tabloyu kapatarak).

1.8 Bağlantılar

- **Önceki hafta → bu bölüm:** İşaretsiz ikilik sayılar, 2'ye tümleyenin "pozitif yarısı" olarak aynen kullanılıyor.
- **Bu bölüm → sonraki bölümler:** Buraya kadar bitlerin *ne anlama geldiğini* konuştuk. Ama `1111 0110` örüntüsü bir kablodan karşıya giderken artık "−10" değil, sadece bir dizi voltaj seviyesidir. 2. bölümde bitlerin fiziksel yolculuğuna geçiyoruz.
- **İleride:** Bilgisayar Organizasyonu ve Mantık Tasarımı derslerinde bu bölümün devre hâlini (toplayıcı, çıkarıcı, taşma bayrağı) göreceksin. C ya da Java'da `int` tipinin aralığı tam olarak $-2^{31} \ldots 2^{31} - 1$'dir; nedenini artık biliyorsun.

⚠ SINAVDA DİKKAT

- **Önce gösterimi oku.** Soru "8-bit two's complement" mi, "signed magnitude" mı, "one's complement" mı diyor? Aynı örüntü farklı cevap verir.
- **Bit genişliğini koru.** 8 bitlik soruda cevabı 8 bit yaz; dış eldeyi at, sol taraftaki sıfırları silme.
- **Aralık formülleri:** SM ve 1C simetrik, $-(2^{n-1} - 1) \ldots 2^{n-1} - 1$; 2C asimetrik, $-2^{n-1} \ldots 2^{n-1} - 1$.
- **Taşma = işaret kuralı**, dış elde değil.
- **`1000 0000` = −128** (8 bit, 2C). Pozitif karşılığı yok.

❖ BU KONUDAN NASIL SORU GELİR?

1. Kavramsal soru: *"Explain why two's complement is preferred over signed magnitude."* İpucu kelimeler: *preferred, advantage, why*. Cevaba şu üç başlıkla başla: tek sıfır, aynı toplayıcıyla çıkarma (ayrı çıkarıcı yok), bir ekstra negatif değer. Bir küçük örnekle destekle (işaret-büyüklükte $+3 + (-3)$ toplayıcıda -6 veriyor).
2. Dönüşüm sorusu: *"What is the decimal value of 1110 1001 in 8-bit two's complement?"* İpucu: en soldaki bit. 1 ise ters çevir, 1 ekle, eksi koy. (-23)
3. Aritmetik + taşma: *"Compute 75 + 60 using 8-bit two's complement. Is there overflow?"*

İpucu: aynı işaretli iki sayı. Bitlerle topla, sonucun işaretine bak. (Sonuç `1000 0111` = −121, taşma var.)
4. Aralık: *"What is the range of a 10-bit two's complement number?"* $(-512 \dots 511)$

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "2'ye tümleyende negatif sayıyı okumak için sadece işaret bitini atarım." → **Doğrusu:** Ters çevir, 1 ekle, sonra oku. → **Neden:** Kalan bitler büyüklük değildir; `1011`'deki `011` = 3 ama sayı −5.
- **Yanlış:** "Dış elde çıktıysa taşma vardır." → **Doğrusu:** Taşmaya işaretlerle karar verilir. → **Neden:** 58 − 40'ta dış elde çıktı ama sonuç doğru (18).
- **Yanlış:** "1'e tümleyende −5 = 1011." → **Doğrusu:** 1'e tümleyende −5 = `1010`; `1011` 2'ye tümleyendir. → **Neden:** İki yöntem tam bir "+1" farkıyla ayrılır.
- **Yanlış:** "4 bitlik 2'ye tümleyen −7...+7'dir." → **Doğrusu:** −8...+7. → **Neden:** Tek sıfır olduğu için bir örüntü artar ve negatif tarafa gider (`1000`).
- **Yanlış:** 8 bitlik −3'ü `0000 1101` yazmak. → **Doğrusu:** `1111 1101`. → **Neden:** Negatif sayıyı genişletirken sola işaret bitinin kopyası (1) eklenir.

⚡ HIZLI TEKRAR

- Bit örüntüsünün anlamı gösterime bağlıdır; önce gösterimi oku.
- **SM:** işaret biti + büyüklük. **1C:** her biti ters çevir. **2C:** ters çevir + 1 $(= 2^n - x)$.
- SM ve 1C'de iki sıfır, simetrik aralık; 2C'de tek sıfır, $-2^{n-1} \dots 2^{n-1} - 1$.
- 2C'de $A - B = A + (-B)$: aynı toplayıcı; dış elde atılır.
- Taşma: aynı işaretliler toplanıp ters işaret çıkarsa. Farklı işaretlilerde asla.
- **Ezberle:** üç kural ve aralıklar. **Türetebil:** aralık formüllerinin nedeni, $\bar{x} + 1 = 2^n - x$.

Bu bölümden sonra şunları yapabiliyor olmalısın: Herhangi bir 4 veya 8 bitlik örüntüyü üç gösterimde okumak; negatif sayı yazmak; 2C'de çıkarma yapmak; taşmayı gerekçesiyle tespit etmek; 2C'nin üstünlüklerini açıklamak.

1.9 Bölüm cevapları

D1.1 `1000`: işaretsiz 8; işaret-büyüklük −0 (işaret 1, büyüklük 000); 1'e tümleyen −7 (ters çevir: `0111` = 7); 2'ye tümleyen −8 (ters çevir + 1 = `1000` = 8 işaretsiz → −8).

D1.2 `1000` → ters çevir `0111` → +1 → `1000`. Yine kendisi çıkıyor. Mantıklı, çünkü +8 4 bitlik aralığa sığmıyor (en büyük +7). $-(-8) = +8$ hesabı aslında bir taşma; sonuç çemberde bir tur dönüp −8'e düşüyor.

D1.3 Taşma yok. Farklı işaretli iki sayının toplamı iki sayının arasında kalır, aralık dışına çıkamaz. Dış elde 1 olması (ör. 58 − 40) normaldir ve atılır.

Y1.1 -37 = `1101 1011`. $0001\,1001 + 1101\,1011 = 1111\,0100$. Dış elde yok. En soldaki bit 1 → ters çevir `0000 1011`, +1 → `0000 1100` = 12 → **−12**. Gerçekten $25 - 37 = -12$. ✔

K1.1 -100: `0110 0100` → `1001 1011` → `1001 1100`. -50: `0011 0010` → `1100 1101` → `1100 1110`. Toplam: $1001\,1100 + 1100\,1110 = 1\,0110\,1010$. Dış elde atılınca `0110 1010` = **+106**. İki negatifin toplamı pozitif çıktı → **taşma var**. Gerçek sonuç −150 aralığın (-128) dışında; $-150 + 256 = 106$.
✔ python3 ile doğrulandı

K1.2 6 bit 2'ye tümleyen: $-2^5 \dots 2^5 - 1 = -32 \dots +31$. 6 bit işaret-büyüklük: $-(2^5 - 1) \dots + (2^5 - 1) = -31 \dots +31$.

K1.3 `1101`: SM → işaret 1, büyüklük `101` = 5 → **−5**. 1C → ters çevir `0010` = 2 → **−2**. 2C → ters çevir `0010`, +1 → `0011` = 3 → **−3**.

Nasıl soru gelir? kutusundaki cevaplar: `1110 1001` → ters `0001 0110` → +1 `0001 0111` = 23 → −23. $75 + 60$: `0100 1011` + `0011 1100` = `1000 0111` → −121, iki pozitif → negatif → taşma (gerçek 135 > 127). 10 bit: $-512 \dots 511$.

BÖLÜM 2

Veri ve Sinyal: Bitler Kabloda Nasıl Yolculuk Eder?

◆ **BU BÖLÜMDE** Okuma süresi: 45 dk Zorluk: ●●○

- Veri ile sinyalin neden aynı şey olmadığını
- Analog ve sayısal sinyalin farkını; dört veri-sinyal dönüşümünü (ses, bilgisayar verisi, modem, codec)
- Modülasyonun (genlik, frekans, faz) ne yaptığını
- Çok seviyeli sinyalleri ve **bit hızı = sembol hızı × $\log_2(M)$** formülünü, onu türeterek

SINAV ÖNCELİĞİ: ORTA ❖ Kaynak: D1 s. 26–34

Bu bölüm sayısal bir sınavda hesap olarak yalnızca bir formülle (bit hızı) yoklanır; ama sonraki iki bölümün (kanal bozulmaları ve hat kodlama) sözlüğünü burada kuruyoruz. Tanımları ve dört dönüşümü sözlü soruya hazırlıklı bil.

→ BU KONUYA NEDEN GEÇİYORUZ?

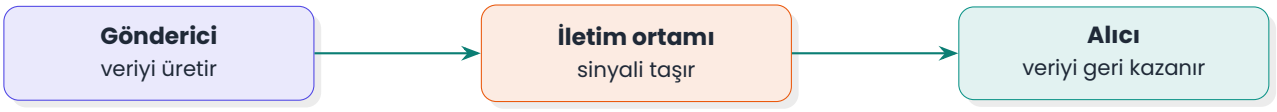
1. bölümde `1111 0110` örüntüsünün ne *anlama* geldiğini konuştuk. Ama bir kablo "−10" nedir bilmez; kabloda sadece voltaj, ışık ya da radyo dalgası vardır. Bu bölüm şu soruyu cevaplıyor: **bir bit, fiziksel dünyada neye dönüşür?** Bu cevabı bilmeden bir sonraki bölümdeki bozulmaları (neden `1` bazen `0` okunuyor?) anlamak mümkün değil.

↺ HATIRLATMA

Frekans ve periyot: Bir olay saniyede f kez tekrar ediyorsa bir tekrarın süresi $T = 1/f$'dir. Saniyede 50 000 kez tekrarlanan bir olayın periyodu $1/50\,000 = 0.00002$ s $= 0.02$ ms'dir. Birimler: 1 ms (milisaniye) $= 10^{-3}$ s; 1 kHz = 1000 Hz.
Logaritma ($\log_2$): $\log_2(M)$, "2'yi kaç kez kendisiyle çarparsam M olur?" sorusunun cevabıdır. $\log_2 2 = 1$, $\log_2 4 = 2$, $\log_2 8 = 3$, $\log_2 16 = 4$. Başka bir deyişle $M = 2^k$ ise $\log_2 M = k$.

2.1 Haberleşmenin üç parçası ❖ Kaynak: D1 s. 27

İki insanın telefonla konuştuğunu düşün. Birisi konuşur (gönderici), ses telefon hattından geçer (iletim ortamı), diğeri dinler (alıcı). Slayt bütün haberleşme sistemlerini bu üç parçaya indiriyor:



Şekil 2.1: Haberleşme modeli (D1 s. 27)

Slayttaki alt kutu önemli: **protokol** (**protocol**), tarafların biçim, sıra, zamanlama ve hata davranışı konusunda anlaştığı ortak kurallardır. İki taraf aynı dili konuşmuyorsa en iyi kablo bile işe yaramaz.

2.2 Veri ≠ sinyal

❖ Kaynak: D1 s. 28

Bu ayrımı sınavda tanım olarak yazmanı isteyebilirler. Mektup benzetmesiyle düşün: **mektubun içindeki yazı** veridir, **mektubu taşıyan kamyon** sinyaldir.

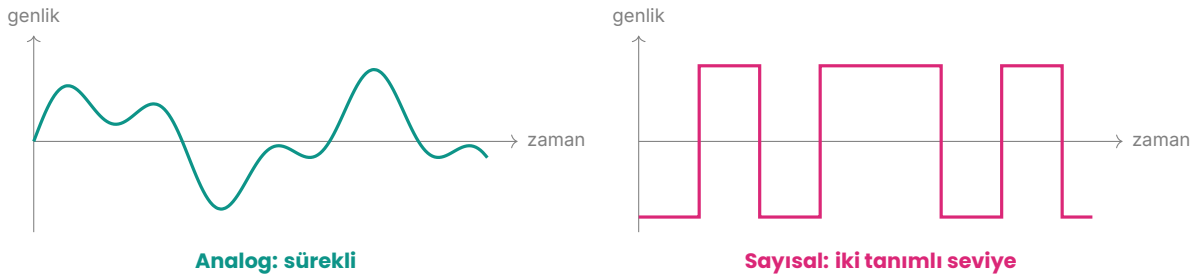
	Veri (data)	Sinyal (signal)
Tanım	Taşınacak ya da işlenecek bilgi	Veriyi fiziksel ortamda temsil eden enerji değişimi
Örnekler	Metin, sayı, görüntü, ses, komut	Elektriksel, optik ya da elektromanyetik değişim
Benzetme	Mektubun içeriği	Mektubu taşıyan araç

Aynı veri farklı sinyallerle taşınabilir (aynı cümleyi telefonla da, mektupla da iletebilirsin); aynı tür sinyal de farklı verileri taşıyabilir.

2.3 Analog mu, sayısal mı?

❖ Kaynak: D1 s. 29

Slaytın tanımı: **analog sinyal** hem zamanda hem genlikte süreklidir; **sayısal sinyal** ise tanımlı birkaç seviye arasında değişir.



Şekil 2.2: Analog (soldaki) ve sayısal (sağdaki) dalga biçimi (D1 s. 29)

Fark ne işe yarıyor? Analog sinyalde **her değer anlamlıdır**: 3.14 V ile 3.15 V farklı seslerdir. Bu yüzden araya giren en küçük gürültü bile bilgiyi değiştirir. Sayısal sinyalde ise alıcı sadece "yüksek mi, alçak mı?" diye sorar; 4.9 V ile 5.1 V'u aynı seviye sayar. Bu dayanıklılığı 3. bölümde göreceğiz.

✎ **DURUP DÜŞÜN**

D2.1 Bir termometredeki cıva sütunu analog mu, sayısal mı? Dijital termometrenin ekranı?

2.4 Dört dönüşüm: veriden sinyale ❖ Kaynak: D1 s. 30–32

Veri analog ya da sayısal olabilir; sinyal de analog ya da sayısal olabilir. $2 \times 2 = 4$ kombinasyon çıkıyor. Slaytlar bunları üç örnekle gösteriyor, dördüncüsünü slayt 32'nin şeması tamamlıyor.

2.4.1 Ses: analog veriden analog sinyale (D1 s. 30)

Mikrofon, hava basıncındaki değişimleri **benzer biçimli** elektriksel değişimlere çevirir. Ses yükselirse voltaj yükselir, perde çıkarsa dalga sıklaşır. Telefon hattında taşınan şey sesin elektriksel "kopyası"dır.

2.4.2 Bilgisayar verisi: sayısaldan sayısala (D1 s. 31)

Klavyeden yazdığın metin ikilik rakamlara çevrilir. Slayttaki örnekte:

- İkilik **1**, **−5 V** ile; ikilik **0**, **+5 V** ile temsil ediliyor.
- Her bitin süresi **0.02 ms** ($= 0.02 \times 10^{-3}$ s).
- Sonuç: veri hızı $= 1/0.00002$ s $= 50\,000$ bit/s $= 50$ kbps.

▶ ÖRNEK 2.1 — Slayt örneği [D1 s. 31]: 0.02 ms'lik bit, hız kaç?

Verilen: Her bit 0.02 ms süreli. **İstenen:** bit hızı ve 10 bitlik bir mesajın süresi.

1. Bir bit 0.02 ms $= 2 \times 10^{-5}$ s sürüyor.
2. Saniyede $1/(2 \times 10^{-5}) = 50\,000$ bit gönderilir → **50 kbps**.
3. 10 bit için süre: $10 \times 0.02 = 0.2$ ms.

Sonuç: 50 kbps; slayttaki 10 bitlik dizi 0.2 ms'de gider. ✔ python3 ile doğrulandı

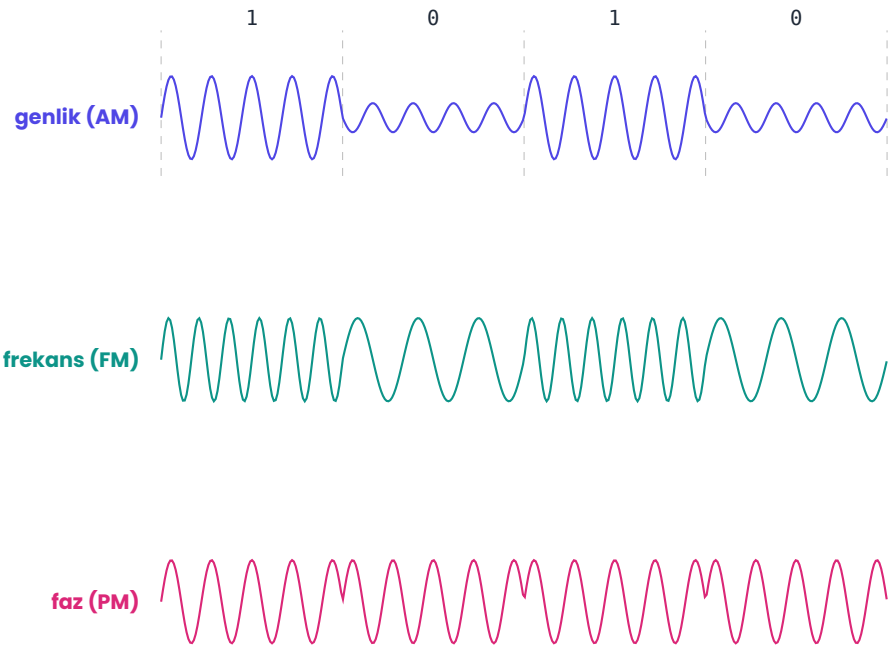
✗ SIK YAPILAN HATA — Kaynak notu: slaytta Ð5 volts yazıyor

D1 s. 31'deki metinde ikilik 1'in voltajı "Ð5 volts" diye bozuk karakterle çıkıyor. Şekilden ve mantıktan bu değer **−5 V**'tur (şekilde alçak seviye). Hocaya da soracağız (bkz. Hocaya Sorulacaklar).

2.4.3 Analog taşıyıcı: modülasyon (D1 s. 32)

Peki bir telefon hattı yalnızca ses frekanslarında (analog) çalışıyorsa, bilgisayarın bitlerini bu hattan nasıl göndereceğiz? Bitleri doğrudan hatta veremeyiz. Çözüm: sürekli ve düzenli bir dalga (**taşıyıcı**, **carrier**) alıp özelliklerini veriye göre değiştirmek. Buna **modülasyon** (**modulation**) denir. Üç özellik değiştirilebilir:

- **Genlik** (**amplitude**): dalganın yüksekliği
- **Frekans** (**frequency**): saniyedeki tekrar sayısı
- **Faz** (**phase**): dalganın başlangıç noktası (kayma)



Şekil 2.3: Aynı 1-0-1-0 verisi üç modülasyonla: 1 iken yüksek genlik, yüksek frekans ya da ters olmayan faz; 0 iken alçak genlik, alçak frekans ya da 180° kaymış faz (örnek çizimdir, slaytta yok).

Slayt 32'nin şeması dört kombinasyonu cihaz adlarıyla gösteriyor:

Veri	Sinyal	Cihaz (slayt)	Ne yapar?
Analog	Analog	Telefon	Sesi benzer biçimli elektrik dalgasına çevirir
Sayısal	Analog	Modem	Bitleri taşıyıcı dalganın genlik, frekans ya da fazına gömer (modüle eder)
Analog	Sayısal	Codec	Analog sinyali sayısal bit akışına çevirir (5. bölümdeki PCM)
Sayısal	Sayısal	Sayısal alıcı-verici (**digital transceiver**)	Bitleri uygun voltaj seviyelerine çevirir (4. bölümdeki hat kodlama)

→ BU KONUYA NEDEN GEÇİYORUZ?

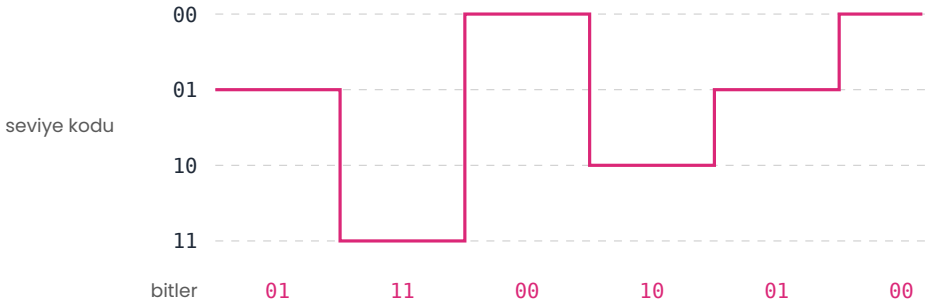
Tablonun üçüncü satırı (**codec**) bu kitabın 5. bölümüdür: sesi örnekleyip bitlere çevirmek (PCM). Dördüncü satır ise 4. bölümdür: bitleri tele hangi voltaj kurallarıyla döküyoruz (hat kodlama).

✎ DURUP DÜŞÜN

D2.2 Wi-Fi'ın (kablosuz ağ) bilgisayar verisini taşıması, tablodaki hangi satıra girer?

2.5 İki seviyeli ve çok seviyeli sinyaller ❖ Kaynak: D1 s. 33–34

Şimdiye kadar iki seviye vardı: yüksek ve alçak. Her seviye 1 bit taşıyor. Peki seviye sayısını artırırsak? Mesela dört voltaj seviyesi kullanalım: her seviye iki bit taşır.



Şekil 2.4: 4 seviyeli sinyal: her sembol (bir seviyede geçirilen süre) 2 bit taşır; 6 sembolle 12 bit gider (örnek çizimdir).

∑ TANIM / FORMÜL — Sembol, sembol hızı ve bit hızı

Sembol (**symbol**): sinyalin sabit kaldığı en küçük birim; bir seviye ya da bir değişim biçimi.
Sembol hızı (**symbol rate**, birimi **baud**): saniyede kaç sembol gönderildiği.
M: ayrı sinyal seviyesi (ya da sembol) sayısı.
Bit hızı (**bit rate**, bit/s): saniyede kaç bit gönderildiği.

$$\text{Bit hızı} = \text{Sembol hızı} \times \log_2(M)$$

Slayttaki örnekler (D1 s. 34): $M = 2 \rightarrow 1$ bit/sembol; $M = 4 \rightarrow 2$ bit/sembol; $M = 8 \rightarrow 3$ bit/sembol.

2.5.1 Formül nereden geliyor?

M farklı seviyeyi ayırt edebiliyorsak, her sembol M farklı mesajdan birini seçiyor demektir. M farklı seçeneği bit cinsinden yazmak için kaç bit lazım? k bit 2^k farklı örüntü verir (1. bölümdeki hatırlatma). $2^k \geq M$ olması gerek; M tam 2'nin kuvveti ise $k = \log_2 M$. Örneğin $M = 4$ seviye için $2^2 = 4$ → 2 bit: `00, 01, 10, 11`.

Her sembol $\log_2 M$ bit taşıyorsa, saniyede S sembol gönderince saniyede $S \times \log_2 M$ bit gider.

▶ ÖRNEK 2.2 — Ek örnek: seviye sayısı hızı nasıl değiştirir?

Kanalımız saniyede **1000 sembol** (1000 baud) taşıyabiliyor.

M	$\log_2 M$	Bit hızı
2	1	1000 × 1 = **1000 bit/s**
4	2	1000 × 2 = **2000 bit/s**
8	3	1000 × 3 = **3000 bit/s**
16	4	1000 × 4 = **4000 bit/s**

✔ python3 ile doğrulandı

Ters soru: 9600 bit/s'yi 16 seviyeli sinyalle göndermek için sembol hızı kaç baud olmalı?
$9600 = S \times 4 \rightarrow S = 2400$ baud.

▶ ÖRNEK 2.3 — Ek örnek: slayt 31'in hızını iki katına çıkar

Slayt 31'de $M = 2$, her sembol 0.02 ms → sembol hızı 50 000 baud ve bit hızı 50 kbps. Aynı sembol süresiyle $M = 4$ kullanırsak: $50\,000 \times \log_2 4 = 100\,000$ bit/s $= 100$ kbps. Hız ikiye katlandı; kablo, sembol süresi aynı kaldı.

2.5.2 Bedeli: seviyeler birbirine yaklaşır

Alt kutuda uyarı var: *"More levels can increase data rate, but noise tolerance may decrease."* Bunu bir sayıyla görelim. Alıcıya ulaşabilen toplam voltaj aralığı sabit, diyelim −5 V ile +5 V arası (10 V genişlik).

- **2 seviye:** −5 V ve +5 V; aralarındaki mesafe 10 V.
- **4 seviye:** −5, −1.67, +1.67, +5 V; komşu seviyeler arası mesafe $10/3 \approx 3.33$ V.
- **8 seviye:** komşu seviyeler arası $10/7 \approx 1.43$ V.

Gürültü sinyale ±1.5 V ekleyebiliyorsa, 2 ve 4 seviyeli sinyal hâlâ doğru okunur (hatta 4 seviyede 3.33 V'luk mesafenin yarısı 1.67 V, sınıra yakın), ama 8 seviyede 1.43 V'luk mesafenin yarısı 0.71 V olduğu için gürültü seviyeyi komşusuna atlatır. ✔ python3 ile doğrulandı

⚠ SINAVDA DİKKAT

Hız–dayanıklılık dengesi hem sınavda hem hayatta tekrar eden bir temadır: daha fazla seviye = daha yüksek bit hızı ama gürültüye daha az dayanıklılık. Bu cümleyi kendi kelimelerinle yazabilmelisin.

✎ KENDİN ÇÖZ — Slayt sorusu [D1 s. 33]: What are the values as Volts?

Slayt 33'te üst şekil, iki seviyeli bir sinyali ve altındaki bit dizisini (`1 0 0 1 0 1 1`) gösteriyor. Alttaki şekil (f(t)), analog bir eğriyi 4–5 seviyeli kırmızı bir basamak sinyaline yaklaştırıyor ve "değerler kaç volt?" diye soruyor.
Şekilde voltaj ölçeği çizilmediği için kesin değerler verilemez. Hocanın beklediği büyük olasılıkla şu okuma: her kırmızı basamağın yüksekliğini, ızgara çizgilerinden (aynı aralıklı seviyeler) okuyup bir sayıya çevirmek; ızgara çizgisi başına bir birim voltaj ve her basamak için o çizgiye en yakın seviye. Bu soruyu hocaya soracağız (bkz. Hocaya Sorulacaklar).
Konunun özü: analog eğriyi sabit seviyelere yuvarlamak, 5. bölümdeki **nicemleme**dir.

✎ KENDİN ÇÖZ — Kendin çöz

K2.1 Bir kanalda 4 seviyeli sinyal saniyede 2400 sembol gönderiyor. Bit hızı kaçtır?
K2.2 3 bit/sembol taşımak için en az kaç seviye gerekir? Bu seviyeler 10 V'luk aralığa eşit yerleştirilirse komşu seviyeler arası mesafe kaç V olur?
K2.3 20 kbps'lik bir akışı 2 seviyeli sinyalle göndermek ile 16 seviyeli sinyalle göndermek arasında sembol hızı farkı nedir?

2.6 Bağlantılar

- **Önceki bölüm → bu bölüm:** Bit örüntüsünün anlamı (işaretli sayılar) bu bölümde anlamsız; tel sadece voltaj taşır. Bitleri voltaja çeviren kurallar 4. bölümde, analog sesi bite çeviren kurallar

5. bölümde.

- **Bu bölüm → sonraki bölüm:** Sayısal sinyaller seviyeleri "ayırt etme" üzerine kurulu; kanal bu ayrımı nasıl bozar? Cevap 3. bölümde: zayıflama, bozulma, gürültü.
- **İleride:** Haberleşme Sistemleri ve Bilgisayar Ağları derslerinde modülasyon (AM, FM, QAM) ve Shannon kapasitesi bu bölümün devamıdır.

⚠ SINAVDA DİKKAT

- **Veri ≠ sinyal.** Veri bilgidir; sinyal onu taşıyan fiziksel değişimdir.
- **Bit hızı ≠ sembol hızı.** Bit hızı = sembol hızı × $\log_2(M)$; 2 seviyede ikisi eşittir, daha fazla seviyede bit hızı büyüktür.
- **Dört kombinasyonu** cihaz adlarıyla bil: telefon, modem, codec, sayısal alıcı-verici.
- **Seviye artınca hız artar, gürültü dayanıklılığı azalır.**

❖ BU KONUDAN NASIL SORU GELİR?

1. Tanım: *"Distinguish between data and signal."* İpucu: tanım + örnek. Veri bilgi, sinyal fiziksel temsil.
2. Hesap: *"A system uses 8 signal levels and sends 3000 symbols per second. What is the bit rate?"* İpucu: formül. ($3000 \times 3 = 9000$ bit/s)
3. Karşılaştırma: *"What is the trade-off of using more signal levels?"* İpucu: hız artar, ayırt etme zorlaşır, gürültü toleransı düşer.
4. Eşleştirme: Telefon, modem, codec, sayısal alıcı-verici hangi veri→sinyal kombinasyonuna karşılık gelir?

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "Kablo sayısal olduğu için sinyal de sayısaldır." → **Doğrusu:** Ortam sinyalin türünü tek başına belirlemez; aynı kablo analog ya da sayısal sinyal taşıyabilir (D1 s. 43). → **Neden:** Sinyali ortam değil, vericinin ürettiği dalga biçimi belirler.
- **Yanlış:** "$M = 8$ seviye 8 bit taşır." → **Doğrusu:** 3 bit taşır. → **Neden:** $\log_2 8 = 3$; 8 seviye sadece 8 farklı mesajdan birini seçer.
- **Yanlış:** "Bit hızı ile baud aynı şeydir." → **Doğrusu:** Yalnızca $M = 2$ iken eşittir. → **Neden:** Çok seviyeli sinyalde bir sembol birden çok bit taşır.

⚡ HIZLI TEKRAR

- Gönderici → iletim ortamı → alıcı; protokol ortak kurallardır.
- Veri = bilgi; sinyal = onu taşıyan fiziksel değişim.
- Analog: zamanda ve genlikte sürekli; sayısal: tanımlı seviyeler.
- Dört dönüşüm: telefon (analog→analog), modem (sayısal→analog), codec (analog→sayısal), sayısal alıcı-verici (sayısal→sayısal).
- Modülasyon: taşıyıcının genliği, frekansı ya da fazı veriye göre değiştirilir.
- **Bit hızı = sembol hızı × $\log_2(M)$.** Seviye artınca hız artar, gürültü dayanıklılığı azalır.
- Slayt 31: 0.02 ms/bit → 50 kbps.

Bu bölümden sonra şunları yapabiliyor olmalısın: Veri ve sinyali ayırmak; dört dönüşümü cihazlarla eşleştirmek; formülü türetmek ve hesap sorusunda kullanmak.

2.7 Bölüm cevapları

D2.1 Cıva sütunu analog: yükseklik her değeri alabilir. Dijital termometre ekranı ise sayısal: görüntülediği değer (ör. 36.6) sonlu basamaklı, ayrık değerlerdir (ama ölçme öncesi sensör analogdur).

D2.2 Wi-Fi, bilgisayarın sayısal verisini radyo dalgasıyla (analog taşıyıcı) taşır: **sayısal veri → analog sinyal** satırı (modem benzeri modülasyon).

K2.1 $2400 \times \log_2 4 = 2400 \times 2 = 4800$ bit/s.

K2.2 $2^3 = 8$ seviye. 10 V aralığına 8 seviye eşit yerleşirse 7 aralık oluşur: $10/7 \approx 1.43$ V.

K2.3 2 seviye: sembol hızı = 20 000 baud; 16 seviye: $20\,000/4 = 5000$ baud. Fark 15 000 baud (16 seviyeli sinyal 4 kat daha yavaş sembol gönderiyor).

Sınav sorusu cevabı: (2) 8 seviye → 3 bit/sembol → $3000 \times 3 = 9000$ bit/s.

BÖLÜM 3

Kanal Bozulmaları ve İletim Ortamları

◆ **BU BÖLÜMDE** Okuma süresi: 40 dk Zorluk: ●●○

- Gönderilen sinyalin alıcıya neden aynen ulaşmadığını: zayıflama, bozulma ve gürültü
- Sinyal-gürültü oranının (SNR) neyi anlattığını
- Bakır kablo, fiber optik ve kablosuz ortamın özelliklerini ve ne zaman hangisinin seçildiğini
- Slayttaki kablo uzunluğu rakamlarını ve bu rakamlardaki tuzakları

SINAV ÖNCELİĞİ: ORTA ❖ Kaynak: D1 s. 35–43

Bu bölümde hesap az, kavram çok. Sınavda üç bozulmanın tanımı ve farkı ile ortam karşılaştırması klasik soru olur; slayt 43'ün alt kutusu ("ortam tek başına analog ya da sayısal belirlemez") özellikle çoktan seçmeli için güzel bir tuzak.

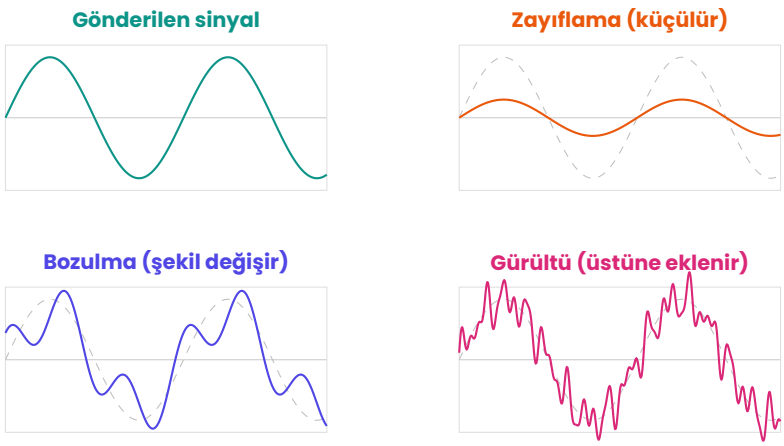
→ BU KONUYA NEDEN GEÇİYORUZ?

2. bölümde sinyali kabloya verdik. Peki alıcı o sinyali **aynen** alıyor mu? Hayır. Kanal, sinyale üç şey yapabilir: **zayıflatır**, **şeklini değiştirir** ya da **üstüne kirli bir şey ekler**. Hat kodlamanın (4. bölüm) neden bu kadar titiz tasarlandığını, bu bozulmaları anlamadan kavrayamazsın.

3.1 Yolda ne olur? Üç bozulma ❖ Kaynak: D1 s. 36–38

Bir mektup benzetmesi: bir arkadaşına bağırarak mesaj gönderiyorsun. Uzaklaştıkça ses **kısılır** (zayıflama). Rüzgâr ve yankı yüzünden sözcüklerin biçimi **bozulur** (bozulma). Yoldan geçen arabaların gürültüsü araya **karışır** (gürültü). Slaytlar tam bu üç bozulmayı işliyor.

Bozulma	Ne olur?	Slayt cümlesi	Çözüm yolu
Zayıflama (**attenuation**)	Sinyal enerjisi mesafeyle azalır	Signal energy decreases with distance	Yükselteç ya da tekrarlayıcı
Bozulma (**distortion**)	Sinyalin şekli değişir	Output shape changes when frequency components are affected at different rates or delays	Kanalı iyi tasarlamak, denkleştirme (ek bilgi)
Gürültü (**noise**)	Araya istenmeyen enerji karışır	Unwanted energy is added to the signal	Güçlü sinyal, korumalı kablo, sağlam kodlama



Şekil 3.1: Aynı sinyal, üç farklı bozulma. Gri kesikli çizgi gönderilen sinyal. Zayıflamada genlik küçülür, bozulmada şekil değişir, gürültüde üstüne rastgele dalgalanma biner (örnek çizimdir; D1 s. 36-38'deki fikrin sade hâli).

3.1.1 Zayıflama (D1 s. 36)

Enerji, yol boyunca ısıya dönüşür ve sinyal küçülür. Telefonunda Wi-Fi çubuklarının modemden uzaklaştıkça azalması aynı şeydir. Alıcı çok küçük bir sinyali artık gürültüden ayıramayabilir. Çare: yol üzerine **yükselteç** (**amplifier**) ya da **tekrarlayıcı** (**repeater**) koymak.

+ EK BİLGİ (KAYNAK DIŞI) — Yükselteç ile tekrarlayıcı aynı şey değil

Slayt ikisini birlikte anıyor. Fark şu: analog yükselteç sinyali **olduğu gibi büyütür**; içindeki gürültü de büyür. Sayısal tekrarlayıcı ise bitleri **yeniden okuyup temiz bir kopya üretir**; gürültü her istasyonda temizlenir. Sayısal sistemlerin uzun mesafede üstün olmasının nedeni budur.

+ EK BİLGİ (KAYNAK DIŞI) — Zayıflama desibel (dB) ile ölçülür

Slaytlarda yok ama kitaplarda hep geçer: $L(\text{dB}) = 10 \log_{10}(P_{\text{giriş}}/P_{\text{çıkış}})$. Güç yarıya inerse yaklaşık 3 dB kayıp var demektir.
Örnek: 100 mW gönderiyorsun, 1 mW ulaşıyor. Oran 100, $\log_{10} 100 = 2$ → kayıp 20 dB.
✔ python3 ile doğrulandı

3.1.2 Bozulma (D1 s. 37)

Slayttaki şekilde girişte düzgün bir sinüs, çıkışta biçimi bozulmuş bir dalga var. Neden? Gerçek sinyaller tek bir frekanstan oluşmaz; farklı frekansların toplamıdır. Kanal bu bileşenlere **farklı hızlarda** ya da **farklı gecikmelerle** davranırsa, toplam dalganın şekli değişir.

Orkestra benzetmesi: davul, keman ve flüt aynı anda çalıyor. Ama flütün sesi salona 1 saniye geç ulaşırsa, parça tanınmaz hâle gelir. Hiçbir enstrüman gürültü eklemedi, hiçbiri susmadı; yalnızca zamanlama bozuldu.

3.1.3 Gürültü (D1 s. 38)

Sinyal **dışından** gelen istenmeyen enerji. Slayttaki şekilde çıkış noktaları rastgele dağılıyor: sinyalin orijinal biçimi hâlâ seçiliyor ama saçılmış. Sonuç: *"making levels and timing harder to distinguish"*; seviyeleri ve zamanlamayı ayırt etmek zorlaşır.

+ EK BİLGİ (KAYNAK DIŞI) — Gürültü türleri (slayt dışı)

- **Isıl gürültü** (**thermal noise**): iletkendeki elektronların rastgele hareketi; kaçınılamaz.
- **Çapraz karışma** (**crosstalk**): komşu kablodaki sinyalin bu kabloya sızması (D1 s. 41'de geçiyor).
- **Dürtü gürültüsü** (**impulse noise**): şimşek, motor kıvılcımı gibi kısa ve şiddetli parazitler.

3.1.4 Üçü yan yana: kafa karıştıranlar

Soru	Zayıflama	Bozulma	Gürültü
Sinyal büyüklüğü değişir mi?	Evet, küçülür	Biçimi değişir	Üstüne eklenir
Dışarıdan bir şey eklenir mi?	Hayır	Hayır	**Evet**
Mesafeyle ilişkisi	Doğrudan artar	Kanalın frekans davranışına bağlı	Mesafeden bağımsız da olabilir
Basit benzetme	Ses kısılır	Ses tonu bozulur	Arka plan gürültüsü

3.2 Sinyal-gürültü oranı ❖ Kaynak: D1 s. 39

Bozulmaların asıl önemi, **sinyal gücü ile gürültü gücünün oranına** bağlıdır. Slayt iki durumu karşılaştırıyor:

	Yüksek SNR	Düşük SNR
Durum	İstenen sinyal gürültüden açıkça güçlü	Gürültü sinyale yaklaşır ya da onu örter
Sonuç	Alıcı büyük olasılıkla doğru karar verir	Hata olasılığı artar; daha sağlam kodlama gerekebilir

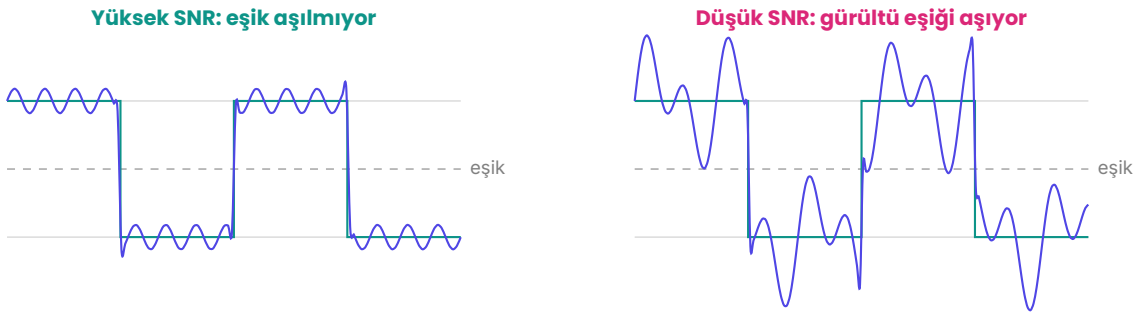
Kalabalık bir kafede konuşmayı düşün: sessiz bir masada fısıldaşsan da anlaşırsın (yüksek SNR); gürültülü bir konserde bağırsan bile anlaşılmazsın (düşük SNR).

+ EK BİLGİ (KAYNAK DIŞI) — SNR sayıyla ve dB ile

$\text{SNR} = P_{\text{sinyal}} / P_{\text{gürültü}}$. Desibelde: $\text{SNR}_{\text{dB}} = 10 \log_{10}(\text{SNR})$.
Örnek: sinyal gücü 1 mW, gürültü gücü 0.01 mW ise $\text{SNR} = 100$; $10 \log_{10} 100 = 20$ dB. Güç 10 katına çıkınca +10 dB. ✔ python3 ile doğrulandı

3.2.1 SNR neden hata yaratır? Eşik fikri

Sayısal bir alıcı çoğunlukla basit bir kural kullanır: gelen voltaj orta eşiğin üstündeyse `1`, altındaysa `0`. Gürültü sinyale eklenince, bir bitin voltajı eşiğin öbür tarafına geçebilir.



Şekil 3.2: Turkuaz: gönderilen sinyal; mor: gürültülü gelen sinyal. Sağdaki durumda gürültü tepeleri eşiğin öbür tarafına geçtiği için bazı bitler yanlış okunur (örnek çizimdir, slaytta yok).

Çözüm: kablonun korumasını artırmak, sinyali güçlendirmek, ya da daha sağlam kodlama kullanmak (slayt: *"more robust encoding may be needed"*). Hat kodlama (4. bölüm) bu "sağlam kodlama"nın bir parçasıdır.

✎ DURUP DÜŞÜN

D3.1 Çok seviyeli sinyalde (2. bölüm), aynı gürültü altında 2 seviyeli ve 8 seviyeli sinyalden hangisinde hata olasılığı daha yüksektir? Neden?

3.3 İletim ortamları

❖ Kaynak: D1 s. 40–43

Sinyalin yolculuk yaptığı yer üç ana aileye ayrılıyor:

Ortam	Taşıdığı	Güçlü yanı	Zayıf yanı (ek yorum)
Bakır kablo (**copper**)	Elektrik sinyali; bükümlü çift (**twisted pair**) ya da koaksiyel	Ucuz, yaygın, kurulumu kolay	Mesafe sınırlı; elektriksel girişime açık
Fiber optik (**optical fiber**)	Işık darbeleri	Yüksek kapasite, elektromanyetik girişime dayanıklı	Kurulum ve uç cihazlar pahalı
Kablosuz (**wireless**)	Radyo ve mikrodalga	Hareketliliği destekler	Ortam paylaşılır; engeller ve girişim kaliteyi düşürür

3.3.1 Bükümlü çift (D1 s. 41)

Slayt başlığı bükümlü çifti (**twisted pair**) anıyor; alt kutusu ise nedenini söylüyor: *Twisting the pairs helps reduce external interference and crosstalk between neighboring pairs.* Yani çiftlerin birbirine sarılması, dış girişimi ve komşu çiftlerle çapraz karışmayı azaltır.

+ EK BİLGİ (KAYNAK DIŞI) — Büküm neden işe yarıyor?

Dışarıdan gelen bir girişim iki tele neredeyse **aynı** miktarda bulaşır. Alıcı iki tel arasındaki **farka** baktığı için bu ortak gürültü birbirini götürür. Teller ne kadar sık bükülürse, iki telin gürültüyü "aynı" alma ihtimali o kadar yükselir.

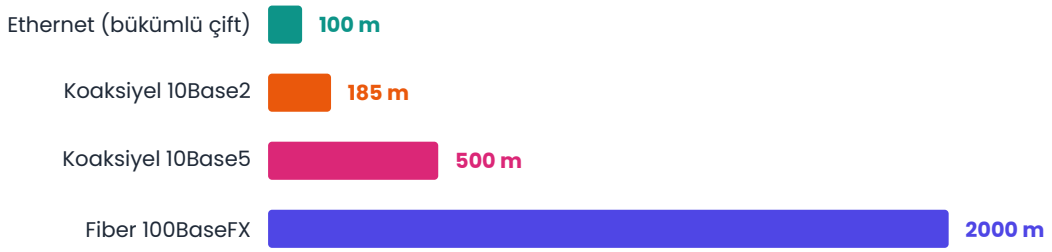
3.3.2 Koaksiyel ve fiber (D1 s. 41--42)

Koaksiyel kablo: ortada bakır tel, etrafında yalıtkan, onun etrafında bakır örgü (**copper mesh**) ve en dışta yalıtkan kılıf. Örgü koruma sağlar (D1 s. 42: *"Coaxial construction provides shielding"*).

Fiber: elektrik yerine **ışıkla** veri taşır. Elektromanyetik girişimden etkilenmez.

Slayt 41'in metin kutusundaki sayılar, sınavda ezber sorusu olabilir:

Kablo	Slayt 41'deki bilgi
Koaksiyel	En fazla uzunluk 10Base5'te 500 m, 10Base2'de 185 m. Kablo TV'de kullanılır.
Ethernet (bükümlü çift)	İki cihaz arası en fazla 100 m. Daha uzun gerekirse ağ cihazı eklenir. LAN kabloları.
Fiber	Aydınlatma (ışık) kullanır. 100BaseFX standardında 2000 m'ye kadar. Kıtalar arası kablolar.



Şekil 3.3: Slayt 41'de verilen en büyük kablo uzunlukları (D1 s. 41). Çubuk uzunluğu metreyle orantılı.

✗ SIK YAPILAN HATA — Slayt 41'de dikkat edilecek üç nokta

1. **Başlık içerikle uyuşmuyor:** Slayt "Twisted-pair cable" başlığını taşıyor ama metin koaksiyel, Ethernet ve fiber uzunluklarını veriyor. Sınavda *kablo adı ile uzunluk* eşleştirirken karışmasın.
2. **"Amplifier" ifadesi:** Slayt, 100 m'den uzun Ethernet için "amplifier gibi ağ cihazları" diyor. Sayısal Ethernet hatlarında bu iş için genellikle **repeater ya da switch** kullanılır; yükselteç analog sistemin terimidir (yukarıdaki kutuya bak).
3. **"Işık evrendeki en hızlı şey":** Boşlukta doğru. Fiber içinde ışık, boşluktaki hızının yaklaşık **2/3'ü** ile gider.

3.3.3 Kablosuz (D1 s. 43)

Slayt dört madde veriyor:

- Sinyal, iletkende değil **uzayda elektromanyetik dalga** olarak yayılır.
- Aynı spektrum **birçok cihaz tarafından paylaşılabilir**.
- Mesafe, engeller, yansımalar, hareket ve diğer vericiler kanal kalitesini etkiler.
- Hem kablolu hem kablosuz ortam, analog ya da sayısal sinyal taşıyabilir.

Alt kutu: *"The medium alone does not determine whether a signal is analog or digital."* Yani **ortam**, sinyalin analog mu sayısal mı olduğunu tek başına belirlemez. Fiber de, bakır da, kablosuz da iki tür sinyali taşıyabilir.

+ EK BİLGİ (KAYNAK DIŞI) — Aynı ortamda iki farklı sinyal örneği

Eski telefon hattı (bakır) hem sesi analog sinyalle (klasik telefon), hem de bilgisayarın bitlerini analog taşıyıcı üzerinde (modem) taşırdı. Bugün aynı bakır çift, sayısal Ethernet sinyali de taşıyabiliyor. Sinyalin türü, kabloyu değil **vericiyi ve kodlamayı** belirler.

→ BU KONUYA NEDEN GEÇİYORUZ?

Burada öğrendiğin bozulmalar ve ortam sınırları, bir sonraki bölümün "iyi bir hat kodu neye benzemeli?" sorusunun cevabını belirliyor: gürültüde bile doğru okunan, saat bilgisini taşıyan, DC bileşeni olmayan bir kod.

✎ KENDİN ÇÖZ — Kendin çöz

K3.1 50 mW gönderilen sinyal alıcıda 0.5 mW olarak ölçülüyor. Kayıp kaç dB?
K3.2 Sinyal gücü 10 mW, gürültü gücü 0.1 mW. SNR değeri ve dB karşılığı nedir?
K3.3 Aşağıdaki durumlarda hangi bozulma var: (a) Sesin uzaktaki hoparlörde kısılması, (b) telefon konuşmasında araya cızırtı karışması, (c) kare dalganın köşelerinin yuvarlanması.

3.4 Bağlantılar

- **Önceki bölüm → bu bölüm:** Çok seviyeli sinyalin "gürültüye az dayanıklı" olduğunu söylemiştik; burada gürültünün ne olduğunu gördük.
- **Bu bölüm → sonraki bölüm:** Gürültü ve zayıflama altında hangi hat kodu hayatta kalır? Cevap hat kodlamada.
- **İleride:** Bilgisayar Ağları dersinde Ethernet kablo kategorileri (Cat5e, Cat6), fiber tipleri ve kablosuz kanal modelleri bu bölümün devamıdır.

⚠ SINAVDA DİKKAT

- **Üç bozulmayı** tanım + sonuç + çare ile bil: zayıflama (yükselteç/tekrarlayıcı), bozulma (şekil değişir), gürültü (istenmeyen enerji eklenir).
- **Ortam sinyal türünü belirlemez** (D1 s. 43).
- **Yüksek SNR = iyi.** Düşük SNR = hata olasılığı artar.
- Slayt 41'deki **sayılar** (500 m, 185 m, 100 m, 2000 m) ve **kablo adı eşleşmesi**.

❖ BU KONUDAN NASIL SORU GELİR?

1. Tanım: *"Define attenuation, distortion and noise."* Üçünü tek cümlede ve farkıyla yaz.
2. Kavramsal: *"Why does a low SNR increase the error probability?"* İpucu: eşik, gürültünün sinyali eşiğin öbür tarafına geçirmesi.
3. Çoktan seçmeli: *"Which statement is true? (a) Fiber can only carry digital signals. (b) The medium alone does not determine whether a signal is analog or digital. (c) Wireless can only carry analog signals."* Cevap: (b).
4. Karşılaştırma: Bakır kablo, fiber ve kablosuz ortamı bir tabloda üç özellikle karşılaştır.

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "Gürültü, sinyalin zayıflamasıdır." → **Doğrusu:** Gürültü **eklenen** enerjidir; zayıflama kaybolan enerjidir. → **Neden:** Biri çıkarma, biri toplama gibi düşün.
- **Yanlış:** "Fiber optik yalnızca sayısal sinyal taşır." → **Doğrusu:** Ortam sinyalin türünü belirlemez. → **Neden:** Analog sinyal de ışığın yoğunluğuyla taşınabilir.
- **Yanlış:** "Kablosuz ortam yalnızca bir cihaza aittir." → **Doğrusu:** Spektrum paylaşılır. → **Neden:** Bu yüzden diğer vericiler kaliteyi düşürür.

⚡ HIZLI TEKRAR

- Üç bozulma: **zayıflama** (enerji azalır), **bozulma** (şekil değişir), **gürültü** (istenmeyen enerji eklenir).
- Zayıflamada yükselteç/tekrarlayıcı; sayısal tekrarlayıcı gürültüyü de temizler.
- **SNR** yüksekse doğru karar kolay; düşükse hata artar, sağlam kodlama gerekir.
- Bakır (ucuz, mesafe sınırlı), fiber (ışık, yüksek kapasite, girişime dayanıklı), kablosuz (hareketlilik, paylaşılan ortam).
- Bükümlü çift: büküm girişimi ve çapraz karışmayı azaltır. Koaksiyel: koruma sağlar.
- Slayt 41 sayıları: 500 m (10Base5), 185 m (10Base2), 100 m (Ethernet), 2000 m (100BaseFX).
- **Ortam sinyal türünü tek başına belirlemez.**

Bu bölümden sonra şunları yapabiliyor olmalısın: Üç bozulmayı ayırt etmek; SNR'ı yorumlamak; üç ortamı karşılaştırmak; slayt 41'in tuzaklarını bilmek.

3.5 Bölüm cevapları

D3.1 8 seviyeli sinyalde. Seviyeler birbirine daha yakın olduğu için aynı gürültü bir seviyeyi komşusuna kolayca atlatır (bkz. 2. bölüm).

K3.1 Oran $50/0.5 = 100$ → $10 \log_{10} 100 = 20$ dB.

K3.2 SNR $= 10/0.1 = 100$ → 20 dB.

K3.3 (a) zayıflama, (b) gürültü, (c) bozulma (yüksek frekanslı bileşenler farklı davranıyor).

Sınav sorusu cevabı: Zayıflama = mesafeyle enerji kaybı; bozulma = frekans bileşenlerinin farklı hız/gecikmesi nedeniyle şekil değişimi; gürültü = araya karışan istenmeyen enerji.

BÖLÜM 4

Bitleri Tele Dökmek: Hat Kodlama

◆ **BU BÖLÜMDE** Okuma süresi: 75 dk Zorluk: ●●●

- Hat kodlamanın (**line coding**) ne olduğunu ve iyi bir hat kodunun hangi 5 şeyi sağlaması gerektiğini
- Alıcının neden saatini göndericiyle uyumlu tutmak zorunda olduğunu; küçük bir saat farkının nasıl büyük hataya dönüştüğünü
- Unipolar NRZ, NRZ-L, NRZ-I, RZ, Manchester ve fark Manchester kodlarını **elle çizmeyi**
- Kodların güçlü ve zayıf yanlarını karşılaştırmayı; blok kodlama ve karıştırmanın amacını

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 44–55

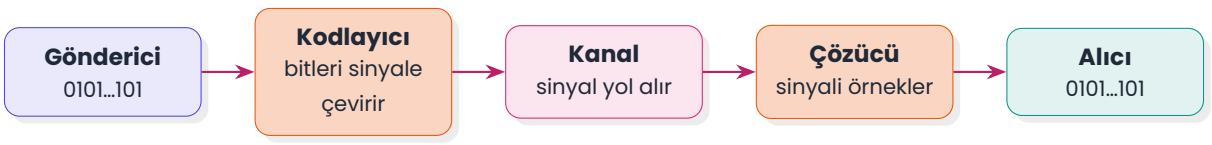
Sınav önceliği yüksek: slaytta üç kod için dalga biçimi çizimi, bir karşılaştırma tablosu ve iki çizim sorusu var (D1 s. 51: `101000110`, s. 52: `01001`, s. 53: `010011`). "Şu bit dizisini şu kodlamayla çiz" hocaların en sevdiği sorulardan biridir; bu yüzden bu bölümde çizim becerisini adım adım kuruyoruz.

→ BU KONUYA NEDEN GEÇİYORUZ?

2. bölümde sayısal sinyalin, "yüksek seviye = 1, alçak seviye = 0" gibi seviyelerle bitleri temsil ettiğini gördük. Ama bu, bitleri tele dökmenin **tek** yolu değil; hatta kötü bir yolu olabilir. 3. bölümde de kanalın sinyali zayıflattığını ve gürültü kattığını öğrendik. Şimdi soru şu: **bitleri öyle bir şekle çevirmeliyiz ki alıcı bozulmuş sinyalden bile, hangi bitin nerede başlayıp bittiğini bulabilsin.** İşte hat kodlama budur.

4.1 Kodlayıcı, kanal, çözücü ❖ Kaynak: D1 s. 45

Slayt 45'teki zincir şöyle:



Şekil 4.1: Sayısal veriden sayısal sinyale: kodlayıcı, kanal ve çözücü (D1 s. 45)

Kodlayıcı (**encoder**) bit dizisini fiziksel bir sinyal örüntüsüne eşler; **çözücü** (**decoder**) gelen sinyali örnekleyip bitleri geri kazanır. Slaytın alt kutusu: *"The sender maps bits to a physical signal pattern; the receiver samples it to recover the bits."* Alıcı, sinyali belli aralıklarla "örnekler" (yani o anki seviyeye bakar). Bu örnekleme anının doğru olması, bütün bölümün ana derdi.

4.2 İyi bir hat kodu neye benzer? ❖ Kaynak: D1 s. 46

Slayt beş istek sayıyor. Ben her birinin yanına **neden** istendiğini yazdım:

İstek (slayt)	Neden istiyoruz?
Alıcı zamanlamayı ve bit sınırlarını geri kazanabilsin	Bit hangi anda başlıyor, hangi anda bitiyor? Bunu bilmeden bitleri sayamayız.
Gereksiz bant genişliği ve enerji kullanılmasın	Kanal paylaşılan ve değerli bir kaynak; enerji de maliyettir.
Uzun 0 ya da 1 dizilerinde senkronizasyon kaybolmasın	Sinyal hiç değişmezse alıcı zamanı şaşırır (aşağıda ayrıntı).
Gerektiğinde DC bileşeni azaltılsın	Bazı hatlar (transformatörlü, kondansatörlü) sabit gerilimi geçiremez.
Hata fark etmeye yardımcı geçiş örüntüleri sağlansın	Beklenen geçiş gelmezse “bir şey yanlış” diyebiliriz.

Bu beş isteğin hepsini birden mükemmel karşılayan kod yok; kodlar arasında **takaslar** var. Bölümün geri kalanı bu takasları görmekle ilgili.

4.3 Saatler neden anlaşmak zorunda? ❖ Kaynak: D1 s. 47–48

Gönderici bitleri belli bir hızla gönderiyor, alıcı da belli bir hızla örnekliyor. İkisinin saati tıpatıp aynı olmak zorunda. Bir benzetme: iki kişi aynı şarkıyı eşzamanlı söylemeye çalışıyor ama biri biraz hızlı. Başta fark yok gibi görünür, ama şarkının sonunda biri bir dize ilerde.

Slayt 47 bu durumu gösteriyor: üstte gönderilen bit dizisi (`1 0 1 1 0 0 0 1 …`), altta alıcının aldığı dizi (`1 1 0 1 1 1 0 0 0 1 1 …`). Alıcı yanlış anda örnekleyince bazı bitleri **atlıyor**, bazılarını **iki kez okuyor** ya da **komşu bitleri karıştırıyor** (slayt alt kutusu).

∑ TANIM / FORMÜL — Saat kayması (clock drift)

Alıcının saatinin göndericininkinden farklı hızda çalışması sonucu örnekleme anının zamanla kaymasıdır. Kayma birikir: her bitte biraz daha uzaklaşır.

4.3.1 Küçük fark, büyük hata (D1 s. 48)

Slaytın örneği: alıcı saati **%0.1** hızlı.

Hız	Gönderilen	Alınan	Fazladan
1 kbps	1000 bit	1001 bit	**1 bit/s**
1 Mbps	1 000 000 bit	1 001 000 bit	**1000 bit/s**

▶ ÖRNEK 4.1 — Slayt örneği [D1 s. 48]: %0.1 hızlı alıcı saatinin etkisi

Verilen: Alıcı saati %0.1 hızlı, yani göndericinin 1 saniyesinde alıcının saati 1.001 saniye geçiriyor. **İstenen:** 1 kbps ve 1 Mbps’te saniyede kaç fazladan bit okunur?

1. Alıcı saniyede 1.001 kat fazla örnekler: fark $= 0.001 = 0.1\%$.
2. 1 kbps: saniyede $1000 \times 0.001 = 1$ fazladan bit.

3. 1 Mbps: saniyede $1\,000\,000 \times 0.001 = 1000$ fazladan bit.

Sonuç: Aynı yüzde hata, hız 1000 kat artınca mutlak hatayı da 1000 kat artırır.

✔ python3 ile doğrulandı

▶ ÖRNEK 4.2 — Ek örnek: kaç bit sonra alıcı yanlış bite bakar?

Alıcı her bitin **tam ortasında** örnekler. Saat %0.1 hızlıysa, her bitte örnekleme noktası 0.001 bit süresi kadar kayar. Örnekleme noktası bitin kenarına (yani yarım bit uzağa) kadar kayınca, komşu bite geçer:

$$n \times 0.001 = 0.5 \;\Rightarrow\; n = 500 \text{ bit.}$$

Yani **sinyal 500 bit boyunca hiç değişmezse** (ve başka bir düzeltme yoksa) alıcı artık yanlış bite bakıyor. ✔ python3 ile doğrulandı

Çıkarım: Alıcının saatini düzeltebilmesi için sinyalde **düzenli geçişler** (0↔1 değişimleri) olmalı. Her geçiş bir "saat ayarlama" fırsatıdır.

→ BU KONUYA NEDEN GEÇİYORUZ?

Bu yüzden listedeki "uzun 0 ya da 1 dizilerinde senkronizasyon kaybolmasın" isteği çok önemli. Aşağıda göreceğin kodların çoğu bu isteği ne kadar iyi karşıladığıyla ayrışıyor.

4.4 Hat kodlama ailesi ❖ Kaynak: D1 s. 49

Slayt 49, kodları aileler hâlinde sınıflandırıyor:



Şekil 4.2: Hat kodlama şemalarının sınıflandırılması (D1 s. 49). Bu haftanın odağı unipolar NRZ, polar NRZ-L / NRZ-I / RZ ve Manchester ailesi.

Aileleri seviyelere göre ayırmak kolay:

Aile	Kullandığı seviyeler	Kısaca
Unipolar	0 ve tek bir kutup (ör. +V)	Sinyal hep aynı yönde (ek bilgi)
Polar	+V ve −V (RZ'de 0 da var)	İki kutup (ek bilgi)
Bipolar (AMI, pseudoternary)	+V, 0, −V	1'ler için kutup sırayla değişir (ek bilgi)

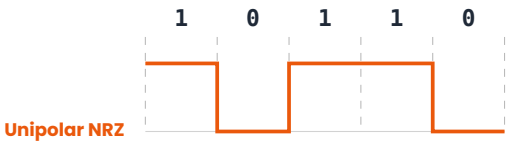
Aile	Kullandığı seviyeler	Kısaca
Çok seviyeli / çok geçişli	3 ya da daha fazla seviye	Daha yüksek hız ya da düşük frekans; 2. bölümün çok seviyeli sinyali (ek bilgi)

Slaytlarda yalnızca aile adları var; ayrıntı verilen kodlar ise unipolar NRZ, NRZ-L, NRZ-I, RZ, Manchester ve fark Manchester. Aşağıda bunları tek tek çizeceğiz.

4.5 Unipolar NRZ: en basit kod

❖ Kaynak: D1 s. 50

NRZ, "Non-Return-to-Zero" demektir: sinyal bit süresi boyunca sıfıra **dönmez**, aynı seviyede kalır. Unipolar'da sinyal tek kutuplu: 1 için +V, 0 için 0 V.



Şekil 4.3: Unipolar NRZ: 1 için +V, 0 için 0 V (D1 s. 50). Sinyal hiç eksiye inmez.

∑ TANIM / FORMÜL — Unipolar NRZ

Kural: Bit 1 ise bit süresi boyunca +V; bit 0 ise 0 V.
Normalize güç (D1 s. 50): sinyalin ortalama gücünü 1 Ω'luk dirençte hesaplarız. Bitlerin yarısı 1, yarısı 0 ise: $\frac{1}{2}V^2 + \frac{1}{2}(0)^2 = \frac{1}{2}V^2$.
Güçlü yan: Çok basit.
Zayıf yanlar (slayt): **güçlü DC bileşen** ve **uzun aynı bit dizilerinde zayıf zamanlama bilgisi**.

4.5.1 DC bileşeni nedir, neden dert?

Sinyalin zaman içindeki **ortalama** değerine DC bileşeni denir. Unipolar NRZ'de sinyal hiç eksiye inmez, bu yüzden ortalama sıfır olmaz: yukarıdaki `10110` örneğinde 3 tane V ve 2 tane 0 var → ortalama $0.6\,V$. Çok sayıda 1 gönderirsen ortalama V'ye yaklaşır.

Bu neden kötü? Birçok iletişim hattı **transformatör ya da kondansatör** üzerinden bağlanır; bunlar sabit gerilimi geçirmez. Sabit bir gerilim hat üzerinde yavaş yavaş sönümlenir ve sinyal bozulur. Bir de enerji sorunu: sabit bileşen bilgi taşımaz ama enerji harcar.

✎ DURUP DÜŞÜN

D4.1 `0000000000` dizisini unipolar NRZ ile çizersen sinyal neye benzer? Alıcı kaç bit olduğunu nasıl bilir?

4.6 Polar NRZ-L ve NRZ-I

❖ Kaynak: D1 s. 51

Polar kodlarda iki kutup var: +V ve −V. Ortalama sıfıra yakın olur (1 ve 0 eşit sayıdaysa), bu yüzden DC bileşeni çok azalır.

∑ TANIM / FORMÜL — NRZ-L (Level) ve NRZ-I (Invert on ones)

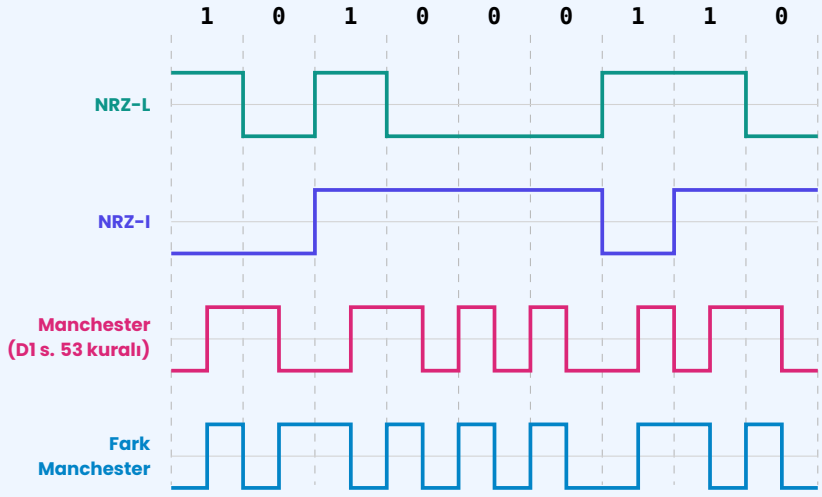
NRZ-L: Seviye bitin kendisini temsil eder. 1 = yüksek (+V), 0 = alçak (−V). (Bazı kitaplarda tersi; slayt 51'deki şekil bu kuralı kullanıyor.)
NRZ-I: Bitin değeri **seviyede** değil, **geçişte** saklıdır. Bit 1 ise bit başında seviye **tersine döner** (geçiş var); bit 0 ise seviye **aynı kalır**. (I: *Invert*, ters çevir.)
Slayt: *"In NRZ-L, the level represents the bit; in NRZ-I, a transition at the start represents a selected bit value."*

▶ ÖRNEK 4.3 — Slayt örneği [D1 s. 51]: 101000110, adım adım

Verilen: Bit dizisi `1 0 1 0 0 0 1 1 0`. Dört kodlamanın hepsi çizilecek; aşağıdaki şekil dört satırı bir arada gösteriyor.
NRZ-L: Her bit için seviyeyi doğrudan yaz: 1 → yüksek, 0 → alçak. `H L H L L L H H L`
NRZ-I: Slayt şeklindeki gibi bit öncesinde seviyenin **yüksek** olduğunu kabul et.

1. Bit 1 (1): geçiş → alçak.
2. Bit 2 (0): geçiş yok → alçak.
3. Bit 3 (1): geçiş → yüksek.
4. Bit 4, 5, 6 (0, 0, 0): geçiş yok → yüksek, yüksek, yüksek.
5. Bit 7 (1): geçiş → alçak.
6. Bit 8 (1): geçiş → yüksek.
7. Bit 9 (0): geçiş yok → yüksek.

Sonuç: `L L H H H H L H H`. Bu, slayttaki NRZ-I çizimiyle birebir aynı. ✔ python3 ile doğrulandı



Şekil 4.4: 101000110 dizisinin dört kodlaması (D1 s. 51 sorusu). NRZ-I için bit öncesi seviye yüksek; fark Manchester için ilk bit alçak seviyede başlar; Manchester s. 53 kuralıyla çizildi (0: yüksek-alçak, 1: alçak-yüksek).
Manchester ve fark Manchester: Bu iki satır bir sonraki konuda ayrıntılı açıklanıyor.

4.6.1 NRZ-I'nın sırrı: polarite değişimine dayanıklılık

NRZ-L'de yüksek seviye 1'dir; telin iki ucunu yanlışlıkla ters bağlarsan bütün bitler tersine döner (1 ↔ 0). NRZ-I'da ise bitin değeri **geçişin varlığında** saklı olduğu için, polarite ters olsa da geçişler yine geçiş olarak kalır. Bu yüzden slayt 54'te NRZ-I'nın güçlü yanı *"Resists polarity inversion"*, polarite ters çevrilmesine dayanıklılık olarak geçiyor.

4.6.2 Yine uzun diziler

Her iki NRZ kodunda da bir zayıflık var: NRZ-L'de uzun 0 ya da 1 dizileri sinyali hiç değiştirmez. NRZ-I'da **uzun 0 dizisi** geçişsiz kalır (1'ler geçiş yaratır, ama uzun 1 dizisi sorun değildir; sürekli geçiş olur). Slayt 54'te bunlar sırayla *"Long runs are problematic"* ve *"Long transition-free runs"* diye geçiyor.

4.7 Polar RZ: her bitte sıfıra dön ❖ Kaynak: D1 s. 52

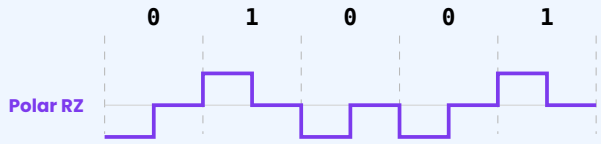
NRZ'nin geçişsiz uzun dizi sorununa bir çözüm: sinyali bit süresinin **ortasında sıfıra döndür**. Böylece her bitte en az bir geçiş garanti olur.

∑ TANIM / FORMÜL — Polar RZ (Return-to-Zero)

Kural (D1 s. 52): Bit 1 ise bit süresinin ilk yarısında **+V**, ikinci yarısında **0**. Bit 0 ise ilk yarısında **−V**, ikinci yarısında **0**.
Üç seviye kullanılır: +V, 0, −V.
Güçlü yan: Zamanlama iyileşir (her bitte geçiş var). **Zayıf yan:** Daha fazla bant genişliği gerekir (sinyal bit başına iki kez değişiyor).
Slayt kutusu: $r = \frac{1}{2}$ ve $S_{ave} = N$. Slayt alt kutusu: *"The signal returns to zero within every bit interval; timing improves, but more bandwidth is required."*

▶ ÖRNEK 4.4 — Slayt örneği [D1 s. 52]: 01001 dizisini RZ ile çiz

1. Bit 0: ilk yarı −V, ikinci yarı 0.
2. Bit 1: ilk yarı +V, ikinci yarı 0.
3. Bit 0: −V, 0.
4. Bit 0: −V, 0.
5. Bit 1: +V, 0.



Şekil 4.5: 01001 dizisinin polar RZ kodlaması: 1 için +V, sonra sıfır; 0 için −V, sonra sıfır (D1 s. 52). Üç seviye vardır.
Bu çizim slayttaki şekille aynı: negatif darbeler 0'ları, pozitif darbeler 1'leri gösteriyor.
✔ python3 ile doğrulandı

4.7.1 Kutudaki r ve S_{ave} ne anlama geliyor?

- r = **bir sinyal elemanının taşıdığı veri elemanı oranı.** RZ'de her bit iki sinyal elemanı (yarım bitlik iki parça: darbe ve sıfır) kullanıyor, dolayısıyla $r = 1/2$. Yani 1 bit için iki parça harcıyoruz; kod "verimsiz" ama zamanlama güçlü.
- $S_{ave} = N$: veri hızı N bit/s ise ortalama sinyal hızı N baud (sembol/s). Kitaplardaki genel formül $S_{ave} = c \times N \times \frac{1}{r}$ biçimindedir (c: duruma bağlı katsayı, ortalama durumda $\frac{1}{2}$; slaytta yok). RZ için $\frac{1}{2} \times N \times 2 = N$ çıkar. Karşılaştırma için: NRZ'de $r = 1$ olduğundan $S_{ave} = N/2$; RZ ve Manchester'da sinyal hızı bunun iki katıdır. Sınav için kutudaki iki değeri ve "bant genişliği geniş" cümlesini bil.
- **Bant genişliği grafiği:** Sağdaki çan eğrisi, gücün frekansa dağılımını gösteriyor: güç

çoğunlukla f/N eksenin 0 ile 2 arasında toplanıyor. Yani bant genişliği yaklaşık $2N$'e kadar uzanıyor; bu, NRZ'den geniş.

⚠ SINAVDA DİKKAT

Sınavda r ve S_{ave} değerleri sorulursa: RZ ve Manchester için $r = 1/2$, $S_{ave} = N$ (D1 s. 52–53). Hesaplı soru gelirse hocanın ders notundaki tanımı kullan.

4.8 Manchester ve fark Manchester ❖ Kaynak: D1 s. 53

RZ'nin garantili geçiş fikrini devam ettirelim ama sinyali sıfıra çekmek yerine **her bitin ortasında yön değiştirelim**. Böylece her bitte hem yüksek hem alçak seviye var: ortalama her zaman sıfır (DC yok!) ve her bitin ortasında geçiş var (saat bilgisi!).

∑ TANIM / FORMÜL — Manchester (D1 s. 53)

Kural: Her bitin **ortasında** geçiş vardır; geçişin yönü bitin değerini verir.

- **0** = yüksekten alçağa (bit ortasında aşağı doğru geçiş)
- **1** = alçaktan yükseğe (bit ortasında yukarı doğru geçiş)

Bu kural slayt 53'teki sarı kutuda çizili ("0 is" ve "1 is" ifadeleri).
Kendinden saatli (**self-clocking**): Alıcı, saat bilgisini sinyalin kendisinden çıkarabilir; çünkü her bitte garanti geçiş var.

∑ TANIM / FORMÜL — Fark Manchester (differential Manchester)

Kural (D1 s. 53): Her bitin **ortasında** geçiş **her zaman** vardır (saat için). Bitin değeri ise **bit sınırındaki** duruma bakar:

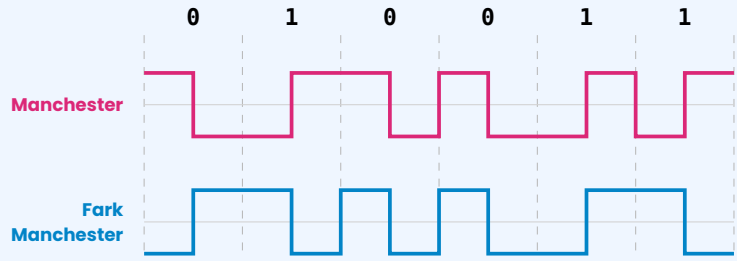
- Bit sınırında geçiş **yoksa** ("no inversion") → sıradaki bit **1**.
- Bit sınırında geçiş **varsa** ("inversion") → sıradaki bit **0**.

Yani veri, **geçişin varlığında** saklıdır (tıpkı NRZ-I gibi), saat bilgisi ise orta geçişte.

▶ ÖRNEK 4.5 — Slayt örneği [D1 s. 53]: 010011, adım adım

Manchester: her biti kendi geçişine çevir. `0 1 0 0 1 1` → yarım-bit seviyeleri: `HL, LH, HL, HL, LH, LH` (H: yüksek, L: alçak).
Fark Manchester: İlk bitin **alçak** seviyede başladığını kabul edelim (slayttaki şekilde de böyle).

1. Bit 0 (ilk bit): başlangıç alçak; orta geçiş → yüksek. Bit sonu: **yüksek**.
2. Bit 1: sınırda geçiş yok → yüksek ile başlar; orta geçiş → alçak. Bit sonu: **alçak**.
3. Bit 0: sınırda geçiş var → yükselir (alçak → yüksek); orta geçiş → alçak. Bit sonu: **alçak**.
4. Bit 0: sınırda geçiş var → yüksek; orta → alçak. Bit sonu: **alçak**.
5. Bit 1: sınırda geçiş yok → alçak; orta → yüksek. Bit sonu: **yüksek**.
6. Bit 1: sınırda geçiş yok → yüksek; orta → alçak. Bit sonu: **alçak**.



Şekil 4.6: 010011 dizisi: Manchester ve fark Manchester (D1 s. 53). Fark Manchester'da bit sınırında geçiş = 0, geçiş yok = 1; orta noktada geçiş her zaman vardır.
Sonuç slayttaki şekille aynı: siyah noktalar (geçiş = 0) ve beyaz daireler (geçiş yok = 1) bit sınırlarında yukarıdaki sırayla yer alıyor. ✔ python3 ile doğrulandı

4

✘ SIK YAPILAN HATA — Slayt 51 ile slayt 53 Manchester kuralını ters gösteriyor

Slayt 51'in şeklinde Manchester satırı **1 = yüksek-alçak, 0 = alçak-yüksek** çiziyor. Slayt 53'ün kutusu ise **0 = yüksek-alçak, 1 = alçak-yüksek** diyor. İki farklı konvansiyon da kitaplarda var (53'teki kural IEEE 802.3 Ethernet kuralıdır; ters olan G. E. Thomas konvansiyonudur).
Sınavda ne yapmalı? Soru hangi kuralı veriyorsa onu kullan. Kural verilmemişse çiziminin yanına kuralı yaz (ör. "0: yüksek-alçak, 1: alçak-yüksek") ve tutarlı kal. Bu konu hocaya sorulacaklar listesinde.

4.8.1 Manchester'ın bedeli

Her bitte en az bir geçiş (bazen iki) var: yüksek geçiş hızı ve dolayısıyla **geniş bant genişliği**. Slaytta çan eğrisi $f/N = 0 \ldots 2$ arasında yayılıyor; RZ ile benzer şekilde bant genişliği NRZ'ye göre yaklaşık iki kat. Slayt alt kutusu bunu tarihle bağlıyor: *"Classic Manchester was used in early 10 Mbps Ethernet; modern high-speed Ethernet uses other encodings."*

★ MERAKLISINA (SINAV DIŞI) — Manchester neden eski 10 Mbps Ethernet'in kodu?

10 Mbps'lik klasik Ethernet'te hat hızı çok yüksek değildi; bant genişliğinin ikiye katlanması sorun olmadı, buna karşılık güvenilir senkronizasyon ve DC'siz sinyal çok değerliydi. Hızlar yükselince bant genişliği pahalılaştı; Fast Ethernet (100 Mbps) ve sonrası daha verimli kodlara (4B/5B + MLT-3, 4D-PAM5 gibi) geçti. Slayt 49'daki ailelerde bu kodları görüyorsun.

4.9 Hepsini yan yana: karşılaştırma ❖ Kaynak: D1 s. 54

Kod	Bit nasıl temsil edilir?	Güçlü yan	Sınırlama
Unipolar NRZ	Sıfır ya da pozitif seviye	Çok basit	DC ve zayıf zamanlama
NRZ-L	Pozitif/negatif seviye	Basit polar yapı	Uzun diziler sorunlu
NRZ-I	Geçiş / geçiş yok	Polarite ters çevrilmesine dayanıklı	Uzun geçişsiz diziler
RZ	Bit içinde sıfıra dönüş	Daha iyi zamanlama	Daha fazla bant genişliği
Manchester	Bit ortasında geçiş	Kendinden saatli	Yüksek geçiş hızı

4.9.1 Hangi kod hangi isteği karşılıyor?

Bölümün başındaki beş isteği bir ısı tablosuna koyalım (✔ iyi, △ orta, ✘ zayıf). Bu tablo slaytta yok; slayt 54'ün cümlelerini ve yukarıdaki türetmeleri özetleyen **benim yorumum**:

İstek	Unipolar NRZ	NRZ-L	NRZ-I	RZ	Manchester
Zamanlama (uzun dizilerde)	✘	✘	△ (1'lerde iyi, 0'larda zayıf)	✔	✔
Düşük bant genişliği	✔	✔	✔	✘	✘
DC bileşeni az	✘	△	△	△	✔
Basitlik	✔	✔	✔	△	△

✎ DURUP DÜŞÜN

D4.2 Yalnızca bant genişliğini önemsiyorsan hangi kodu seçersin? Peki bir kablo uzun dizilerde senkronizasyonu kaybetmesin istiyorsan?

4.10 Hat kodlamanın ötesi: blok kodlama ve karıştırma

❖ Kaynak: D1 s. 55

Slayttaki son konu, yukarıdaki basit kodların zayıflıklarını **başka bir katmanda** onarmak:

	Blok kodlama (block coding)	Karıştırma (scrambling)
Ne yapar?	Veri bloklarına **fazladan (artık) bit** ekler; geçişleri garanti eder ve sınırlı hata tespiti sağlar	Uzun aynı-bit dizilerini **kontrollü ihlaller ya da dönüşümlerle** kırar
Amaç	Alıcı zamanlamasını korumak ve kanalın bant genişliği ve enerji özelliklerine uymak	(aynı amaç)

+ EK BİLGİ (KAYNAK DIŞI) — Blok kodlama örneği: 4B/5B

4 veri bitini 5 bitlik bir koda çevirir; 5 bitlik kodlar, ardışık çok sayıda 0 içermeyecek şekilde seçilir. Bedeli **%25 fazladan bit**: 100 Mbps veri göndermek için hat üzerinde $100 \times 5/4 = 125$ Mbaud gerekir. ✔ python3 ile doğrulandı

Bu slaytta adı geçmiyor ama 2B/1Q, 8B/6T ve 4D-PAM5 (slayt 49) da bu ailenin üyeleridir.

→ BU KONUYA NEDEN GEÇİYORUZ?

Bu bölümde **bitlerin** (zaten sayısal olan veri) sinyale dönüşümünü gördük: slayt 32'deki tablonun "sayısal veri → sayısal sinyal" satırı. Bir sonraki bölümde, bu zincirin **başına** bir adım ekliyoruz: veri henüz bit değilse (ses gibi analog bir şeyse), önce nasıl bite çevireceğiz?

✎ KENDİN ÇÖZ — Kendin çöz

K4.1 `110010` dizisini (a) NRZ-L (b) NRZ-I (bit öncesi seviye alçak) (c) polar RZ ile çiz. Her biri için seviyeleri `H/L` olarak yaz.
K4.2 `0110` dizisini (D1 s. 53 kuralıyla) Manchester ve fark Manchester (ilk bit alçakta başlıyor) ile kodla.
K4.3 100 kbps'lik bir hatta alıcı saati %0.05 hızlı. Saniyede kaç fazladan bit okunur? Bu hatta, örnekleme noktası yarım bit kayana kadar kaç ardışık geçişsiz bit gönderilebilir?

4.11 Bağlantılar

- **Önceki bölümler → bu bölüm:** 2. bölümde sinyalin seviyelerle bit taşıdığını, 3. bölümde gürültünün ve zayıflamanın bunu bozduğunu gördük. Hat kodlama, bu bozulmalara karşı **tasarım kararı**dır.
- **Bu bölüm → sonraki bölüm:** PCM'de analog sesi bitlere çeviriyoruz; ortaya çıkan bit akışı da sonunda bu bölümdeki kodlardan biriyle tele dökülür.
- **İleride:** Bilgisayar Ağları'nda Ethernet'in fiziksel katmanı (10BASE-T Manchester, 100BASE-TX 4B/5B + MLT-3, 1000BASE-T 4D-PAM5) bu bölümün devamıdır.

⚠ SINAVDA DİKKAT

- **Çizim sorusunda:** Önce bit sınırlarını (dikey kesikli çizgiler) çiz, sonra her biti kuralıyla işle. Başlangıç seviyesi ve kuralı (özellikle Manchester) yaz.
- **NRZ-I:** 1 = geçiş, 0 = geçiş yok. **Fark Manchester:** bit sınırında geçiş = **0**, geçiş yok = **1**.
- **Manchester:** her bitte orta geçiş vardır; fark Manchester'da da vardır. Sadece fark Manchester'da veri **sınırdaki** geçişte saklı.
- $r = 1/2$**,** $S_{ave} = N$**:** RZ ve Manchester için (D1 s. 52–53).
- Karşılaştırma tablosundaki her hücre sınavlık (slayt 54).

❖ BU KONUDAN NASIL SORU GELİR?

1. Çizim: *"Draw the Manchester encoding of 10110."* İpucu: kuralı yaz, orta geçişleri çiz.
2. Karşılaştırma: *"Compare NRZ-L and Manchester in terms of synchronization and bandwidth."* İpucu: tablodaki sütunları kullan.
3. Kavramsal: *"Why is a DC component undesirable?"* İpucu: transformatör/kondansatör, enerji harcayan ama bilgi taşımayan sabit bileşen.
4. Hesap: *"A receiver clock is 0.1% fast. How many extra bits per second are received at 1 Mbps?"* (1000 bit/s)
5. Çoktan seçmeli: *"Which code has a mid-bit transition in every bit? (a) NRZ-L (b) NRZ-I (c) Manchester (d) Unipolar NRZ."* Cevap: (c).

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "NRZ-I'da 0 geçiş yaratır." → **Doğrusu:** 1 geçiş yaratır, 0 seviyeyi korur. → **Neden:** NRZ-I = *Invert on ones* (slayt: geçiş "seçilmiş bir bit değerini" temsil eder; slayt 51'in çiziminde bu değer 1'dir).
- **Yanlış:** "Fark Manchester'da bit sınırında geçiş = 1." → **Doğrusu:** geçiş = 0, geçiş yok = 1

(D1 s. 53). → **Neden:** NRZ-I'nın tersi gibi duruyor; ezberleme, kutudaki *No inversion: next bit is 1* cümlesini hatırla.

- **Yanlış:** "RZ iki seviyelidir." → **Doğrusu:** Üç seviye (+V, 0, −V). → **Neden:** Sıfıra dönüş üçüncü seviyedir.
- **Yanlış:** "Manchester bant genişliğini azaltır." → **Doğrusu:** Artırır (yüksek geçiş hızı); karşılığında DC yok ve kendinden saatli.
- **Yanlış:** "Polar kodlarda DC bileşeni her zaman sıfırdır." → **Doğrusu:** NRZ-L'de 1 ve 0 sayısı eşit değilse DC bileşeni oluşur; yalnızca Manchester sayıdan bağımsız sıfırdır.

⚡ HIZLI TEKRAR

- Hat kodlama: bitleri sinyale çeviren kurallar. İyi kod: zamanlama + düşük bant genişliği + uzun dizilerde senkronizasyon + düşük DC + hata ipuçları.
- Saat kayması birikir: %0.1 → 1 kbps'te 1 bit/s, 1 Mbps'te 1000 bit/s fazladan bit.
- **Unipolar NRZ:** 1 = +V, 0 = 0; DC güçlü, zamanlama zayıf; normalize güç $\frac{1}{2}V^2$.
- **NRZ-L:** seviye = bit. **NRZ-I:** 1 = geçiş (polarite ters olsa da çalışır).
- **RZ:** 1 = +V sonra 0, 0 = −V sonra 0; zamanlama iyi, bant genişliği fazla.
- **Manchester:** orta geçiş her zaman var; 0 = yüksek-alçak, 1 = alçak-yüksek (D1 s. 53). **Fark Manchester:** sınırda geçiş = 0, yok = 1.
- Blok kodlama (fazladan bit) ve karıştırma (uzun dizileri kırar) zamanlamayı korur.
- Ezberle: karşılaştırma tablosu (D1 s. 54). Türetebil: saat kayması hesapları.

Bu bölümden sonra şunları yapabiliyor olmalısın: Verilen bit dizisini altı kodla çizmek; kodların artı-eksilerini tablodan söylemek; saat kayması hesabını yapmak.

4.12 Bölüm cevapları

D4.1 Sinyal 0 V'ta düz bir çizgi olur. Alıcı bit sayısını ancak önceden bilinen bir uzunluk ya da ayrı bir saat sinyaliyle bilir; sinyalde geçiş olmadığı için kendi başına sayamaz. İşte unipolar NRZ'nin uzun dizi zayıflığı.

D4.2 Bant genişliği için NRZ ailesi (unipolar NRZ, NRZ-L, NRZ-I); senkronizasyon için Manchester (ya da RZ).

K4.1 `1 1 0 0 1 0`: (a) NRZ-L: `H H L L H L`. (b) NRZ-I (bit öncesi alçak, 1 = geçiş): bit1 (1) → H; bit2 (1) → L; bit3 (0) → L; bit4 (0) → L; bit5 (1) → H; bit6 (0) → H → `H L L L H H`. (c) RZ: `+V0 +V0 −V0 −V0 +V0 −V0` yani her bit için ilk yarı (+V ya da −V) ve ikinci yarı 0. ✔ python3 ile doğrulandı

K4.2 `0 1 1 0`: Manchester (0: yüksek-alçak, 1: alçak-yüksek): yarım-bit seviyeleri `HL, LH, LH, HL`. Fark Manchester (ilk bit alçakta başlıyor): bit 0: LH (bit sonu H); bit 1: sınırda geçiş yok, H ile başlar: HL (bit sonu L); bit 1: geçiş yok, L ile başlar: LH (bit sonu H); bit 0: sınırda geçiş var, H'tan L'ye atlar ve L ile başlar: LH. Yani `LH, HL, LH, LH`. ✔ python3 ile doğrulandı

K4.3 $100\,000 \times 0.0005 = 50$ fazladan bit/s. Yarım bit kayması: $n \times 0.0005 = 0.5 \Rightarrow n = 1000$ ardışık geçişsiz bit. ✔ python3 ile doğrulandı

Sınav sorusu cevapları: (4) $1\,000\,000 \times 0.001 = 1000$ bit/s. (5) Manchester.

BÖLÜM 5

Sesten Bitlere: PCM ve Delta Modülasyonu

◆ **BU BÖLÜMDE** Okuma süresi: 75 dk Zorluk: ●●●

- Analog bir sinyalin neden sayısallaştırıldığını ve bunun bedelini
- PCM'in üç adımını: örnekleme, nicemleme, kodlama
- Nyquist kuralını ($f_s > 2f_{max}$) ve örtüşmeyi (**aliasing**)
- Slayttaki nicemleme tablosunu satır satır okumayı ve PCM bit hızını hesaplamayı
- Delta modülasyonunu, hatalarını (eğim aşırı yüklenmesi, granüler gürültü) ve PCM'den farkını

SINAV ÖNCELİĞİ: YÜKSEK ❖ Kaynak: D1 s. 56–65

Sınav önceliği yüksek: bu bölümde formül (Nyquist, PCM bit hızı), tablo okuma (nicemleme) ve karşılaştırma (PCM ile DM) bir arada. "64 kbps nereden geliyor?" sorusu neredeyse garanti.

→ BU KONUYA NEDEN GEÇİYORUZ?

4. bölümde zaten bit olan veriyi tele döktük. Peki veri bit değilse? Sesin, sıcaklık sensörünün, bir fotoğrafın değerleri **sürekli**. Sayısal bir sistem (bilgisayar, sayısal hat) ancak bitlerle çalışır. Bu bölüm 2. bölümdeki tablonun üçüncü satırını, yani **codec**'i (analog veri → sayısal sinyal) açıyor: sürekli değişen bir değeri sonlu sayıda bitle nasıl yazarız?

↺ HATIRLATMA

Bit sayısı ve seviye sayısı: n bit 2^n farklı örüntü verir (1. bölüm). 3 bit → 8, 8 bit → 256, 16 bit → 65 536 örüntü.
Aralık ve adım: Bir değer aralığını L eşit parçaya bölersen her parçanın genişliği aralık$/L$ olur. 0–10 V aralığını 5 parçaya bölersen parça 2 V.
Kayan nokta yok: Bu bölümde ondalık ayırıcı nokta (ör. 1.50).

5.1 Neden sayısallaştırıyoruz? ❖ Kaynak: D1 s. 57

Ses dalgası zaten bir sinyal; neden uğraşıp bitlere çeviriyoruz? Slayt dört neden sayıyor:

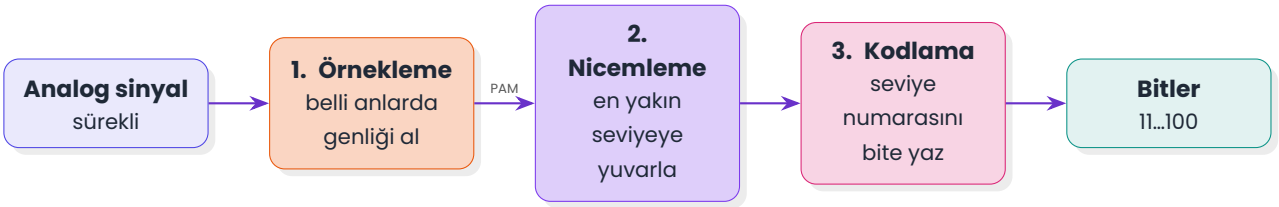
1. **Bilgisayarla saklamak, işlemek ve kopyalamak kolaylaşır.** Bir MP3'ü kopyalarken ses bozulmaz; bir kaseti kopyalayınca bozulur.
2. **Hata denetimi, sıkıştırma ve şifreleme uygulanabilir.** Bunlar bit işleridir.
3. **Tekrarlayıcılar zayıflamış darbeleri yeniden üretebilir** (3. bölümdeki sayısal tekrarlayıcı).

4. **Dönüşüm kayıpsız değildir:** örnekleme ve nicemleme ayrıntıyı sınırlar.

Alt kutu önemli: *"Digital is not always 'superior'; there is a trade-off between processing convenience and conversion cost."* Sayısal her zaman üstün değildir; işlem kolaylığı ile dönüştürme maliyeti arasında bir takas vardır. Sınavda "sayısallaştırmanın dezavantajı nedir?" diye sorulursa cevap: dönüşümün kayıpsız olmaması (nicemleme hatası) ve dönüşüm maliyeti.

5.2 PCM: üç adımlı zincir ❖ Kaynak: D1 s. 58

PCM (**Pulse Code Modulation**, darbe kod modülasyonu) analog sinyali üç adımda bite çevirir:



Şekil 5.1: PCM kodlayıcı zinciri (D1 s. 58). Örnekleme sonrası sinyale PAM (darbe genlik modülasyonu) denir; nicemlenmiş sinyal ise yukarıdaki üç adımdan sonra kodlanır.

Slaytın alt kutusu: *"Basic chain: sample → quantize → encode. The receiver reconstructs an approximate analog waveform."* Alıcı, bitlerden **yaklaşık** bir analog dalga geri üretir; "yaklaşık" kelimesi nicemlemenin kayıplı olmasından geliyor.

Bir benzetme: bir tepenin yüksekliğini kâğıda aktarmak istiyorsun. (1) Her 10 metrede bir yüksekliği ölçersin (**örnekleme**), (2) ölçümleri en yakın 5 metreye yuvarlarsın (**nicemleme**), (3) her yuvarlak değere bir numara verip numaraları yazarsın (**kodlama**). Bu üç adım, tepeyi kâğıda düşük çözünürlükte ama bitlerle taşınabilir şekilde aktarır.

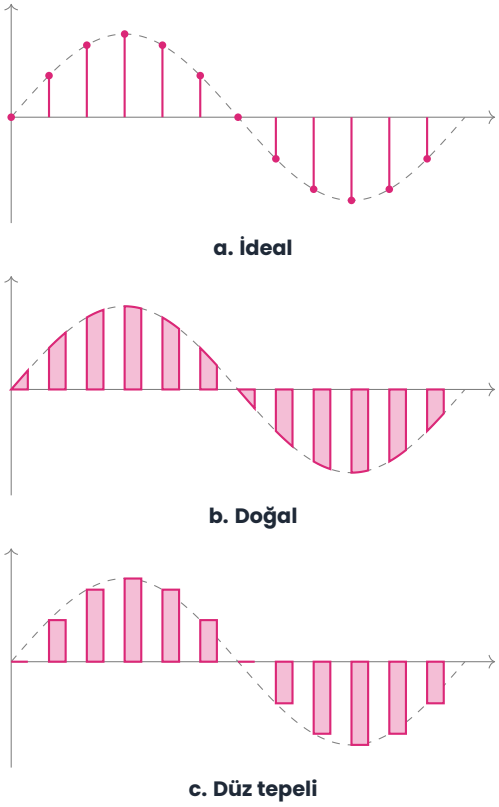
5.3 Adım 1: Örnekleme ❖ Kaynak: D1 s. 59–60

5.3.1 Üç görünüm (D1 s. 59)

Slayt örneklemenin üç devre modelini gösteriyor:

Model	Nasıl çalışır?
İdeal (**ideal**) örnekleme	Sonsuz dar bir anda genliği alır: dikey çizgiler (şekil a)
Doğal (**natural**) örnekleme	Kısa bir süre boyunca sinyalin eğrisini izleyen darbeler (şekil b)
Düz tepeli (**flat-top**) örnekleme	Kısa bir süre boyunca ilk anın değerini sabit tutar (şekil c)

Slayt cümlesi: üçü de *"capture amplitude at selected times"*, yani seçili anlarda genliği yakalar. Aradaki fark devre modelinden gelir; sınav için ana fikir aynı.



Şekil 5.2: Üç örnekleme modeli. Kesikli eğri analog sinyal, pembe şekiller örneklerdir (D1 s. 59'daki şeklin sade bir çizimi).

5.3.2 Ne kadar sık örneklemeliyiz? Nyquist (D1 s. 60)

Örnekler arasındaki süre T_s; saniyedeki örnek sayısı $f_s = 1/T_s$ (**örnekleme hızı**). Soru: f_s ne kadar olmalı?

Sezgi: bir dalgayı iki noktayla anlatmak istiyorsan, her **tepe ve çukuru** yakalamalısın. Bir dalga periyodunda en az 2 örnek lazım; bunu bir periyodun içinde "bir yukarı, bir aşağı" noktası olarak düşün. Dalga ne kadar hızlıysa (frekans ne kadar yüksekse) örnekleme de o kadar sık olmalı. Oyunlardaki kare hızı (FPS) tam bir örnekleme hızıdır: hızlı hareket eden bir nesne düşük FPS'te sıçrar gibi görünür, çünkü kare sayısı hareketi yakalamaya yetmez.

∑ TANIM / FORMÜL — Nyquist örnekleme kuralı (D1 s. 60)

Bant sınırlı (**band-limited**) bir sinyal için ideal koşul:

$$f_s > 2 \times f_{max}$$

f_s: saniyedeki örnek sayısı. f_{max}: sinyaldeki **en yüksek frekans**.
Slayt notu: Pratik sistemler, filtreleme ve güvenlik payı için **daha yüksek** bir hız seçer.

▶ ÖRNEK 5.1 — Ek örnek: telefon sesi için örnekleme hızı

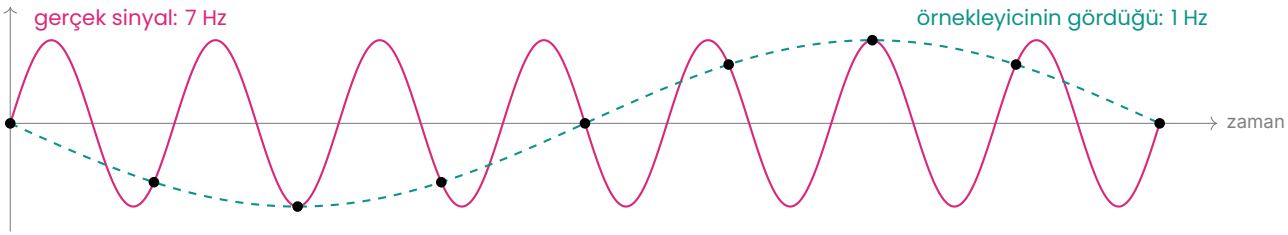
Telefon sesi yaklaşık 4 kHz'e kadar frekanslar taşır. $f_{max} = 4000$ Hz ise $f_s > 2 \times 4000 = 8000$ örnek/s. İşte slayt 62'deki **8 000 örnek/s**'in kaynağı. (Slayt "neden 8 000?" demiyor; bu bağlantı ek bilgidir.) ✔ python3 ile doğrulandı

Müzik CD'si: İnsan kulağı yaklaşık 20 kHz'e kadar duyar → $f_s > 40$ kHz. CD'ler 44.1 kHz kullanır; 4.1 kHz'lik fazlalık, filtreleme ve güvenlik payıdır.

5.3.3 Yetersiz örnekleme: örtüşme

Alt kutu: *"Undersampling makes high frequencies appear as lower frequencies: aliasing."* Yani **yetersiz örnekleme yüksek frekansları düşük frekans gibi gösterir**: örtüşme.

Sinema filmlerinde ileri giden arabanın tekerleğinin geriye döndüğünü gördün mü? Kamera saniyede sınırlı sayıda kare (örnek) yakalıyor; tekerlek bir turu kareler arasında neredeyse tamamlayınca, her karede sanki biraz geri gitmiş gibi görünür. İşte örtüşme budur.



Şekil 5.3: Saniyede 8 örnek ($f_s = 8$ Hz) ile 7 Hz'lik sinüsü örneklersek, siyah noktalar 1 Hz'lik (ters fazlı) bir sinüsün üzerine düşer (örnek çizimdir).

▶ ÖRNEK 5.2 — Ek örnek: 7000 Hz'lik ton 8000 örnek/s ile örneklenirse

$f_s = 8000$ Hz için Nyquist sınırı 4000 Hz. 7000 Hz bu sınırın üstünde → örtüşme. Görünen (alias) frekans: $|7000 - 8000| = 1000$ Hz. Alıcı 7000 Hz'lik tonu **1000 Hz** diye geri üretir.

✔ python3 ile doğrulandı

Çare: Örneklemeden önce 4000 Hz üstünü kesen bir **alçak geçiren süzgeç** koymak (slayttaki "filtreleme"). Bu yüzden telefon hattı 4 kHz'te sınırlanır.

✎ DURUP DÜŞÜN

D5.1 Bir sinyalin en yüksek frekansı 10 kHz. Nyquist'e göre minimum örnekleme hızı nedir? Pratikte neden daha yükseğini seçersin?

5.4 Adım 2 ve 3: Nicemleme ve kodlama

❖ Kaynak: D1 s. 61

Örnekleme bize bir **sayı listesi** verdi: 7.5, 16.2, −5.5, … Ama bu sayılar sonsuz hassas; her biri için sonsuz bit gerekir. Çözüm: izin verilen **sonlu sayıda seviye** belirleyip her örneği **en yakın seviyeye yuvarlamak** (**nicemleme**), sonra seviyenin numarasını sabit uzunluklu bitlerle yazmak (**kodlama**). Slayt cümlesi: *"Each sample is rounded to the nearest allowed level; the level number is encoded with fixed-length bits."*

5.4.1 Tablonun anatomisi

Slayt 61'deki şekil 8 seviyeli (3 bitlik) bir nicemleyiciyi gösteriyor:

- Düşey eksen **normalize genlik**: $-4D \ldots +4D$ arasında 8 şerit; her şerit D genişliğinde. Şeritler sarı ve mavi dönüşümlü boyalı.

- Solda **nicemleme kodları** 0'dan 7'ye: en alt şerit 0, en üst şerit 7.
- Her şeridin **orta noktası** o şeridin nicemlenmiş değeri: $D = 1$ alırsak şerit merkezleri $-3.5, -2.5, -1.5, -0.5, +0.5, +1.5, +2.5, +3.5$. Kod k olan şeridin merkezi $k - 3.5$'tir.
- Alt kısımda 9 örnek için beş satır var: **normalize PAM değeri**, **normalize nicemlenmiş değer**, **normalize hata**, **nicemleme kodu**, **kodlanmış kelime**.

Slayttaki örneklerin ham değerleri (-6.1, 7.5, 16.2, ...) ile normalize PAM değerleri arasındaki ilişki: **5'e bölmek**. Örneğin $7.5/5 = 1.50$, $16.2/5 = 3.24$.

▶ ÖRNEK 5.3 — Slayt örneği [D1 s. 61]: ilk örnek, adım adım

Verilen: Ham örnek -6.1 (normalize PAM değeri -1.22). 8 seviye, şerit genişliği $D = 1$.

1. **Şeridi bul.** -1.22 değeri -2 ile -1 arasında bir şeritte (−2 ... −1). Bu şeridin merkezi -1.5.
2. **Nicemlenmiş değer:** -1.50 (en yakın izin verilen seviye; -0.5 merkezi daha uzak).
3. **Hata** (nicemlenmiş − gerçek): $-1.50 - (-1.22) = -0.28$.
4. **Kod numarası:** merkez $k - 3.5 = -1.5 \Rightarrow k = 2$.
5. **Kodlanmış kelime:** 2 → 3 bitte `010`.

✔ python3 ile doğrulandı

✘ SIK YAPILAN HATA — Slayt 61'deki ilk hata değeri tutmuyor

Slayt, ilk örnek için hata satırına **−0.38** yazıyor. Doğru hesap: $-1.50 - (-1.22) = -0.28$. Tablodaki diğer sekiz sütun hata formülüne uyuyor (kontrol aşağıda). Bu yüzden büyük olasılıkla bir yazım hatası. Sınavda hata hesaplanırken **−0.28** kullan; hocaya da soracağız (bkz. Hocaya Sorulacaklar).

5.4.2 Bütün tablo (kendi hesabımız)

Slayttaki dokuz örneği yeniden hesapladım (ham değer ÷ 5 → en yakın seviye → hata → kod). Son sütundaki "slayt" işareti, slayttaki satırla aynı olduğunu gösteriyor:

#	Ham	PAM (÷5)	Nicemlenmiş	Hata	Kod	Kelime	Slayt ile
1	−6.1	−1.22	−1.50	**−0.28**	2	`010`	slayt: −0.38
2	7.5	1.50	1.50	0	5	`101`	aynı
3	16.2	3.24	3.50	+0.26	7	`111`	aynı
4	19.7	3.94	3.50	−0.44	7	`111`	aynı
5	11.0	2.20	2.50	+0.30	6	`110`	aynı
6	−5.5	−1.10	−1.50	−0.40	2	`010`	aynı
7	−11.3	−2.26	−2.50	−0.24	1	`001`	aynı
8	−9.4	−1.88	−1.50	+0.38	2	`010`	aynı
9	−6.0	−1.20	−1.50	−0.30	2	`010`	aynı

✔ python3 ile doğrulandı

Bütün hataların mutlak değeri $0.5 = D/2$'den küçük: nicemleme hatası **en fazla yarım şerit** olabilir, çünkü her örnek en yakın merkeze yuvarlanıyor. Gönderilen bit dizisi satır satır: `010 101 111 111 110 010 001 010 010`.

✎ DURUP DÜŞÜN

D5.2 4. örnekte ham değer 19.7 ve PAM 3.94. En yüksek şerit 3.0–4.0 arası. Örnek bu şeride düştüğü için merkez 3.5. Hata neden eksi? Bu, nicemlenmiş değerin gerçeğin altında kaldığını mı gösterir, üstünde mi?

5.4.3 Daha fazla bit, daha az hata

Hata her zaman şerit genişliğinin yarısı ile sınırlı. Bit sayısını artırırsan şeritler incelir ve hata azalır:

Bit	Seviye (2^n)	Şerit genişliği ($10\,\text{V}/2^n$)	En büyük hata (yarım şerit)
3	8	1.25 V	0.625 V
8	256	0.039 V	0.0195 V
16	65 536	0.00015 V	0.000076 V

(−5 V … +5 V aralığı varsayıldı; ek örnek.) ✔ python3 ile doğrulandı

+ EK BİLGİ (KAYNAK DIŞI) — Her ek bit yaklaşık 6 dB kazandırır

Bir bit eklemek seviye sayısını ikiye katlar, şerit genişliğini yarıya indirir; nicemleme gürültüsü gücü dörtte bire düşer. Dörtte bir, $10\log_{10}4 \approx 6$ dB demektir. Yani sinyal-nicemleme-gürültü oranı bit başına yaklaşık 6 dB iyileşir. (Slaytlarda yok.) ✔ python3 ile doğrulandı

5.5 PCM bit hızı: 64 kbps nereden geliyor? ❖ Kaynak: D1 s. 62

∑ TANIM / FORMÜL — PCM bit hızı (D1 s. 62)

$$\text{Bit hızı} = \text{örnek/s} \times \text{bit/örnek} \times \text{kanal}$$

Slayt örneği (tek kanal): örnekleme 8 000 örnek/s, nicemleme 8 bit/örnek → sonuç $8000 \times 8 = 64\,000$ bit/s $= 64$ kbps.
Alt kutu: *"More samples or more bits can improve quality, but also increase storage and transmission cost."* Daha çok örnek ya da daha çok bit kaliteyi artırır ama depolama ve iletim maliyetini de artırır.

Ters çevrilmiş kılıkları (sınavda bunlar da gelir): 64 kbps ve 8 bit/örnek verilirse örnek hızı $64\,000/8 = 8000$ örnek/s; 8000 örnek/s ve 64 kbps verilirse bit/örnek $64\,000/8000 = 8$ olur; stereo (2 kanal) olduğunu unutursan sonucu yarıya düşürürsün. ✔ python3 ile doğrulandı

Formül çarpmanın doğal sonucu: her saniyede f_s örnek var, her örnek n bit, her kanalda aynısı; üstüne kanal sayısı kadar çarp.

▶ ÖRNEK 5.4 — Slayt örneği [D1 s. 62]: telefon sesi

$8000 \times 8 \times 1 = 64\,000$ bit/s $= 64$ kbps. Bu, klasik dijital telefon hattının bir ses kanalının hızıdır. 1 dakikalık konuşmanın boyutu: $64\,000 \times 60/8 = 480\,000$ bayt ≈ 480 kB.
✔ python3 ile doğrulandı

▶ ÖRNEK 5.5 — Ek örnek: müzik CD'si (stereo)

$44\,100$ örnek/s, 16 bit/örnek, 2 kanal: $44\,100 \times 16 \times 2 = 1\,411\,200$ bit/s ≈ 1.41 Mbps. 3 dakikalık bir şarkı: $1\,411\,200 \times 180/8 \approx 31.75$ MB. ✔ python3 ile doğrulandı

✎ KENDİN ÇÖZ — Yarı çözümlü örnek

Y5.1 Bir ses sinyalinin en yüksek frekansı 6 kHz. 10 bitle nicemleniyor, tek kanal.

1. En az örnekleme hızı: $f_s > 2 \times 6000 = ?$ örnek/s. Yuvarlak bir değer seç: 13 000.
2. Seviye sayısı: $2^{10} = ?$
3. Bit hızı: $13\,000 \times 10 \times 1 = ?$ bit/s.

5.6 Alternatif: delta modülasyonu

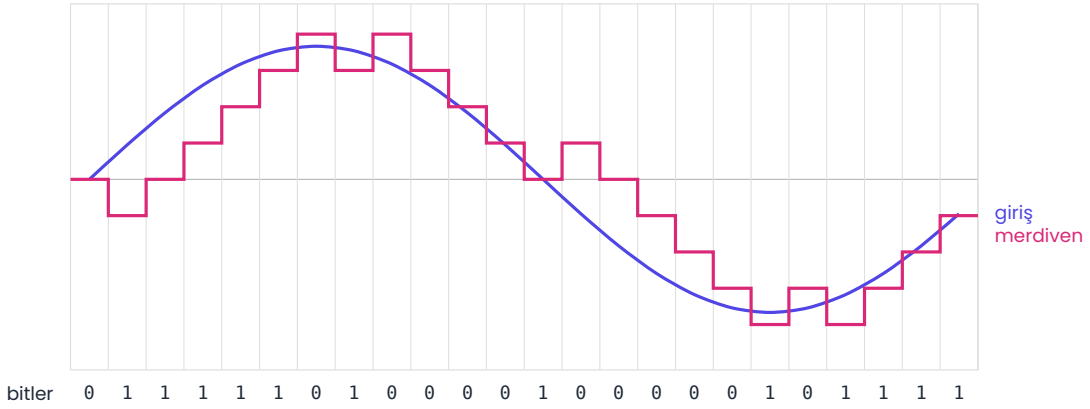
❖ Kaynak: D1 s. 63–64

PCM her örnek için **tam değeri** yazıyor ve bunun için çok bit harcıyor. Başka bir fikir: değer ne olursa olsun, bir önceki örneğe göre **yukarı mı gitti, aşağı mı**? Sadece bunu söylemek **tek bit** yeter.

Bir benzetme: kör bir yürüyüşçüye bir tepenin yüksekliğini tarif ediyorsun. PCM: "şu an 342 m'desin." Delta: "bir adım çık" ya da "bir adım in". Yürüyüşçü adımları sayarak yüksekliği yeniden kurar.

∑ TANIM / FORMÜL — Delta modülasyonu (DM)

Her örnekleme anında: 1. Analog giriş, **tahmin** (merdiven) ile karşılaştırılır. 2. Giriş tahminden **yüksekse** bit **1** gönderilir ve tahmin bir **basamak (d) yukarı** çıkar. 3. Giriş tahminden **düşükse** bit **0** gönderilir ve tahmin bir **basamak aşağı** iner.
Slayt: *"At each sample, the estimate moves up or down; often one bit per sample is sufficient."*



Şekil 5.4: Delta modülasyonu: mor eğri analog giriş, pembe merdiven alıcının/göndericinin tahmini. Giriş tahminden yüksekse bit 1 ve merdiven bir basamak çıkar; değilse bit 0 ve bir basamak iner (basamak d = 0.6). Örnek çizimdir.

Slayt 63'teki şekil de aynı mantık: kareli kâğıtta pembe analog eğri ve onu izleyen gri merdiven; altta üretilen ikili veri (`0 1 1 1 1 1 1 0 0 0 0 0 0 1 1`). Yukarı çıkan her adım 1, aşağı inen her adım 0'a karşılık geliyor. Eğri yükselirken 1'ler, alçalırken 0'lar peşpeşe geliyor.

5.6.1 Delta modülatörün iç yapısı (D1 s. 64)

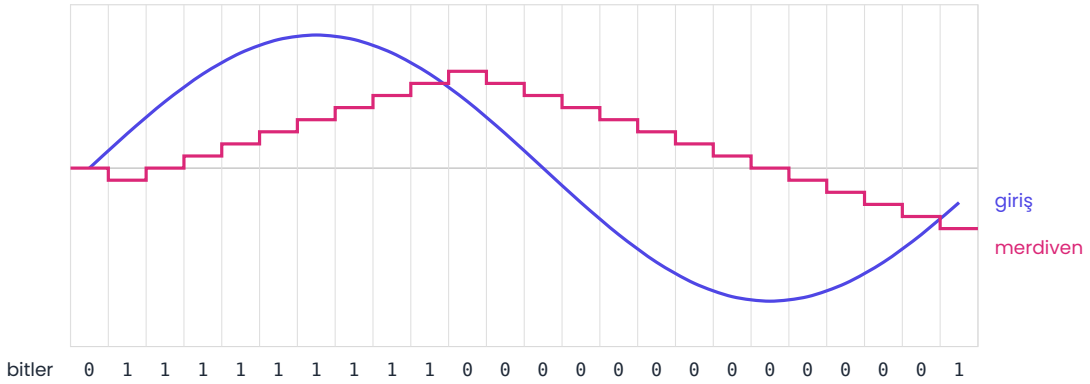
Slayttaki blok şema dört kutu içeriyor:

Blok	Görevi
Karşılaştırıcı (comparator)	Analog girişi tahminle karşılaştırır; sonuç yön kararıdır (1 ya da 0) ve çıkışa gider
Merdiven üretici (staircase maker)	Karardan gelen bite göre tahmini $\pm d$ günceller
Gecikme birimi (delay unit)	Bir önceki tahmini saklar, bir sonraki örneğe taşır
(Karşılaştırıcıya geri besleme)	Güncellenmiş tahmin tekrar karşılaştırıcıya gider

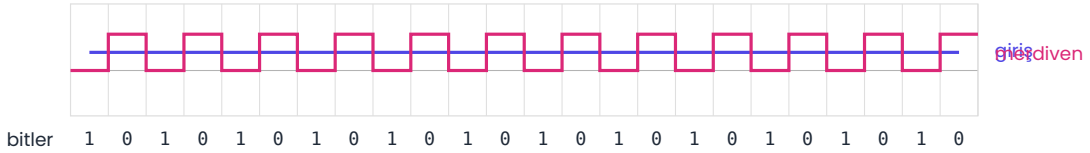
Slayt alt kutusu bu işlevi *"the integrator updates the previous estimate step by step"* diye anlatıyor (integratör = basamakları toplayarak tahmini güncelleyen devre). Şekilde aynı blok **staircase maker** diye geçiyor; ikisi aynı işlevi anlatıyor (bkz. Hocaya Sorulacaklar).

Alıcı tarafı: Alıcı aynı merdiven üreticiyi kullanır: gelen her bit için bir basamak çıkar ya da iner; elde edilen merdiveni (genelde bir alçak geçiren süzgeçle) pürüzsüzleştirir ve analog dalgayı geri kazanır. Yani alıcı, göndericinin içindeki merdiveni tekrar kuruyor.

5.6.2 DM'nin iki hatası (D1 s. 65)



Şekil 5.5: Eğim aşırı yüklenmesi (slope overload): basamak (d = 0.2) çok küçük; giriş hızlı yükselince merdiven yetişemez ve bitler uzun süre 1 kalır.



Şekil 5.6: Granüler gürültü (granular noise): giriş neredeyse sabit, ama basamak (d = 0.6) büyük; merdiven girişin etrafında 1-0-1-0 diye zıplar.

Hata	Ne zaman olur?	Sonuç
Eğim aşırı yüklenmesi (slope overload)	Giriş, merdivenin çıkabildiğinden **hızlı** değişirse (basamak d küçük)	Merdiven girişten geride kalır; bitler uzun süre aynı kalır
Granüler gürültü (granular noise)	Giriş **yavaş** değişirken ya da sabitken basamak d **büyükse**	Merdiven girişin etrafında 1-0-1-0 diye sürekli zıplar

İkisi zıt yönlü bir takas: d'yi büyütürsen eğim aşırı yüklenmesi azalır ama granüler gürültü artar; d'yi

küçültürsen tersi.

+ EK BİLGİ (KAYNAK DIŞI) — Eğim aşırı yüklenmesi ne zaman başlar?

Bir basamak her örnekleme aralığında ($T = 1/f_s$) en fazla d kadar yükselebilir; merdivenin en büyük eğimi d/T'dir. Giriş sinüs ise $A\sin(2\pi ft)$ en dik eğimi $2\pi fA$'dır. Eğim aşırı yüklenmesi olmaması için:

$$\frac{d}{T} \geq 2\pi f A.$$

Örnek: $A = 1$ V, $f = 1$ kHz, $f_s = 8$ kHz ($T = 125\,\mu$s). Gerekli $d \geq 2\pi \cdot 1000 \cdot 1 \cdot 125 \times 10^{-6} \approx 0.785$ V. ✔ python3 ile doğrulandı
Bu hesap slaytlarda yok; ama "neden DM'de örnekleme hızı genelde Nyquist'ten çok yüksek seçilir?" sorusunun cevabı da buradan çıkar: T küçülünce aynı sinyal için gereken d küçülür.

5.7 PCM ile delta modülasyonu karşılaştırması

❖ Kaynak: D1 s. 65

Özellik	PCM	Delta modülasyonu
Kodlanan bilgi	Nicemlenmiş **örnek genliği**	Bir önceki tahmine göre **artış/azalış**
Bit sayısı	Örnek başına **birden çok** bit	Genelde örnek başına **1 bit**
Güçlü yan	Doğrudan ve yaygın kullanılan temsil	Basit kodlayıcı ve bit akışı
Ana hata	**Nicemleme gürültüsü**	**Eğim aşırı yüklenmesi** ya da **granüler gürültü**

Dikkat: DM'nin "1 bit" olması "çok daha az bit" demek değil; DM'de örnekleme hızı genelde yükseltilir. Örneğin DM 64 000 örnek/s × 1 bit ile 64 kbps üretir; PCM 8 000 × 8 ile aynı 64 kbps'i. Hangisinin kalitesi daha iyi olduğu d ve sinyale bağlı. (Ek yorum.) ✔ python3 ile doğrulandı

→ BU KONUYA NEDEN GEÇİYORUZ?

PCM'in çıkışı bir bit akışı; o akış 4. bölümdeki hat kodlarından biriyle tele dökülür (ör. Manchester ya da NRZ). Dolayısıyla "sesim kabloya nasıl çıktı?" sorusunun tam cevabı: **ses → (PCM) bitler → (hat kodlama) sinyal → kanal**.

✎ KENDİN ÇÖZ — Kendin çöz

K5.1 Bir sinyalin en yüksek frekansı 3.4 kHz. Nyquist'e göre minimum örnekleme hızı? 4 bitle nicemleme ve 8 000 örnek/s ile bit hızı nedir?
K5.2 12 bitlik bir nicemleyici kaç seviye kullanır? Kaç seviye için 10 bit gerekir (en yakın üst kuvvet)?
K5.3 Aralık $-4 \ldots +4$ (8 seviye, merkezler $\pm 0.5, \pm 1.5, \pm 2.5, \pm 3.5$). 2.9 değerli örneğin nicemlenmiş değerini, hatasını, kodunu ve kelimesini bul.
K5.4 1000 Hz'lik bir ton 1500 örnek/s ile örnekleniyor. Alias frekansı nedir?
K5.5 DM'de giriş bir süre sabit kalıyor ve çıkış bit dizisi `1010101010`. Bu hangi hata? Ne yaparsın?

5.8 Bağlantılar

- **Önceki bölümler → bu bölüm:** 2. bölümde bit hızı = sembol hızı × $\log_2$(M); burada PCM bit hızı = örnek/s × bit/örnek × kanal: aynı "hız = sıklık × her seferdeki bit" fikri. 4. bölümde bit akışının nasıl tele döküldüğünü gördük; burada o akışı **üretiyoruz**.
- **Bu bölüm → sonraki bölüm:** Elimizde artık bit akışı var; peki bu bitler kabloya **aynı anda mı yoksa tek tek mi** gider? 6. bölümün konusu.
- **İleride:** Sinyaller ve Sistemler, Sayısal Sinyal İşleme ve Multimedya derslerinde örnekleme, süzgeçleme ve nicemleme bu bölümün derinleşmiş hâli.

⚠ SINAVDA DİKKAT

- **Nyquist:** $f_s > 2f_{max}$; yetersiz örnekleme **örtüşme** yaratır (yüksek frekans düşük frekans gibi görünür).
- **Nicemleme hatası** en fazla **yarım şerit** olabilir.
- **PCM bit hızı** = örnek/s × bit/örnek × kanal. 8 000 × 8 = 64 kbps.
- **DM:** 1 bit/örnek; hata türleri **eğim aşırı yüklenmesi** (basamak küçük) ve **granüler gürültü** (basamak büyük).
- **Slayt 61'deki ilk hata**: −0.38 değil, −0.28.

❖ BU KONUDAN NASIL SORU GELİR?

1. Hesap: *"A voice signal is sampled at 8000 samples/s with 8 bits per sample. What is the bit rate?"* (64 kbps)
2. Kavramsal: *"Explain aliasing and how to prevent it."* İpucu: yetersiz örnekleme, yüksek frekans düşük gibi görünür; önlem: yüksek örnekleme hızı + alçak geçiren süzgeç.
3. Karşılaştırma: *"Compare PCM and delta modulation."* İpucu: s. 65 tablosundaki dört satır.
4. Hata türü: *"What is slope overload and when does it occur?"* İpucu: giriş merdivenden hızlı değişir.
5. Tablo: Bir örnek için PAM değeri, nicemlenmiş değer, hata, kod ve kelimeyi bul (slayt 61 gibi).

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "Nyquist hızı, sinyalin en yüksek frekansına eşittir." → **Doğrusu:** İki katından büyük olmalı: $f_s > 2f_{max}$. → **Neden:** Her dalgada tepe ve çukuru yakalamak gerek.
- **Yanlış:** "Nicemleme hatası kaldırılabilir." → **Doğrusu:** Yalnızca azaltılır (bit artırarak); dönüşüm kayıplıdır (D1 s. 57).
- **Yanlış:** "8 bit 8 seviye demektir." → **Doğrusu:** $2^8 = 256$ seviye. → **Neden:** Seviye sayısı bit sayısının üssüdür.
- **Yanlış:** "DM tam genliği iletir." → **Doğrusu:** Yalnızca yönü (artış/azalış) iletir.
- **Yanlış:** "Granüler gürültü, giriş hızlı değişince olur." → **Doğrusu:** Giriş yavaş/sabitken büyük basamak yüzünden olur; hızlı değişim **eğim aşırı yüklenmesi**dir.

⚡ HIZLI TEKRAR

- Sayısallaştırma: saklama, işleme, kopyalama, hata denetimi, sıkıştırma, şifreleme, tekrarlayıcı; ama dönüşüm kayıplıdır ve maliyetlidir.
- **PCM:** örnekle → nicemle → kodla. Alıcı yaklaşık analog dalgayı geri üretir.
- **Nyquist:** $f_s > 2f_{max}$. **Örtüşme:** yetersiz örneklemede yüksek frekans düşük görünür.
- Nicemleme: en yakın seviyeye yuvarla; hata ≤ yarım şerit; n bit = 2^n seviye.
- **Bit hızı** = örnek/s × bit/örnek × kanal. Ses: $8000 \times 8 = 64$ kbps.
- **DM:** 1 bit/örnek, yön bilgisi; eğim aşırı yüklenmesi (hızlı giriş) ve granüler gürültü (yavaş giriş).
- Ezberle: s. 65 tablosu. Türetebil: seviye sayısı, hata sınırı, alias frekansı.

Bu bölümden sonra şunları yapabiliyor olmalısın: PCM'in üç adımını anlatmak; Nyquist ve örtüşmeyi açıklamak; nicemleme tablosunu okumak; PCM bit hızını hesaplamak; DM ile PCM'i karşılaştırmak.

5.9 Bölüm cevapları

D5.1 $f_s > 2 \times 10\,000 = 20\,000$ örnek/s. Pratikte süzgeçlerin ideal olmadığı ve güvenlik payı gerektiği için daha yüksek seçilir.

D5.2 Hata = nicemlenmiş − gerçek $= 3.50 - 3.94 = -0.44$. Eksi, nicemlenmiş değerin gerçeğin **altında** kaldığını gösterir (en yüksek şeridin merkezi, örneğin üst kısmının altında).

Y5.1 (1) $f_s > 12\,000$; seçilen 13 000. (2) $2^{10} = 1024$ seviye. (3) $13\,000 \times 10 \times 1 = 130\,000$ bit/s $= 130$ kbps. ✔ python3 ile doğrulandı

K5.1 $f_s > 6800$ örnek/s. 4 bit, 8 000 örnek/s: $8000 \times 4 = 32\,000$ bit/s $= 32$ kbps.

K5.2 12 bit → $2^{12} = 4096$ seviye. 10 bit → 1024 seviye.

K5.3 2.9, 2–3 şeridinde; merkez 2.5. Nicemlenmiş değer **2.5**; hata $2.5 - 2.9 = -0.4$; kod: $k - 3.5 = 2.5 \Rightarrow k = 6$; kelime `110`. ✔ python3 ile doğrulandı

K5.4 $f_s = 1500$; Nyquist sınırı 750 Hz. 1000 Hz bu sınırın üstünde → alias $|1000 - 1500| = 500$ Hz. ✔ python3 ile doğrulandı

K5.5 Granüler gürültü: basamak giriş için fazla büyük. Basamak boyutunu (d) küçült.

Sınav sorusu cevabı: (1) $8000 \times 8 = 64\,000$ bit/s.

BÖLÜM 6

Paralel mi, Seri mi? İletim Modları

◆ **BU BÖLÜMDE** Okuma süresi: 30 dk Zorluk: ●○○

- Paralel ve seri iletimin farkını, artılarını ve eksilerini
- Seri iletimin üç zamanlama düzenini: asenkron, senkron, izokron
- Neden modern yüksek hızlı bağlantıların çoğunun seri olduğunu

SINAV ÖNCELİĞİ: ORTA ❖ Kaynak: D1 s. 66–70

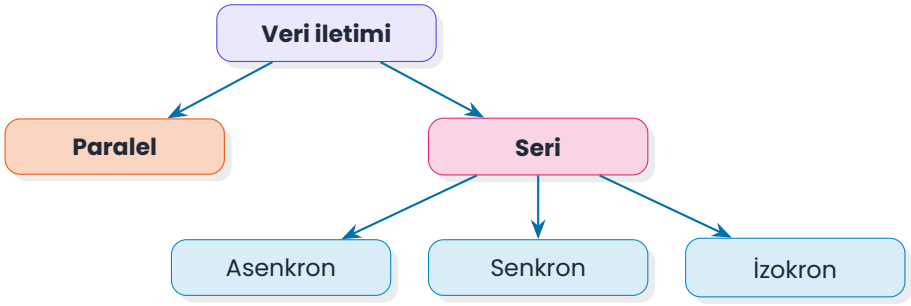
Bu bölüm kısa ve kavram ağırlıklı; sınavda karşılaştırma ve tanım sorusu olarak gelir. Slayt 70'in tablosunu hücre hücre bilmek yeterli.

→ **BU KONUYA NEDEN GEÇİYORUZ?**

Elimizde artık bitler var (1. bölüm), onların sinyale dönüşümü var (2. ve 4. bölüm) ve analog veriyi de bite çevirebiliyoruz (5. bölüm). Kalan soru çok basit görünüyor ama önemli: **8 biti karşıya nasıl göndereceğiz? Hepsini aynı anda, 8 ayrı telle mi; yoksa sırayla, tek telle mi?**

6.1 İletim modları ailesi ❖ Kaynak: D1 s. 67

Slayt 67 bir ağaç gösteriyor:

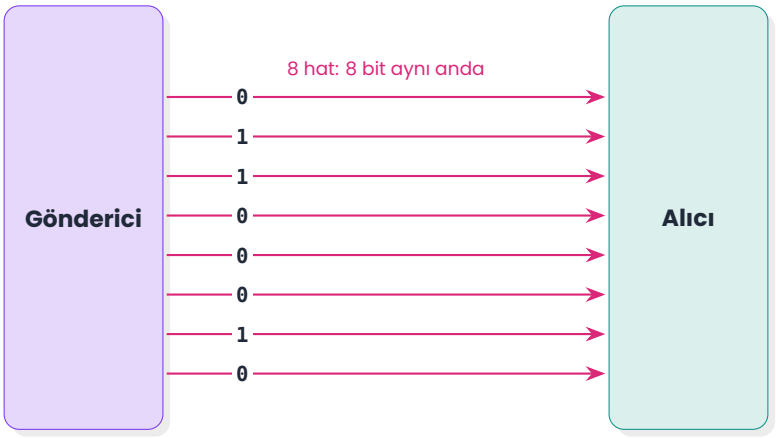


Şekil 6.1: İletim modlarının sınıflandırılması (D1 s. 67)

Alt kutu: *"Serial transmission may be organized using asynchronous, synchronous, or isochronous timing."* Yani ağaçta **iki ana dal** var (paralel ve seri); seri dalın da **üç zamanlama düzeni** var. Paralelin altı boş: zamanlama sorunu orada farklı biçimde çözülüyor (bir ortak saat hattı gibi; ek bilgi).

6.2 Paralel iletim: çok bit, çok hat ❖ Kaynak: D1 s. 68

Slayt örneği: 8 bitlik `01100010` dizisi (D1 s. 68). Gönderici ile alıcı arasında **8 ayrı hat** var; her bit kendi hattından aynı anda gidiyor.



Şekil 6.2: Paralel iletim: 01100010 örneği, 8 bit 8 hattan birden gider (D1 s. 68'deki şeklin sade çizimi).

Slayttaki balon notları: *"The 8 bits are sent together"* (8 bit birlikte gönderilir) ve *"We need eight lines"* (sekiz hat gerekir).

∑ TANIM / FORMÜL — Paralel iletim (parallel transmission)

Bir saat adımında **birden çok bit** gönderilir; her bit ayrı bir hattan gider. Bit sayısı kadar hat gerekir.
Slayt cümlesi: *"Multiple bits can be sent per clock step; the line count and timing-alignment cost may be acceptable over short distances."* Hat sayısı ve zamanlama hizalama maliyeti **kısa mesafelerde** kabul edilebilir.

6.2.1 Artı ve eksileri

	Paralel iletim
Artı	Her saat adımında çok bit gider; bir bitlik hızın katlarında veri hızı (ör. 8 hat → 8 kat)
Eksi	Çok hat = pahalı kablo ve konnektör; hatlar arası **zamanlama hizalaması** gerekir
Nerede?	**Kısa mesafe**: bilgisayarın iç yolları, bellek ve işlemci arası (ek bilgi)

+ EK BİLGİ (KAYNAK DIŞI) — Hatlar neden hizadan çıkar? (skew)

Hatların uzunlukları ve elektriksel özellikleri birebir aynı olamaz; aynı anda yola çıkan bitler alıcıya biraz **farklı zamanlarda** varabilir. Buna kayma (**skew**) denir. Saat hızı arttıkça bit süresi kısalır ve aynı kayma bitleri birbirine karıştırır. Üstüne komşu hatlar birbirine gürültü sızdırır (3. bölümdeki çapraz karışma). Bu yüzden uzun mesafede ve yüksek hızda paralel zorlaşır.

6.3 Seri iletim: bitler sırayla ❖ Kaynak: D1 s. 69

Aynı `01100010` dizisi bu kez **tek hat** üzerinden, bitler peş peşe gidiyor. Gönderici tarafta bir **paralel/seri çevirici** (**parallel/serial converter**) 8 biti sıraya dizer; alıcı tarafta **seri/paralel çevirici** (**serial/parallel converter**) sırayla gelen bitleri tekrar 8 bitlik bir kelimeye toplar.



Şekil 6.3: Seri iletim (D1 s. 69): bir hat, 8 bit art arda

Balon notları: *"The 8 bits are sent one after another"* (8 bit art arda gönderilir) ve *"We need only one line (wire)"* (yalnızca bir hat gerekir). Alt kutu: *"It uses fewer physical lines and is common in modern high-speed links over both short and long distances."* Daha az fiziksel hat kullanır ve modern yüksek hızlı bağlantılarda hem kısa hem uzun mesafelerde yaygındır.

6.3.1 Aynı bit, farklı yol: süre karşılaştırması

▶ ÖRNEK 6.1 — Ek örnek: 8 bitlik karakteri göndermek ne kadar sürer?

`01100010`, ASCII'de küçük **b** harfidir ($98 = 64 + 32 + 2$). Saat 1 MHz olsun: bir bit adımı 1 µs.

- **Paralel (8 hat):** 8 bit **1 adımda** gider → **1 µs**. Hız $= 8 \times 1$ Mbit/s $= 8$ Mbps.
- **Seri (1 hat), aynı saatle:** 8 bit **8 adımda** gider → **8 µs**. Hız $= 1$ Mbps.
- **Seri, 8 MHz saatle:** 8 adım $\times$ 0.125 µs $= 1$ µs → paralelle **aynı süre**, ama tek hat ve 8 kat hızlı saat.

✔ python3 ile doğrulandı

Çıkarım: Seri iletim hat sayısı yerine **saat hızından** ödün verir. Teknoloji yüksek saat hızlarını güvenilir kıldıkça (iyi hat kodları, saat kurtarma, 4. bölüm), seri avantajlı hâle geldi.

∑ TANIM / FORMÜL — Seri iletim (serial transmission)

Bitler **tek bir hat üzerinden sırayla** gönderilir. Gönderici tarafta paralel/seri çevirici, alıcı tarafta seri/paralel çevirici vardır.
Artı: Az hat, ucuz kablo, uzun mesafede kolay. **Eksi:** Aynı veri hızı için daha hızlı saat gerekir; bit sınırlarını alıcının bulması (senkronizasyon) şarttır.

✎ DURUP DÜŞÜN

D6.1 Seri iletimde alıcı hangi bitin "8 bitlik kelimenin başı" olduğunu nereden bilecek? (Aşağıda üç yanıt var: asenkron, senkron, izokron.)

6.3.2 Paralel ile seriyi yan yana

	Paralel	Seri
Hat sayısı	Bit sayısı kadar (ör. 8)	Tek (ya da çift yönlü için birkaç)

	Paralel	Seri
Bir adımda giden bit	Çok	Bir
Zamanlama sorunu	Hatlar arası hizalama (skew)	Alıcı saatini bulma (senkronizasyon)
Uygun mesafe	Kısa	Kısa ve uzun
Maliyet (kablo)	Yüksek	Düşük

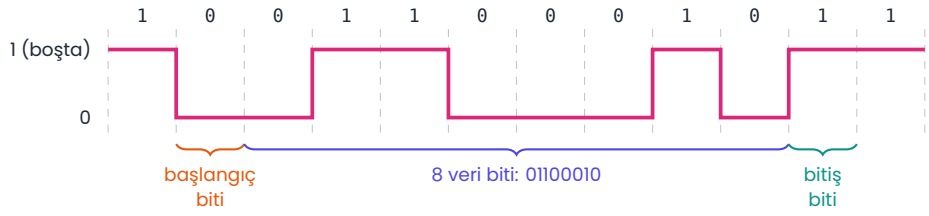
6.4 Seri iletimin üç zamanlama düzeni ❖ Kaynak: D1 s. 70

Seri iletimin asıl problemi 4. bölümdeki saat meselesi: alıcı hangi anda hangi bite bakacağını nasıl bilecek? Üç farklı cevap var. Slayt 70'in tablosu:

Mod	Zamanlama	Tipik yapı	Öncelik
Asenkron (asynchronous)	Karakterler **bağımsız zamanlarda** gelir	**Başlangıç/bitiş biti** (start/stop)	**Basitlik**
Senkron (synchronous)	Sürekli akış; **paylaşılan ya da kurtarılan** zamanlama	**Çerçeveler** ve **saat kurtarma**	**Verimlilik**
İzokron (isochronous)	Teslim aralıkları **düzenli** tutulur	**Zamana duyarlı** akış	**Gecikme / jitter sınırı**

6.4.1 Asenkron: karakter karakter

İsmin kökü: "senkron değil", yani gönderici ile alıcı **sürekli** aynı saati paylaşmaz. Karakterler **istedikleri zaman** gelir (ör. klavyede her tuşa bastığında). Alıcıyı uyandırmak için her karakterin başına bir **başlangıç biti**, sonuna bir **bitiş biti** eklenir.



Şekil 6.4: Asenkron bir karakter çerçevesi örneği: hat boştayken 1 seviyesinde bekler; başlangıç biti (0) alıcıyı uyandırır; 8 veri biti gelir; bitiş biti (1) çerçeveyi kapatır (örnek çizimdir, D1 s. 70'te yalnızca "start/stop bits" ifadesi var).

▶ ÖRNEK 6.2 — Ek örnek: başlangıç ve bitiş bitlerinin bedeli

Her 8 bitlik karakter için 1 başlangıç + 1 bitiş biti = 10 bit gider. Faydalı veri oranı $8/10 = 80\%$. 9600 bit/s'lik bir hatta saniyede $9600/10 = 960$ karakter geçer. ✔ python3 ile doğrulandı
Çıkarım: Asenkron basit ama **%20 fazladan bit** harcar. İşte "öncelik: basitlik" ifadesinin bedeli.

6.4.2 Senkron: sürekli akış, çerçeveler

Senkron iletimde bitler **aralıksız bir akış** hâlinde gider; veriler **çerçevelere** (**frame**) bölünür ve alıcı **saatini akıştan kurtarır** (4. bölümde gördüğümüz Manchester gibi kendinden saatli kodlar

bunun için ideal). Her karakter için ayrı başlangıç/bitiş biti yok; sadece her çerçevede küçük bir başlık/sonluk.

▶ ÖRNEK 6.3 — Ek örnek: senkron iletimin verimi

1000 baytlık veri taşıyan bir çerçeve, 20 bayt başlık ve sonluk ekliyor. Faydalı oran $1000/1020 \approx 98\%$. Asenkronun %80'iyle karşılaştır: büyük veri akışlarında senkron çok daha verimli. ✔ python3 ile doğrulandı

6.4.3 İzokron: düzenli teslim

İzokron (**isochronous**) "eşit zamanlı" demek. Burada öncelik **hız ya da verim değil**, bitlerin **düzenli aralıklarla** teslim edilmesi. Ses ve görüntü akışı gibi **zamana duyarlı** akışlar için: ses her 125 µs'de bir geliyorsa, 125 µs aralığın tutması şart; gecikme (**latency**) ve **jitter** (teslim aralığındaki oynama) bir sınırın altında kalmalı.

▶ ÖRNEK 6.4 — Ek örnek: telefon sesi için izokron aralık

5. bölümde 8 000 örnek/s demiştik. Bir örnekten diğerine süre $1/8000 = 125\ \mu$s. Bu örnekler alıcıya **her 125 µs'de bir** düzenli ulaşmalı; bir örnek geç gelirse ses "kesilir" ya da cızırdar. ✔ python3 ile doğrulandı

★ MERAKLISINA (SINAV DIŞI) — Günlük hayatta üç mod

Eski seri port ve klavye gibi cihazlarda asenkron (UART) çerçeveler çok yaygındı; Ethernet ve disk bağlantıları gibi hızlı veri yolları senkron çerçeveleme kullanır; USB'nin ses/video için ayrılmış "izokron" aktarım türü, düzenli teslimi garanti etmeye çalışır. (Slaytlarda yok.)

✎ KENDİN ÇÖZ — Kendin çöz

K6.1 16 bitlik bir kelimeyi 16 hatlı paralel yol ile 2 MHz saatle göndermek kaç µs sürer? Aynı kelimeyi tek hatlı seri ile 2 MHz saatle göndermek kaç µs sürer?
K6.2 Asenkron iletimde karakter başına 1 başlangıç, 8 veri, 2 bitiş biti var. Veri verimi yüzde kaçtır?
K6.3 Aşağıdakileri uygun moda eşleştir: (a) klavyeden basılan tuş, (b) ses akışı, (c) büyük dosya aktarımı.

6.5 Bağlantılar

- **Önceki bölümler → bu bölüm:** 4. bölümdeki saat kayması ve senkronizasyon, seri iletimin **ana sorunu**dur. 5. bölümdeki 8000 örnek/s, izokron aralığı (125 µs) belirliyor.
- **Bu bölüm → sonraki hafta:** Slayt 72: *Data transmission and protocols in networks*, yani bitler artık ağda protokollerle taşınacak. Bu bölümdeki çerçeve (**frame**) fikri orada önem kazanıyor.
- **İleride:** Bilgisayar Ağları, Bilgisayar Organizasyonu ve Gömülü Sistemler derslerinde (UART, SPI, I²C, PCIe, Ethernet) bu bölümün yaygın örneklerini göreceksin.

⚠ SINAVDA DİKKAT

- **Paralel:** çok hat, kısa mesafe; hat sayısı ve zamanlama hizalaması maliyeti.
- **Seri:** tek hat; hem kısa hem uzun mesafe; modern yüksek hızlı bağlantılarda yaygın.
- **Üç seri mod:** asenkron (start/stop bitleri, basitlik), senkron (çerçeve + saat kurtarma, verimlilik), izokron (düzenli teslim, gecikme/jitter).
- Slayt 70 tablosundaki 4 sütunu hücre hücre bil.

❖ BU KONUDAN NASIL SORU GELİR?

1. Karşılaştırma: *"Compare parallel and serial transmission."* İpucu: hat sayısı, mesafe, zamanlama sorunu.
2. Tanım: *"What are start and stop bits used for?"* İpucu: asenkron, alıcıyı uyandırma ve çerçeveyi kapatma.
3. Eşleştirme: Asenkron, senkron, izokron için "öncelik" ve "tipik yapı" sütunlarını doldur.
4. Hesap: *"A serial link sends 1 start, 8 data and 1 stop bit per character at 9600 bit/s. How many characters per second?"* (960)

✘ SIK YAPILAN HATA — Sık yapılan hatalar

- **Yanlış:** "Paralel iletim her zaman seriden hızlıdır." → **Doğrusu:** Aynı saat hızında paralel daha çok bit taşır ama yüksek saat hızında hat hizalaması bozulur; seri çok yüksek saat hızına çıkabilir. → **Neden:** Hız = hat sayısı × saat hızı; ikisi de sınırlı.
- **Yanlış:** "Asenkron, senkron olmayan, yani saati olmayan iletimdir." → **Doğrusu:** Alıcının her karakter başında yeniden hizalandığı iletimdir.
- **Yanlış:** "İzokron, en hızlı mod demektir." → **Doğrusu:** Düzenli teslim demektir; öncelik gecikme/jitter sınırıdır.

⚡ HIZLI TEKRAR

- **Paralel:** bir adımda çok bit, bit sayısı kadar hat; kısa mesafe.
- **Seri:** tek hat, bitler sırayla; paralel/seri ve seri/paralel çevirici; modern yüksek hızlı bağlantılarda yaygın.
- **Asenkron:** start/stop bitleri, basitlik. **Senkron:** çerçeveler + saat kurtarma, verimlilik. **İzokron:** düzenli teslim, gecikme/jitter sınırı.
- 8 bit paralel: 8 hat, 1 adım. 8 bit seri: 1 hat, 8 adım.

Bu bölümden sonra şunları yapabiliyor olmalısın: Paralel ile seriyi karşılaştırmak; üç seri modu tablodan açıklamak; basit süre ve verim hesapları yapmak.

6.6 Bölüm cevapları

D6.1 Asenkronda başlangıç bitinden; senkronda çerçeve başlığından ve saat kurtarmadan; izokronda düzenli zamanlama ve (genelde) çerçeve yapısından.

K6.1 Saat 2 MHz → adım $0.5\ \mu$s. Paralel: 1 adım → $0.5\ \mu$s. Seri: 16 adım → $8\ \mu$s. ✔ python3 ile doğrulandı

K6.2 Karakter başına $1 + 8 + 2 = 11$ bit; veri 8 bit → $8/11 \approx 72.7\%$. ✔ python3 ile doğrulandı

K6.3 (a) asenkron, (b) izokron, (c) senkron.

Sınav sorusu cevabı: (4) $9600/10 = 960$ karakter/s.

Konular Arası Bağlantılar

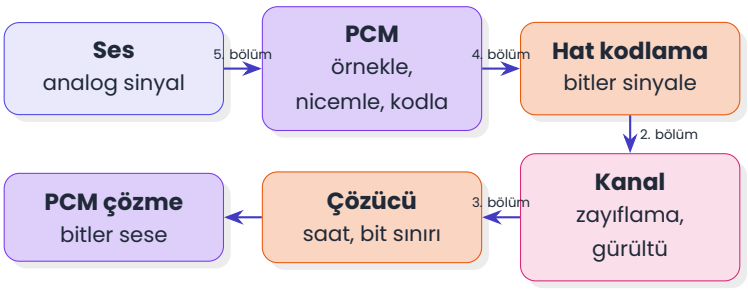
❖ Kaynak: D1 s. 71–72

Slayt 71 haftanın altı ana fikrini listeliyor. Aşağıda her birini hangi bölümde işlediğimizi ve birbirine nasıl bağlandığını görüyorsun.

Slayt 71'deki ana fikir	Bölüm	Bu fikri tek cümleyle
İkiye tümleyen, işaretli tam sayılar için **tek sıfır** ve **basit aritmetik** sağlar	1. Negatif sayılar	Çıkarma da toplamadır; taşma işaretlerle denetlenir
Veri soyut bilgidir; sinyal onu fiziksel ortamda temsil eder	2. Veri ve sinyal	Bit hızı = sembol hızı × $\log_2(M)$
Zayıflama, bozulma ve gürültü alıcının kararını etkiler	3. Kanal bozulmaları	Düşük SNR hata olasılığını yükseltir
Hat kodlama bitleri seviye ve geçişlere eşler ve zamanlamayı destekler	4. Hat kodlama	Uzun aynı-bit dizileri senkronizasyonu bozar
PCM örnekler, nicemler, kodlar; delta modülasyonu değişimin yönünü izler	5. PCM ve DM	8000 × 8 = 64 kbps
Paralel ve seri iletim farklı kablolama, zamanlama ve ölçeklenme takasları sunar	6. İletim modları	Hat sayısı mı, saat hızı mı?

Büyük resim: bir sesin yolculuğu

Bu haftanın bütün konularını tek bir hikâyede toplayabilirsin. Telefonda bir cümle söylediğini düşün:



Şekil 6.5: Bir sesin yolculuğu: analogdan bite, bitten sinyale, kanaldan geri (bu kitabın bölümleri bir arada)

1. Ses mikrofona girer, **analog sinyal** olur (2. bölüm).
2. **PCM** onu örnekler, nicemler, bitlere çevirir: örneğin 8000 × 8 = 64 kbps (5. bölüm).
3. Bitler bir **hat kodu** ile voltaj seviyelerine ve geçişlere çevrilir (4. bölüm).
4. Sinyal **kanalda** zayıflar, bozulur, gürültü karışır (3. bölüm).
5. Alıcı sinyali örnekler, **saatini** bulur, bitleri geri kazanır (4. bölüm).

6. Bitler **işaretli sayılar** ya da başka verilerle (1. bölüm) çözülür ve PCM tersine çalışarak sesi üretir.
7. Bu bitler tek hatta sırayla (**seri**) ya da çok hatta birden (**paralel**) gider (6. bölüm).

Haftanın kalıpları

Altı bölümde aynı üç fikir tekrar tekrar karşına çıktı. Bunları fark edersen sınavda yeni bir soruyla bile karşılaşsan yönünü bulursun:

- **Bit sayısı = seçenek sayısı:** n bit 2^n örüntü (1. bölüm), $\log_2 M$ bit/sembol (2. bölüm), 2^n nicemleme seviyesi (5. bölüm).
- **Hız ve güvenilirlik takası:** Daha çok seviye = daha yüksek hız ama gürültüye zayıf (2.); Manchester = güvenilir ama geniş bant (4.); daha çok bit = iyi kalite ama pahalı (5.); paralel = çok hat, seri = hızlı saat (6.).
- **Zamanlama her şeydir:** Saat kayması (4.), örnekleme anı (5.), hat hizalaması ve izokron teslim (6.).

Gelecek hafta

Slayt 72: **Data transmission and protocols in networks** (Ağlarda veri iletimi ve protokoller). Bu haftanın seri iletim, çerçeve ve protokol fikirleri orada ağ katmanlarına dönüşecek.

Sınav Öncesi Hızlı Tekrar

Bu bölüm anlatımın yerini **tutmaz**; anlatımı okuduktan sonra son bir gün önce hızlı bakman için. Tanımadığın bir şey varsa ilgili bölüme dön.

⚡ **HIZLI TEKRAR**

1. İşaretli sayılar SINAV ÖNCELİĞİ: YÜKSEK - Bit örüntüsünün anlamı gösterime bağlı. SM: işaret + büyüklük. 1C: her biti ters çevir. 2C: ters çevir + 1 ($= 2^n - x$). - Aralıklar: SM ve 1C $-(2^{n-1}-1) \ldots +(2^{n-1}-1)$, iki sıfır. 2C $-2^{n-1} \ldots +(2^{n-1}-1)$, tek sıfır. - 4 bit 2C: −8…+7. 8 bit: −128…+127. 16 bit: −32 768…+32 767. - Okuma: en soldaki bit 1 ise ters çevir, 1 ekle, eksi koy. `1111 0110` = −10; `1000 0000` = −128. - Çıkarma: $A - B = A + (-B)$; dış eldeyi at. 58 − 40 = 18. - Taşma: **aynı işaretli** iki sayının toplamı ters işaret verirse. Farklı işaretlilerde asla. Dış elde ≠ taşma. 100 + 60 → −96 (taşma).

⚡ **HIZLI TEKRAR**

2. Veri ve sinyal SINAV ÖNCELİĞİ: ORTA - Veri = bilgi; sinyal = fiziksel temsil. Analog: sürekli; sayısal: tanımlı seviyeler. - Dört dönüşüm: telefon (A→A), modem (S→A), codec (A→S), sayısal alıcı-verici (S→S). Modülasyon: genlik, frekans ya da faz. - **Bit hızı = sembol hızı × $\log_2(M)$.** 0.02 ms/bit → 50 kbps. Çok seviye = hızlı ama gürültüye açık.

⚡ **HIZLI TEKRAR**

3. Kanal bozulmaları SINAV ÖNCELİĞİ: ORTA - **Zayıflama:** enerji azalır (yükselteç/tekrarlayıcı). **Bozulma:** şekil değişir. **Gürültü:** istenmeyen enerji eklenir. - Yüksek SNR = doğru karar; düşük SNR = hata artar. - Bakır (ucuz, kısa), fiber (ışık, yüksek kapasite, girişime dayanıklı), kablosuz (hareketlilik, paylaşılan ortam). **Ortam sinyal türünü tek başına belirlemez.** - Slayt 41: 500 m (10Base5), 185 m (10Base2), Ethernet 100 m, fiber 2000 m (100BaseFX).

⚡ **HIZLI TEKRAR**

4. Hat kodlama SINAV ÖNCELİĞİ: YÜKSEK - İyi kod: zamanlama, düşük bant genişliği, uzun dizilerde senkronizasyon, düşük DC, hata ipuçları. - Saat %0.1 hızlı: 1 kbps'te 1 bit/s, 1 Mbps'te 1000 bit/s fazla. - **Unipolar NRZ** (1 = +V, 0 = 0; DC güçlü, güç ½V²). **NRZ-L** (seviye = bit). **NRZ-I** (1 = geçiş; polarite ters olsa da çalışır). **RZ** (1 = +V sonra 0; 0 = −V sonra 0; üç seviye). **Manchester** (orta geçiş her zaman; 0 yüksek-alçak, 1 alçak-yüksek, D1 s. 53). **Fark Manchester** (orta geçiş her zaman; sınırda geçiş = 0, yok = 1). - RZ ve Manchester: $r = 1/2$, $S_{ave} = N$. Manchester kendinden saatli, DC'siz, geniş bantlı. - Blok kodlama: fazladan bit. Karıştırma: uzun dizileri kırar.

⚡ **HIZLI TEKRAR**

5. PCM ve delta modülasyonu **SINAV ÖNCELİĞİ: YÜKSEK** - PCM: **örnekle → nicemle → kodla.** Nyquist: $f_s > 2f_{max}$; yetersiz örnekleme = örtüşme. - n bit = 2^n seviye; nicemleme hatası ≤ yarım şerit. Slayt 61 ilk hata −0.28 (slaytta −0.38 yazıyor). - **Bit hızı = örnek/s × bit/örnek × kanal.** 8000 × 8 = 64 kbps. - **DM:** 1 bit/örnek, yön bilgisi. Hızlı giriş → **eğim aşırı yüklenmesi**; yavaş giriş + büyük basamak → **granüler gürültü**. - PCM: çok bit/örnek, ana hata nicemleme gürültüsü. DM: 1 bit/örnek, basit kodlayıcı.

⚡ **HIZLI TEKRAR**

6. İletim modları **SINAV ÖNCELİĞİ: ORTA** - Paralel: çok hat, bir adımda çok bit, kısa mesafe. Seri: tek hat, sırayla, modern yüksek hızlı bağlantılarda yaygın. - Seri modlar: **asenkron** (start/stop, basitlik), **senkron** (çerçeve + saat kurtarma, verimlilik), **izokron** (düzenli teslim, gecikme/jitter sınırı).

Ezberlenecek tek sayfa

Ne?	Değer
4 bit 2C aralığı	−8 … +7
8 bit 2C aralığı	−128 … +127
$-x$ (2C)	$2^n - x$ = ters çevir + 1
Bit hızı	sembol hızı × $\log_2(M)$
Nyquist	$f_s > 2f_{max}$
PCM bit hızı	örnek/s × bit/örnek × kanal
Telefon sesi	8000 × 8 = 64 kbps
Saat kayması %0.1	1 kbps → 1 bit/s; 1 Mbps → 1000 bit/s
RZ / Manchester	$r = 1/2$, $S_{ave} = N$
Nicemleme hatası	≤ yarım şerit

On kritik tuzak

1. Aynı bit örüntüsü gösterime göre farklı sayı: **önce gösterimi oku.**
2. 2C aralığı asimetrik: −8…+7, **−7…+7 değil.**
3. Taşma = işaret kuralı, **dış elde değil.**
4. 8 seviye 3 bit taşır, 8 bit değil.
5. Ortam sinyal türünü belirlemez.
6. NRZ-I'da **1 geçiş** yaratır; fark Manchester'da sınırda geçiş **0**'dır.
7. Manchester'ın yönü kurala bağlı; **kuralı yaz** (0: yüksek-alçak, 1: alçak-yüksek).
8. RZ üç seviyelidir.
9. Nyquist **2 katından büyük**; yetersizse örtüşme.
10. Granüler gürültü ≠ eğim aşırı yüklenmesi: biri yavaş girişte büyük basamaktan, diğeri hızlı girişte küçük basamaktan.

Sınav öncesi kontrol listesi

Her satırı bakmadan yapabiliyorsan işaretle. Yapamadığının bölümüne dön.

- ☐ Bir sayının işaret-büyüklük, 1'e tümleyen ve 2'ye tümleyen gösterimini yazabiliyorum (Bölüm 1)
- ☐ Verilen 8 bitlik örüntüyü 2'ye tümleyen olarak okuyabiliyorum (Bölüm 1)
- ☐ $A - B$ işlemini toplamaya çevirip taşmayı işaret kuralıyla yorumlayabiliyorum (Bölüm 1)
- ☐ Veri ile sinyalin farkını ve dört dönüşümü sayabiliyorum (Bölüm 2)
- ☐ Bit hızı = sembol hızı × $\log_2 M$ hesabını yapabiliyorum (Bölüm 2)
- ☐ Zayıflama, bozulma, gürültüyü şekil ve tanımla ayırt edebiliyorum (Bölüm 3)
- ☐ NRZ-L, NRZ-I, RZ, Manchester ve fark Manchester dalga biçimlerini kuralıyla çizebiliyorum (Bölüm 4)
- ☐ Saat kaymasının neden sorun olduğunu bir hesapla anlatabiliyorum (Bölüm 4)
- ☐ Nyquist kuralını uygulayıp örtüşmeyi açıklayabiliyorum (Bölüm 5)
- ☐ Bir örneği nicemleyip kod kelimesini ve hatayı bulabiliyorum; PCM bit hızını hesaplayabiliyorum (Bölüm 5)
- ☐ Eğim aşırı yüklenmesi ile granüler gürültüyü ayırt edebiliyorum (Bölüm 5)
- ☐ Paralel/seri ve asenkron/senkron/izokron farklarını sayabiliyorum (Bölüm 6)
- ☐ Deneme sınavını süre tutarak çözdüm ve cevap anahtarıyla karşılaştırdım

Deneme Sınavı

Süre: 90 dakika. **Toplam:** 100 puan (çoktan seçmeli 40, klasik 60). Hesap makinesi kullanabilirsin ama bitleri elle yap. Sorular İngilizce, altlarında Türkçe çevirileri var (sınav dili bilinmediği için ikisini birlikte verdim). Her sorunun altında zorluk ve kaynak slayt yazıyor; gerçek sınavda bunlar olmayacak. Önce kendin çöz, sonra "Cevap Anahtarı"na bak.

Kabuller (soruda belirtilmediyse): NRZ-L'de 1 = yüksek (H), 0 = alçak (L). Manchester'da yön, sorunun içinde verilmiştir.

Bölüm A çoktan seçmeli, 20 soru, 40 puan

A1. What is the decimal value of `1111 1010` in 8-bit two's complement?

8 bitlik 2'ye tümleyende `1111 1010` değerinin onluk karşılığı nedir?

(A) −5 (B) −6 (C) 250 (D) −122

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 14–17

A2. What is the range of a 6-bit two's-complement integer?

6 bitlik 2'ye tümleyen tam sayının aralığı nedir?

(A) −31 ... +31 (B) −32 ... +32 (C) −32 ... +31 (D) −63 ... +63

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 19

A3. Which representation has exactly one zero?

Hangi gösterimde tam olarak bir sıfır vardır?

(A) Signed magnitude (B) One's complement (C) Signed magnitude and one's complement (D) Two's complement

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 18

A4. In 8-bit two's complement, 90 + 50 is computed. Which statement is correct?

8 bitlik 2'ye tümleyende 90 + 50 hesaplanıyor. Hangi ifade doğrudur?

(A) Overflow occurs; the bit pattern reads as −116. (B) No overflow; the result is 140. (C) Overflow occurs because the carry-out is 1. (D) No overflow; the result is −116.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 23

A5. In 4-bit two's complement, 5 − 7 is computed as 5 + (−7). What is the outcome?

4 bitlik 2'ye tümleyende 5 − 7 işlemi 5 + (−7) olarak yapılıyor. Sonuç nedir?

(A) Overflow; the result is invalid. (B) No overflow; the result is `1110` = −2. (C) No overflow; the result is `0010` = 2. (D) Overflow; `1110` is read as −2.

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 20–23

A6. A modem converts which kind of data into which kind of signal?

Bir modem hangi tür veriyi hangi tür sinyale çevirir?

(A) Analog data to digital signal (B) Digital data to digital signal (C) Digital data to analog signal (D) Analog data to analog signal

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 30–32

A7. A signal uses 8 levels and 2500 symbols per second. What is the bit rate?

Bir sinyal 8 seviye kullanıyor ve saniyede 2500 sembol gönderiyor. Bit hızı nedir?

(A) 2500 bit/s (B) 10 000 bit/s (C) 20 000 bit/s (D) 7500 bit/s

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 33–34

A8. System X: 4 levels, 3000 symbols/s. System Y: 2 levels, 5000 symbols/s. Which statement is correct?

Sistem X: 4 seviye, 3000 sembol/s. Sistem Y: 2 seviye, 5000 sembol/s. Hangisi doğrudur?

(A) X has the higher bit rate (6000 vs 5000 bit/s). (B) Y has the higher bit rate (5000 vs 3000 bit/s). (C) Both have the same bit rate. (D) X has 3000 bit/s, Y has 10 000 bit/s.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 34

A9. A signal's shape changes because its frequency components travel at different speeds. This impairment is called:

Bir sinyalin şekli, frekans bileşenleri farklı hızlarda ilerlediği için değişiyor. Bu bozulmanın adı:

(A) Attenuation (B) Distortion (C) Noise (D) Aliasing

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 37

A10. NRZ-I encodes the bits `1 1 0 1` (1 = transition at the start of the bit; the level before the first bit is low). Which level sequence results?

NRZ-I ile `1 1 0 1` bitleri kodlanıyor (1 = bit başında geçiş; ilk bitten önceki seviye alçak). Hangi seviye dizisi çıkar?

(A) L H H L (B) H H L L (C) H L L H (D) L L H H

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 51

A11. Which is a disadvantage of Manchester coding?

Manchester kodunun hangisi bir dezavantajıdır?

(A) It has a DC component. (B) It cannot provide synchronization. (C) Long runs of zeros make the clock disappear. (D) It needs about twice the bandwidth of NRZ.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 53–54

A12. A receiver's clock runs 0.1% faster than the sender's, at 2 Mbps. By how many bits per second do the two counts drift apart?

Alıcının saati göndericiden %0,1 hızlı, hat 2 Mbps. İki taraftaki bit sayımı saniyede kaç bit kayar?

(A) 2000 (B) 200 (C) 20 000 (D) 2

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 47–48 (+ ek hesap)

A13. How many voltage levels does polar RZ use?

Kutupsal (polar) RZ kaç gerilim seviyesi kullanır?

(A) 2 (B) 3 (C) 4 (D) 1

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 52

A14. A signal's highest frequency is 10 kHz. Which sampling rate satisfies the Nyquist idea?

Bir sinyalin en yüksek frekansı 10 kHz. Hangi örnekleme hızı Nyquist fikrini sağlar?

(A) 10 kHz (B) 15 kHz (C) 25 kHz (D) 5 kHz

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 60

A15. Audio is sampled at 16 kHz with 12 bits per sample, 2 channels. What is the PCM bit rate?

Ses 16 kHz'de, örnek başına 12 bitle, 2 kanal olarak örnekleniyor. PCM bit hızı nedir?

(A) 192 kbps (B) 768 kbps (C) 28 kbps (D) 384 kbps

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 62

A16. A 3-bit quantizer has centers ±0.5, ±1.5, ±2.5, ±3.5 (codes 0–7 from the bottom). The sample 1.1 gives:

3 bitlik nicemleyicinin merkezleri ±0.5, ±1.5, ±2.5, ±3.5 (kodlar alttan 0–7). 1.1 örneği için sonuç:

(A) 1.5; code `101`; error +0.4 (B) 1.5; code `100`; error −0.4 (C) 0.5; code `100`; error −0.6 (D) 1.5; code `101`; error −0.4

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 61

A17. In delta modulation the input is almost constant, but the output bits alternate 1 0 1 0 ... What is the problem and the fix?

Delta modülasyonunda giriş neredeyse sabit, ama çıkış bitleri 1 0 1 0 ... diye dönüyor. Sorun ve çözüm nedir?

(A) Slope overload; increase the step size (B) Granular noise; decrease the step size (C) Aliasing; raise the sampling rate (D) Slope overload; decrease the step size

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 63–64

A18. Which statement about delta modulation (DM) compared with PCM is correct?

Delta modülasyonunun (DM) PCM ile karşılaştırması hakkında hangisi doğrudur?

(A) DM sends several bits for every sample. (B) PCM sends exactly one bit per sample. (C) DM sends one bit per sample (up/down information). (D) DM has no quantization problems at all.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 65

A19. Which transmission sends the bits of a byte at the same time over separate wires?

Hangi iletim, bir baytın bitlerini ayrı teller üzerinden aynı anda gönderir?

(A) Serial (B) Asynchronous (C) Isochronous (D) Parallel

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 68

A20. Asynchronous frame: 1 start bit + 8 data bits + 1 stop bit. What fraction of the transmitted bits is data?

Asenkron çerçeve: 1 start biti + 8 veri biti + 1 stop biti. Gönderilen bitlerin ne kadarı veridir?

(A) 80% (B) 100% (C) 90% (D) 70%

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 70 (+ ek hesap)

Bölüm B klasik sorular, 60 puan

B1. (8 puan) (a) Write −23 in 8-bit signed magnitude, one's complement and two's complement. (6 pts) (b) The 8-bit pattern `1100 1110` is in two's complement. What is its decimal value? (2 pts)

(a) −23 sayısını 8 bitlik işaret-büyüklük, 1'e tümleyen ve 2'ye tümleyen olarak yaz. (b) 2'ye tümleyendeki 8 bitlik `1100 1110` örüntüsünün onluk değeri nedir?

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 7–17

B2. (8 puan) In 8-bit two's complement: (a) compute 45 − 23 as an addition and state whether overflow occurs. (4 pts) (b) compute (−100) + (−60), give the resulting bit pattern, its decimal reading, and decide whether overflow occurs. (4 pts)

8 bitlik 2'ye tümleyende: (a) 45 − 23 işlemini toplama olarak yap, taşma olup olmadığını belirt. (b) (−100) + (−60) işlemini yap; sonuç bit örüntüsünü, onluk yorumunu ve taşma olup olmadığını yaz.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 20–23

B3. (6 puan) A student writes: "In 8-bit two's complement, 127 + 1 = 128, because there is no carry out of the leftmost bit, there is no overflow." Find and correct all errors in this statement.

Bir öğrenci şöyle yazıyor: "8 bitlik 2'ye tümleyende 127 + 1 = 128'dir; en soldaki bitten elde çıkmadığı için taşma yoktur." Bu ifadedeki bütün hataları bul ve düzelt.

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 23

B4. (4 puan) (a) A signal uses 16 levels at 2500 symbols per second. What is the bit rate? (2 pts) (b) A modem must reach 9600 bit/s at 2400 symbols per second. How many levels are needed? (2 pts)

(a) Bir sinyal 16 seviye ve saniyede 2500 sembol kullanıyor. Bit hızı nedir? (b) Bir modem 2400 sembol/s ile 9600 bit/s'ye ulaşmalı. Kaç seviye gerekir?

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 33–34

B5. (8 puan) Encode the bit sequence `1001101` using (a) NRZ-L (3 pts), (b) NRZ-I with the level before the first bit low (3 pts), (c) Manchester with 0 = high-to-low and 1 = low-to-high (2 pts). Write H/L sequences (or draw) and state your conventions.

`1001101` bit dizisini (a) NRZ-L, (b) NRZ-I (ilk bitten önceki seviye alçak), (c) Manchester (0 = yüksekten alçağa, 1 = alçaktan yükseğe) ile kodla. H/L dizisi yaz ya da çiz; kabullerini belirt.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 50–53

B6. (6 puan) (a) Encode `0110` with differential Manchester. Assume the level at the end of the previous bit is high; use the rule "transition at the start of the bit = 0, no transition = 1, always a transition in the middle". (3 pts) (b) A receiver's clock is 0.05% too fast on a 1 Mbps link. How many extra bits per second does it count, and what is the accumulated drift in bit periods over a run of 1000 identical bits in a code with no transitions? Why is this a problem? (3 pts)

(a) `0110` bitlerini fark Manchester ile kodla. Önceki bitin bitiş seviyesi yüksek kabul et; kural: bit başında geçiş = 0, geçiş yok = 1, bitin ortasında her zaman geçiş. (b) 1 Mbps bir hatta alıcı saati

%0,05 hızlı. Saniyede kaç bit fazla sayar? Geçişsiz bir kodda 1000 aynı bitlik dizide biriken kayma kaç bit süresidir? Bu neden sorundur?

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 47–48, 53

B7. (8 puan) A 3-bit quantizer has 8 levels with centers ±0.5, ±1.5, ±2.5, ±3.5 (codes 0 to 7 from the bottom). (a) For the normalized samples 2.7, −0.3 and −3.6, find the quantized value, the error (quantized − actual) and the code word. (6 pts) (b) A voice signal with maximum frequency 3.4 kHz is sampled at 8000 samples/s with 8 bits per sample. Compute the PCM bit rate and explain why 8000 is a valid choice. (2 pts)

3 bitlik bir nicemleyicinin merkezleri ±0.5, ±1.5, ±2.5, ±3.5 olan 8 seviyesi var (kodlar alttan 0–7). (a) Normalize örnekler 2.7, −0.3 ve −3.6 için nicemlenmiş değeri, hatayı (nicemlenmiş − gerçek) ve kod kelimesini bul. (b) En yüksek frekansı 3.4 kHz olan bir ses sinyali 8000 örnek/s ve örnek başına 8 bitle örnekleniyor. PCM bit hızını hesapla; 8000'in neden geçerli olduğunu açıkla.

Zorluk: Orta · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 60–62

B8. (4 puan) (a) State one difference between PCM and delta modulation. (2 pts) (b) State two differences between parallel and serial transmission. (2 pts)

(a) PCM ile delta modülasyonu arasında bir fark yaz. (b) Paralel ve seri iletim arasında iki fark yaz.

Zorluk: Kolay · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 65, 68–69

B9. (8 puan) A voice signal has a highest frequency of 4 kHz. (a) What is the minimum sampling rate by the Nyquist idea? If 8000 samples/s and 8 bits per sample are used, what is the PCM bit rate? (3 pts) (b) These bits are sent as 8-bit characters in asynchronous serial mode with 1 start bit and 1 stop bit per character. What is the bit rate on the line, and what percentage is useful data? (3 pts) (c) If the line then used Manchester coding, why would it need more bandwidth than NRZ? (2 pts)

Bir ses sinyalinin en yüksek frekansı 4 kHz. (a) Nyquist fikrine göre en az kaç örnek/s gerekir? 8000 örnek/s ve örnek başına 8 bit kullanılırsa PCM bit hızı nedir? (b) Bu bitler asenkron seri iletimde, her karakter için 1 start ve 1 stop biti ile 8 bitlik karakterler hâlinde gönderiliyor. Hattaki bit hızı nedir, yüzde kaçı yararlı veridir? (c) Hat Manchester kodlamaya geçseydi neden NRZ'den fazla bant genişliği gerekirdi?

Zorluk: Zor · ❖ Kaynak: D1 (CMP103_05-2.pdf), s. 53, 60–62, 70

Sınav İngilizcesi kalıpları

Sınav sorularında sık geçen fiiller ve ne istedikleri. Slaytlardaki etkinlik soruları da bu kalıpları kullanıyor ("What are...", "Draw..."). Tablodaki yorumlar genel sınav alışkanlığıdır, slayt içeriği değildir.

İngilizce	Ne yapmalısın?
Determine / Compute / Calculate	Sayısal sonucu adım adım hesapla, birimini yaz
Draw / Sketch	Şekil çiz; eksenleri, seviyeleri (H/L) ve bit sınırlarını işaretle
Explain / Describe	Kısa ve net cümlelerle nedenini anlat
Explain the difference between...	İki kavramı aynı ölçütte yan yana karşılaştır
State / List	Gerekçe istenmez; maddeler hâlinde sadece yaz
Find the error(s)	Hatalı ifadeyi bul, doğrusunu yaz, nedenini ver

İngilizce	Ne yapmalısın?
Show your work	Ara adımlar puan getirir; yazmazsan puan kaybedersin
Justify	Verdiğin cevabın neden doğru olduğunu kanıtla
Assume...	Soruda verilen kabulü aynen kullan, kabulünü yazarak başla

Deneme Sınavı Cevap Anahtarı

Önce kendi çözümünü yap, sonra karşılaştır. Bütün sayısal cevaplar python ile hesaplanarak doğrulandı, ayrıca sınavı cevap anahtarını görmeden çözen bağımsız bir çözücü 29 sorunun hepsinde aynı cevapları buldu. ✔ python3 + bağımsız çözüm

Bölüm A çoktan seçmeli cevapları

Cevap şeridi: A1 B · A2 C · A3 D · A4 A · A5 B · A6 C · A7 D · A8 A · A9 B · A10 C · A11 D · A12 A · A13 B · A14 C · A15 D · A16 A · A17 B · A18 C · A19 D · A20 A

Soru	Doğru	Neden doğru, yanlış şıklar hangi yanılgıdan
A1	**B** (−6)	İlk bit 1 → ters çevir `0000 0101`, +1 → `0000 0110` = 6 → −6. A: "sadece ters çevir" (1'e tümleyen) yanılgısı. C: işaretsiz okuma. D: son 7 biti büyüklük sanmak.
A2	**C**	$-2^5 \ldots 2^5 - 1$. A: işaret-büyüklük aralığı. B: +32 sığmaz. D: 6 bit değil 7 bit mantığı.
A3	**D**	İşaret-büyüklük ve 1'e tümleyende ±0 var; 2'ye tümleyende tek sıfır. A ve C: bu gösterimlerde de tek sıfır olduğunu söylüyor.
A4	**A**	$90 + 50 = 140 > 127$. `0101 1010 + 0011 0010 = 1000 1100` = −116. İki pozitif → negatif → taşma. B: 140 sığmaz. C: dış elde 0, taşma elde ile ölçülmez. D: sonuç yanlış olduğu hâlde "taşma yok" demek.
A5	**B**	$5 + (-7)$: `0101 + 1001 = 1110` = −2. Farklı işaretli sayılar taşma yapamaz. A ve D: "negatif sonuç = taşma" yanılgısı. C: sonucu pozitif okumak.
A6	**C**	Modem: sayısal veri → analog sinyal (modülasyon). A ve D: yönü ters ya da analog-analog. B: bu hat kodlamadır, modem değil.
A7	**D**	$2500 \times \log_2 8 = 7500$. A: seviye sayısını hiç hesaba katmamak. B: $\log_2 8$ yerine 4 kullanmak. C: $\log_2 8$ yerine 8 kullanmak.
A8	**A**	X: $3000 \times 2 = 6000$; Y: $5000 \times 1 = 5000$. Seviyesi çok olan her zaman hızlı değil, sembol hızı da hesaba girer. B: sadece sembol hızına bakmak. C: iki sistemin eşit olduğunu varsaymak. D: Y için seviye sayısını bit sanıp yanlış çarpmak.
A9	**B**	Şekil değişimi = bozulma (distortion). A: genlik küçülmesi. C: üste eklenen enerji. D: örtüşme örneklemeyle ilgili, kanalla değil.
A10	**C**	Alçaktan başla: 1 → H, 1 → L, 0 → L, 1 → H = `H L L H`. A: başlangıcı yüksek almak. B ve D: 1'i "seviye = 1" gibi (NRZ-L) okumak.
A11	**D**	Her bitte iki seviye değişimi gerekir, bant genişliği yaklaşık 2 katıdır. A–C: Manchester'ın tam tersi özellikleri; DC yoktur, senkronizasyon verir, uzun 0 dizisinde bile geçiş vardır.
A12	**A**	$2\,000\,000 \times 0{,}001 = 2000$ bit/s. B: %0,01 yanılgısı. C: %1. D: 2 Mbps'i 2 ile karıştırmak.
A13	**B**	Polar RZ: +, 0, −. A: iki seviye NRZ'dir. C ve D: slayt dışı.
A14	**C**	$f_s > 2 \times 10 = 20$ kHz. Sadece 25 kHz sağlar. A ve D: eşit/küçük. B: 15 < 20, örtüşme olur.
A15	**D**	$16\,000 \times 12 \times 2 = 384\,000$ bit/s. A: kanal sayısını unutmak (192). B: ×4. C: toplamak (16+12).

Soru	Doğru	Neden doğru, yanlış şıklar hangi yanılgıdan
A16	**A**	1.1 → şerit 1...2, merkez 1.5, kod 5 = `101`, hata = 1.5 − 1.1 = +0.4. B: kod yanlış (`100` = 0.5). C: yanlış şerit. D: hata işareti ters (hata = nicemlenmiş − gerçek).
A17	**B**	Giriş yavaş/sabit, çıkış 1-0-1-0 → granüler gürültü; basamak küçültülür. A ve D: eğim aşırı yüklenmesi hızlı değişen girişte olur. C: örtüşme başka olay.
A18	**C**	DM örnek başına 1 bit (yukarı/aşağı) gönderir. A: PCM'in özelliği. B: PCM'i DM sanmak. D: DM'nin kendi sorunları (eğim aşırı yüklenmesi, granüler gürültü) vardır.
A19	**D**	Paralel iletim: birçok hat, aynı anda. A: tek hat. B ve C: seri iletimin zamanlama düzenleri.
A20	**A**	$8/10 = \%80$. B: başlık bitlerini unutmak. C: 9/10 yazmak; çerçevede 2 ek bit var, 1 değil. D: kaynaksız tahmin.

↺ HATIRLATMA — Yanlışlarından öğren

Şıklar sık yapılan yanılgılara göre kuruldu. Yanlış yaptığın soruda şıkkın neden çekici geldiğini bul; aynı tuzağa ikinci kez düşmezsin.

6

Bölüm B klasik soru cevapları

B1 (8 puan). (a) 6 puan, (b) 2 puan.

(a) $23 = 0001\,0111$.

- İşaret-büyüklük: işaret biti 1 + büyüklük `0010111` → **`1001 0111`**. (2 puan)
- 1'e tümleyen: her bit ters → **`1110 1000`**. (2 puan)
- 2'ye tümleyen: `1110 1000` + 1 → **`1110 1001`**. (2 puan)

(b) `1100 1110`: ilk bit 1 → negatif. Ters çevir: `0011 0001`, +1: `0011 0010` = 50. Sonuç **−50**. (2 puan: ters çevirme 1, +1 ve işaret 1) ✔ python3 ile doğrulandı

Kısmi puan: Yöntem doğru, son +1 unutulmuşsa (−49) 1 puan.

B2 (8 puan). (a) 4, (b) 4.

(a) $45 - 23 = 45 + (-23)$. $45 = 0010\,1101$, $-23 = 1110\,1001$. Topla: `0010 1101 + 1110 1001 = 1 0001 0110`; dış elde atılır → `0001 0110` $= 22$. (3 puan) Taşma: toplananlar farklı işaretli → **taşma yok**. (1 puan)

(b) $-100 = 1001\,1100$ ve $-60 = 1100\,0100$ (2 puan: iki dönüşüm). `1001 1100 + 1100 0100 = 1 0110 0000`; dış elde atılır → **`0110 0000`** $= 96$ (1 puan). **Taşma var:** iki negatif toplandı, sonuç pozitif. Gerçek sonuç $-160 < -128$; $-160 + 256 = 96$. (1 puan; kural: aynı işaretli iki sayının toplamı ters işaret verirse taşma. Dış elde taşmanın kanıtı değildir.) ✔ python3 ile doğrulandı

Alternatif çözüm (a): $45 - 23$ için 23'ü çıkarmak yerine -23'ü doğrudan toplamak zaten asıl yöntemdir; ayrıca $22 = 0001\,0110$ sonucu sayı doğrulamasıyla da kontrol edilebilir.

B3 (6 puan). İfadede üç hata var (her biri 2 puan):

1. $127 + 1$ sonucu 8 bit 2'ye tümleyende **+128 değil**: `0111 1111 + 0000 0001 = 1000 0000`, bu örüntü **−128** okunur. (+128 aralık dışı; aralık −128 ... +127.)
2. **Taşma vardır:** iki pozitif sayı toplandı, sonuç negatif çıktı.

3. "Elde çıkmadığı için taşma yok" yanlış. Taşma, **işaret kuralıyla** anlaşılır (aynı işaretli iki sayı → ters işaretli sonuç); dış elde tek başına ölçüt değildir. Burada dış elde zaten 0'dır ve taşma yine de vardır. ✔ python3 ile doğrulandı

B4 (4 puan).

(a) $2500 \times \log_2 16 = 2500 \times 4 =$ **10 000 bit/s**. (2 puan)

(b) $9600/2400 = 4$ bit/sembol $\Rightarrow M = 2^4 =$ **16 seviye**. (2 puan) ✔ python3 ile doğrulandı

B5 (8 puan). Bit dizisi `1 0 0 1 1 0 1`.

(a) **NRZ-L** (1 = H, 0 = L): `H L L H H L H`. (3 puan)

(b) **NRZ-I** (ilk bitten önce L; 1 = bit başında geçiş, 0 = geçiş yok): `H H H L H H L`. (3 puan) Adımlar: 1 → H; 0 → H; 0 → H; 1 → L; 1 → H; 0 → H; 1 → L.

(c) **Manchester** (0: H→L, 1: L→H), yarım-bit seviyeleri: `LH, HL, HL, LH, LH, HL, LH`. (2 puan) ✔ python3 ile doğrulandı

Kabullerin yazılması puan getirir; kabulü belirtmeden ters sonuç yazarsan yarım puan kırılabilir.

B6 (6 puan).

(a) (3 puan) Önceki bitin sonu H. Her bitte ortada geçiş var; bit başında geçiş = 0, geçiş yok = 1.

- Bit 0: başta H→L geçiş, ortada L→H: `LH` (sonu H).
- Bit 1: başta geçiş yok (H), ortada H→L: `HL` (sonu L).
- Bit 1: başta geçiş yok (L), ortada L→H: `LH` (sonu H).
- Bit 0: başta H→L geçiş, ortada L→H: `LH`.

Sonuç: **`LH, HL, LH, LH`**.

(b) (3 puan) $10^6 \times 0{,}0005 =$ **500 bit/s** fazla sayar (1 puan). 1000 bit $\times 0{,}0005 =$ **0,5 bit süresi** kayma (1 puan). Örnekleme noktası bit ortasından bit sınırına kayar, bitler yanlış okunur; geçişsiz uzun dizide saati düzeltecek bir kenar yoktur (1 puan). (Aynı hızla 2000 bit sonra tam bir bit kayar.) ✔ python3 ile doğrulandı

B7 (8 puan).

(a) (6 puan: örnek başına 2)

Örnek	Şerit	Nicemlenmiş	Hata (nicemlenmiş − gerçek)	Kod	Kelime
2.7	2 … 3	2.5	−0.2	6	`110`
−0.3	−1 … 0	−0.5	−0.2	3	`011`
−3.6	−4 … −3	−3.5	+0.1	0	`000`

✔ python3 ile doğrulandı

(b) (2 puan) Bit hızı $= 8000 \times 8 = 64\,000$ bit/s $= 64$ kbps (1 puan). Geçerli çünkü $8000 > 2 \times 3400 = 6800$ (1 puan).

B8 (4 puan). Her doğru fark 1 puan; örnek cevaplar:

(a) PCM örnek başına birkaç bit (nicemlenmiş değer) gönderir; delta modülasyonu örnek başına **1 bit** (yukarı/aşağı) gönderir. (Ek olarak: DM'de eğim aşırı yüklenmesi ve granüler gürültü, PCM'de nicemleme hatası ana sorundur.) (2 puan)

(b) Paralel: çok hat, bitler aynı anda, kısa mesafe, hatlar arası kayma (skew) sorunu. Seri: tek hat, bitler sırayla, uzun mesafede ucuz, alıcı saatini bulmak gerekir. (2 puan: iki fark)

B9 (8 puan).

(a) (3 puan) En az 2×4 kHz $= 8$ kHz (slayt kuralı $f_s > 2f_{max}$ olduğu için 8000 sınırdadır; pratikte 8000 kullanılır). PCM bit hızı $8000 \times 8 =$ **64 kbps**.

(b) (3 puan) Her 8 bitlik karakter hatta 10 bit olur: $64\,000 \times 10/8 =$ **80 kbps**. Yararlı veri $8/10 =$ **%80**.

(c) (2 puan) Manchester'da her bitte (ortada) en az bir geçiş vardır; seviye saniyede NRZ'nin yaklaşık iki katı kez değişebilir, bu da yaklaşık **2 kat bant genişliği** demektir. ✔ python3 ile doğrulandı

Puan tablosu

Puan	Değerlendirme
85–100	Çok iyi; sadece küçük tuzakları tekrar et
70–84	İyi; yanlış yaptığın konunun bölümünü ve "Sık yapılan hatalar" kutusunu oku
50–69	Orta; 1., 4. ve 5. bölümleri yeniden çöz, çizim sorularını kâğıtta tekrarla
0–49	Bölümleri sırayla yeniden oku; her örneği kendin çöz

Hangi soru hangi bölümü sınıyor?

Bölüm	Sorular
1. İşaretli sayılar	A1–A5, B1, B2, B3
2. Veri ve sinyal	A6–A8, B4
3. Kanal bozulmaları	A9
4. Hat kodlama	A10–A13, B5, B6
5. PCM ve delta	A14–A18, B7, B8
6. İletim modları	A19, A20, B8, B9

Terim Sözlüğü

İngilizce slaytlardaki terimler, İngilizce adlarına göre alfabetik. Son sütun, terimin ilk anlatıldığı bölüm.

İngilizce	Türkçe	Kısa tanım	Bölüm
aliasing	örtüşme	Yetersiz örneklemede yüksek frekansın düşük görünmesi	5
amplifier / repeater	yükselteç / tekrarlayıcı	Zayıflayan sinyali güçlendirir / yeniden üretir	3
analog / digital signal	analog / sayısal sinyal	Sürekli / tanımlı seviyeli	2
asynchronous	asenkron	Karakterler bağımsız zamanlarda; start/stop bitleri	6
attenuation	zayıflama	Mesafeyle sinyal enerjisinin azalması	3
bit rate	bit hızı	Saniyedeki bit sayısı = sembol hızı × $\log_2(M)$	2
block coding	blok kodlama	Veri bloklarına fazladan bit ekleme	4
carrier	taşıyıcı	Veriyi taşımak için kullanılan düzenli dalga	2
clock drift	saat kayması	Alıcı ve gönderici saatlerinin farklı hızda olması	4
coaxial cable	koaksiyel kablo	Örgü korumalı bakır kablo	3
codec	kodlayıcı-çözücü (codec)	Analog sinyali bitlere, bitleri analog sinyale çevirir	2
crosstalk	çapraz karışma	Komşu hatların birbirine sinyal sızdırması	3
data	veri	Taşınacak ya da işlenecek bilgi	2
DC component	DC bileşeni	Sinyalin ortalama (sabit) değeri	4

İngilizce	Türkçe	Kısa tanım	Bölüm
delta modulation (DM)	delta modülasyonu	Her örnekte yukarı/aşağı yön bilgisi	5
differential Manchester	fark Manchester	Orta geçiş her zaman; sınırdaki geçiş 0'ı gösterir	4
distortion	bozulma	Sinyalin şeklinin değişmesi	3
encoder / decoder	kodlayıcı / çözücü	Bitlerden sinyale / sinyalden bitlere	4
end-around carry	çevrimsel elde	1'e tümleyende taşan eldenin sonuca geri eklenmesi	1
granular noise	granüler gürültü	Yavaş girişte büyük basamak yüzünden zıplama	5
isochronous	izokron	Düzenli aralıklarla teslim; gecikme/jitter sınırı	6
jitter	jitter (titreşim)	Teslim aralığındaki oynama	6
line coding	hat kodlama	Bitleri sinyal seviyelerine ve geçişlere eşleme kuralları	4
magnitude	büyüklük	Sayının mutlak değeri	1
Manchester	Manchester kodu	Her bitin ortasında geçiş; yön biti verir	4
modulation	modülasyon	Taşıyıcının genlik, frekans ya da fazını veriye göre değiştirmek	2
noise	gürültü	Sinyale eklenen istenmeyen enerji	3
NRZ / NRZ-L / NRZ-I	sıfıra dönmeyen kodlar	Seviye bitti temsil eder / 1'de geçiş	4
Nyquist	Nyquist kuralı	$f_s > 2f_{max}$	5
one's complement	1'e tümleyen	Negatif sayı = pozitifin bütün bitlerinin tersi	1
optical fiber	fiber optik	Işık darbeleriyle iletim	3
overflow	taşma	Sonucun bit genişliğine sığmaması; aynı işaretli toplamın ters işaret vermesi	1
PAM	darbe genlik modülasyonu	Örnekleme sonrası darbe dizisi	5
parallel transmission	paralel iletim	Çok bit, çok hat, bir adımda	6

İngilizce	Türkçe	Kısa tanım	Bölüm
PCM	darbe kod modülasyonu	Örnekle, nicemle, kodla	5
protocol	protokol	Biçim, sıra, zamanlama ve hata davranışı kuralları	2
quantization	nicemleme	Örneği en yakın izinli seviyeye yuvarlamak	5
quantization error	nicemleme hatası	Nicemlenmiş değer – gerçek değer	5
RZ	sıfıra dönen kod	Her bitte sıfıra dönüş; üç seviye	4
sampling	örnekleme	Belli anlarda genliği almak	5
scrambling	karıştırma	Uzun aynı-bit dizilerini kırma	4
self-clocking	kendinden saatli	Alıcı saati sinyalden çıkarabilir	4
serial transmission	seri iletim	Bitler tek hattan sırayla	6
sign bit	işaret biti	0 = pozitif, 1 = negatif	1
sign extension	işaret genişletme	Bit genişliğini artırırken sola işaret bitini kopyalamak	1
signal	sinyal	Veriyi fiziksel ortamda temsil eden enerji değişimi	2
signed magnitude	işaret-büyüklük	En soldaki bit işaret, kalanı büyüklük	1
skew	kayma	Paralel hatlarda bitlerin farklı zamanda varması	6
slope overload	eğim aşırı yüklenmesi	Giriş merdivenden hızlı değişir	5
SNR	sinyal-gürültü oranı	Sinyal gücünün gürültü gücüne oranı	3
symbol	sembol	Sinyalin sabit kaldığı en küçük birim	2
symbol rate (baud)	sembol hızı	Saniyedeki sembol sayısı	2
synchronous	senkron	Sürekli akış, çerçeve ve saat kurtarma	6
twisted pair	bükümlü çift	Büküm, girişimi ve çapraz karışmayı azaltır	3
two’s complement	2’ye tümleyen	Ters çevir + 1; tek sıfır; modern bilgisayarların kullandığı	1

Hocaya Sorulacaklar

Slaytları satır satır okurken fark ettiğim tutarsızlıklar ve belirsizlikler. Hocaya e-postayla ya da derste sorabilirsin. Sınavda hangi kuralı kullanman gerektiği için en önemlileri başa aldım.

Sınavı doğrudan etkileyebilecekler

#	Slayt	Sorun	Önerilen soru
1	s. 51 ve s. 53	Manchester yönü ters: s. 51 şekli 1 = yüksek-alçak, s. 53 kutusu 0 = yüksek-alçak	"Sınavda Manchester için hangi konvansiyonu kullanalım? Soruda verilecek mi?"
2	s. 61	İlk örneğin normalize hatası −0.38 yazıyor; −1.50 − (−1.22) = −0.28	"İlk sütundaki hata −0.28 olmalı değil mi? Yazım hatası mı?"
3	s. 51	NRZ-I için bit öncesi seviye ve hangi bitin geçiş yaratacağı net değil; çizim 1 = geçiş, başlangıç yüksek gösteriyor	"NRZ-I'da 1'in geçiş yaratması ve bit öncesi seviye sınavda soruda verilecek mi?"
4	s. 50–53	Unipolar/NRZ-L polaritesi bazı kitaplarda tersi	"NRZ-L'de 1 = yüksek (pozitif) kabulü doğru mu?"
5	s. 52–53	r ve S_{ave} değerleri kutuda var ama türetilmemiş	"Sınavda r, S_{ave} hesaplı çıkar mı, yoksa sadece değerleri bilmek mi yeterli?"
6	s. 60	Nyquist $f_s > 2f_{max}$ yazıyor (çoğu kitap $\geq$)	"Sınavda $>$ mi $\geq$ mı kullanalım?"
7	s. 33	"What are the values as Volts?" sorusunun ölçeği şekilde yok	"Bu etkinliğin cevabı nedir? Basamak başına kaç volt?"

Küçük düzeltmeler

#	Slayt	Sorun
8	s. 15	4 bit 2'ye tümleyen tablosu −7'de bitiyor, −8 (`1000`) satırı yok; alt yazı −8...+7 diyor.
9	s. 31	İkilik 1'in voltajı "Ð5 volts" olarak bozuk karakterle yazılmış; −5 V olmalı.
10	s. 41	Başlık "Twisted-pair cable" ama içerik koaksiyel, Ethernet ve fiber uzunlukları; "amplifier" yerine repeater/switch daha uygun; fiberde ışık boşluktaki hızın yaklaşık 2/3'üyle gider.
11	s. 64	Alt yazı "integrator" diyor, şekilde "Staircase maker" yazıyor. İkisi aynı blok mu?
12	s. 49	Bipolar, çok seviyeli ve çok geçişli kodlar yalnızca adla geçiyor; sınavlık mı?

Genel sınav soruları

13. Sınav tarihi, süresi ve formatı (çoktan seçmeli / klasik / karışık)?
14. Sınav dili: sorular İngilizce mi Türkçe mi, Türkçe cevap yazılabilir mi?
15. Hesap makinesi ya da formül kâğıdı serbest mi?
16. Hangi bölümler en çok puan getirir? (Ben işaretli sayılar, hat kodlama ve PCM'i yüksek öncelikli saydım.)
17. Geçmiş yıl soruları paylaşılabilir mi?

Kaynak Kapsam Kontrolü

D1 = CMP103_05-2.pdf (72 slayt). Her slaytın kitapta nerede işlendiği aşağıda. Kapsanmayan slayt yok; durum sütununda “tam” her slaytın tamamen işlendiği anlamına gelir (işlenemeyen ya da kısmi işlenen slayt çıkmadı). Slayt numaraları slaytın üzerinde basılı numaradır; PDF sayfa numarasıyla aynıdır. Okunurluğu şüpheli noktalar (s. 15, 31, 33, 41, 51, 53, 60, 61, 64) “Hocaya Sorulacaklar” bölümünde listelendi.

Slayt	Konu	Kitapta nerede	Durum
1–3	Kapak, haftanın soruları, öğrenme çıktıları	Başlangıç	tam
4–6	Neden özel gösterim? Üç yaklaşım	Bölüm 1: Eksi işareti için tuş yok	tam
7–9	İşaret-büyüklük	Bölüm 1: İlk akla gelen fikir	tam
10–13	1’e tümleyen	Bölüm 1: Bitleri ters çevir	tam
14–17	2’ye tümleyen kuralı, tablo, onluğa çevirme	Bölüm 1: Ters çevir, 1 ekle	tam
18	Üç gösterimin karşılaştırması	Bölüm 1: Üçünü yan yana koyalım	tam
19	2’ye tümleyen aralığı	Bölüm 1: Ters çevir, 1 ekle (aralık ve asimetri)	tam
20–22	Toplama kuralları, çıkarma = toplama, 58 − 40	Bölüm 1: Çıkarma = toplama	tam
23	Taşma (100 + 60)	Bölüm 1: Taşma	tam
24	Sınıf etkinliği (dört örüntü)	Bölüm 1: Ters çevir, 1 ekle (bir örüntüyü onluğa çevirmek)	tam
25	Özet (checkpoint)	Bölüm 1: Hızlı tekrar kutusu, Sınav Öncesi Hızlı Tekrar	tam
26–27	Bölüm 2 ayracı, haberleşme modeli	Bölüm 2: Haberleşmenin üç parçası	tam
28	Veri ≠ sinyal	Bölüm 2: Veri ≠ sinyal	tam
29	Analog ve sayısal sinyal	Bölüm 2: Analog mu, sayısal mı?	tam
30–32	Ses, bilgisayar verisi, modülasyon	Bölüm 2: Dört dönüşüm	tam

Slayt	Konu	Kitapta nerede	Durum
33–34	Çok seviyeli sinyal, bit hızı ve sembol hızı	Bölüm 2: İki seviyeli ve çok seviyeli sinyaller	tam
35–38	Bölüm 3 ayracı, zayıflama, bozulma, gürültü	Bölüm 3: Yolda ne olur? Üç bozulma	tam
39	SNR sezgisi	Bölüm 3: Sinyal-gürültü oranı	tam
40–43	Ortamlar: bakır, fiber, kablosuz	Bölüm 3: İletim ortamları	tam
44–45	Bölüm 4 ayracı, kodlayıcı-kanal-çözücü	Bölüm 4: Kodlayıcı, kanal, çözücü	tam
46	İyi hat kodu	Bölüm 4: İyi bir hat kodu neye benzer?	tam
47–48	Senkronizasyon ve saat kayması	Bölüm 4: Saatler neden anlaşmak zorunda?	tam
49	Hat kodlama sınıflandırması	Bölüm 4: Hat kodlama ailesi	tam
50	Unipolar NRZ	Bölüm 4: Unipolar NRZ	tam
51	NRZ-L ve NRZ-I	Bölüm 4: Polar NRZ-L ve NRZ-I	tam
52	Polar RZ	Bölüm 4: Polar RZ	tam
53	Manchester, fark Manchester	Bölüm 4: Manchester ve fark Manchester	tam
54	Karşılaştırma tablosu	Bölüm 4: Hepsini yan yana	tam
55	Blok kodlama, karıştırma	Bölüm 4: Hat kodlamanın ötesi	tam
56–57	Bölüm 5 ayracı, neden sayısallaştırıyoruz?	Bölüm 5: Neden sayısallaştırıyoruz?	tam
58	PCM zinciri	Bölüm 5: PCM: üç adımlı zincir	tam
59	Örnekleme türleri	Bölüm 5: Adım 1: Örnekleme	tam
60	Nyquist, örtüşme	Bölüm 5: Adım 1: Örnekleme	tam
61	Nicemleme tablosu	Bölüm 5: Adım 2 ve 3	tam
62	PCM bit hızı	Bölüm 5: PCM bit hızı	tam
63–64	Delta modülasyonu ve modülatör	Bölüm 5: Alternatif: delta modülasyonu	tam
65	PCM ve DM karşılaştırması	Bölüm 5: PCM ile delta modülasyonu karşılaştırması	tam
66–67	Bölüm 6 ayracı, iletim modları ağacı	Bölüm 6: İletim modları ailesi	tam
68	Paralel iletim	Bölüm 6: Paralel iletim	tam
69	Seri iletim	Bölüm 6: Seri iletim	tam

Slayt	Konu	Kitapta nerede	Durum
70	Asenkron, senkron, izokron	Bölüm 6: Seri iletimin üç zamanlama düzeni	tam
71	Haftanın ana fikirleri	Konular Arası Bağlantılar	tam
72	Gelecek hafta	Konular Arası Bağlantılar	tam

Kitapta slayt dışı olan (ek bilgi) ve işaretlenmiş içerik

- 1. bölüm: 2'ye tümleyenin neden çalıştığı (çember), işaret genişletme, donanımda taşma tespiti.
- 2. bölüm: seviyelerin gürültü hesabı, formül türetimi.
- 3. bölüm: desibel, SNR formülü, büküm neden işe yarar, yükselteç–tekrarlayıcı farkı.
- 4. bölüm: 500 bitlik saat kayması hesabı, 4B/5B, S_{ave} genel formülü.
- 5. bölüm: Nyquist örnekleri, 6 dB/bit kuralı, eğim aşırı yüklenmesi koşulu, CD örneği.
- 6. bölüm: skew, asenkron/senkron verim örnekleri, izokron 125 µs örneği.

Slayt dışı her bilgi kitapta **EK BİLGİ (KAYNAK DIŞI)** kutusunda ya da "ek örnek" etiketiyle belirtildi.